

Systems

Once you are authorized to make calls to the various services, one of first things you may want to do is view storage and execution resources available to you or create your own. In Tapis a storage or execution resource is referred to as a **system**.

Overview

A Tapis system represents a server or collection of servers exposed through a single host name or IP address. Each system is associated with a specific tenant. A system can be used for the following purposes:

- Running a job, including:
 - Staging files to a system in preparation for running a job.
 - Executing a job on a system.
 - Archiving files and data on a remote system after job execution.
- Storing and retrieving files and data.

Each system is of a specific type (such as LINUX or S3) and owned by a specific user who has special privileges for the system. The system definition also includes the user that is used to access the system, referred to as *effectiveUserId*. This access user can be a static specific user (such as a service account) or dynamically specified as `${apiUserId}`. For the case of `${apiUserId}`, the username is extracted from the identity associated with the request to the service. For more information related to *effectiveUserId* please see the section below on [Effective User Id and Host Login](#).

Model

At a high level a system represents the following information:

id

A short descriptive name for the system that is unique within the tenant.

description

An optional more verbose description for the system.

systemType - Type of system

LINUX, S3, IRODS or GLOBUS

owner

A specific user set at system creation. By default this is the resolved value for `${apiUserId}`, the user making the request to create the system.

host - Host name, IP address or Globus ID

FQDN, IP address, Globus endpoint ID or Globus collection ID.

enabled - Enabled flag

Indicates if system is currently considered active and available for use. By default this is *true*.

effectiveUserId - Effective User

The username to use when accessing the system. A specific user (such as a service account) or the dynamic user `${apiUserId}`. By default this is `${apiUserId}`. For more information please see the section below on [Effective User Id and Host Login](#).

defaultAuthnMethod - Default authentication method

How access authentication is handled by default. Authentication method can also be specified as part of a request. Supported methods: PASSWORD, PKI_KEYS, ACCESS_KEY, TOKEN, TMS_KEYS.

bucketName - Bucket name

For an S3 system this is the name of the bucket.

rootDir - Effective root directory

Directory to be used when listing files or moving files to and from the system. All paths are relative to this directory when using Files to list, copy, move, mkdir, etc. For more information please see [Effective Root Directory](#). May not be updated. Contact support to request a change.

dtnSystemId - DTN system Id

A system that can be used during job execution as a Data Transfer Node (DTN). Use is optional. The DTN is used if the job submission request or the application definition specify *dtnSystemInputDir* or *dtnSystemOutputDir*. *canExec* must be true. This execution system and the DTN system must have the same *rootDir* and the file system must be shared storage. Please see [DTN Configuration](#).

canExec

Indicates if system can be used to execute jobs.

canRunBatch

Indicates if system supports running jobs using a batch scheduler. By default this is *false*.

enableCmdPrefix

Indicates if system allows a job submission request to specify a *cmdPrefix*. Since *cmdPrefix* is a free form command it is a security concern. By default this is *false*.

allowChildren

Indicates if system supports creating child systems using this system as the parent. By default this is *false*.

Job related attributes

Various attributes related to job execution such as *jobRuntimes*, *jobWorkingDir*, *batchScheduler*, *batchSchedulerProfile*, *batchLogicalQueues*

When creating a system the required attributes are: *id*, *systemType*, *host*, *defaultAuthnMethod* and *canExec*. Depending on the type of system and specific values for certain attributes there are other requirements.

Note that a system may be created as a storage-only resource (*canExec=false*) or as a system that can be used for both execution and storage (*canExec=true*).

Dynamically Determined System Attributes

When fetching a system or list of systems, some attributes are computed dynamically and shown in the results. The dynamically determined attributes are:

effectiveUserId

The resolved username to use when accessing the host or service. If system is defined with a dynamic user, i.e. *effectiveUserId* = *\${apiUserId}*, this is filled in based on the identity of the user making a request. For more information please see the section below on [Effective User Id and Host Login](#) .

isDynamicEffectiveUser

Indicates if *effectiveUserId* was resolved using *\${apiUserId}*.

isPublic

Indicates if the system has been shared publicly. For more information please see the section below on [Sharing](#) .

sharedWithUsers

The list of users with whom the system has been shared.

Getting Started

Before going into further details about Systems, here we give some examples of how to create and view systems. In the examples below we assume you are using the TACC tenant with a base URL of `tacc.tapis.io` and that you have authenticated using PySDK or obtained an authorization token and stored it in the environment variable `JWT`, or perhaps both.

Creating a System

Create a local file named `system_example.json` with json similar to the following:

```
{
  "id": "tacc-sample-<userid>",
  "description": "My storage system",
  "host": "tapis-vm.tacc.utexas.edu",
  "systemType": "LINUX",
  "defaultAuthnMethod": "PKI_KEYS",
  "effectiveUserId": "${apiUserId}",
  "rootDir": "/",
  "canExec": false
}
```

where `<userid>` is replaced with your username, and your host name is updated appropriately. Note that although credentials may be included in the definition we have not done so here. For security reasons, it is recommended that login credentials be updated using a separate API call as discussed below.

Using PySDK:

```
import json
from tapipy.tapis import Tapis
t = Tapis(base_url='https://tacc.tapis.io', username='<userid>',
          password='*****')
with open('system_example.json', 'r') as openfile:
    my_storage_system = json.load(openfile)
t.systems.createSystem(**my_storage_system)
```

Using CURL:

```
$ curl -X POST -H "content-type: application/json" -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/systems -d
@system_example.json
```


Effective Root Directory

Correctly defining the system attribute *rootDir* is critical because it serves as an effective root directory when referencing file paths through the Tapis Files or Jobs services. All paths are relative to this directory when using Files to list, copy, move, mkdir, etc. When creating a system there are certain restrictions for this attribute that should be kept in mind:

- Once a system is created, *rootDir* may not be updated. Contact support to request a change.
- Required for systems of type LINUX or IRODS. Must begin with /.
- Optional for systems of type S3 or GLOBUS.
- For S3 type systems, typically will not begin with /.
 - S3 keys are usually created and manipulated using URLs and do not have a leading /.
- Support is provided for resolving the following variables at create time: *apiUserId*, *tenant* and *owner*.

Use of HOST_EVAL at System Creation

There is also a special macro available for *rootDir* that may be used under certain conditions when a system is first created. The macro name is HOST_EVAL. The syntax for the macro is HOST_EVAL(\$var), where var is the environment variable to be evaluated on the system host when the create request is made. Note that the \$ character preceding the environment variable name is optional. If after resolution the final path does not have the required leading slash (/) to make it an absolute path, then one will be prepended. The following conditions must be met in order to use the macro

- System must be of type LINUX
- Credentials must be provided when system is created.
- Macro HOST_EVAL() must only appear once and must be the first element of the path. Including a leading slash is optional.
- The *effectiveUserId* for the system must be static. Note that *effectiveUserId* may be set to *owner*.

Here are some examples

- HOST_EVAL(\$SCRATCH)
- HOST_EVAL(\$HOME)
- /HOST_EVAL(MY_ROOT_DIR)/scratch
- /HOST_EVAL(\$PROJECT_HOME)/projects/*tenant*/*owner*

Effective User Id and Host Login

The attribute *effectiveUserId* determines the host login user, the user used to access the underlying host or service. The attribute can be set to a static string indicating a specific user (such as a service account) or dynamically specified as *apiUserId*. For the case of *apiUserId*, the service resolves the variable by extracting the

identity from the request to the service (i.e. the JWT) and applying a mapping to a host login user if such a mapping has been provided. If no mapping is provided, then the extracted identity is taken to be the host login user.

Host Login User Mapping

A mapping between a Tapis user and a host login user is created when the system has a dynamic *effectiveUserId* and the attribute *loginUser* is included when registering credentials. Please see the section below on [Registering Credentials for a System](#).

For example, if my user id when logging into Tapis is jdoe and my host login user id is jdoe3, then a login user mapping would be required if the system is defined using a dynamic *effectiveUserId*. Note that if my Tapis user id happened to also be jdoe3 then no mapping would be required.

Please note that if the system is defined using a static *effectiveUserId*, then there is no need for a mapping. In this case the *effectiveUserId* is logically independent of the Tapis identity and may be set to any valid host username value. To this end, TAPIS will reject a credential creation request with a *loginUser* in the request body if the system is already created with a static *effectiveUserId*.

Registering Credentials for a System

Now that you have registered a system you will need to register credentials so you can use Tapis to access the host. Various authentication methods can be used to access a system. Supported methods are PASSWORD, PKI_KEYS, ACCESS_KEY, TOKEN and TMS_KEYS. The process of registering credentials can vary significantly depending on the authentication method. For more information please see the appropriate section below under [Authentication Credentials](#).

Please note that there is support for only one set of credentials per user per system. Updating credentials overwrites previously registered data.

Here we will cover registering PKI_KEYS (i.e. ssh keys) as an example. Please note that registering ssh keys requires special care when translating the generated keypair information to json format. For more information please see [Use of PKI_KEYS as credentials](#) under the section on [Authentication Credentials](#).

Create a local file named `cred_tmp.json` with json similar to the following:

```
{
  "publicKey": "<ssh_public_key>",
  "privateKey": "<ssh_private_key>"
}
```

where `<ssh_public_key>` and `<ssh_private_key>` are replaced with your keys. The keys must be encoded on a single line with embedded newline characters. You may find the following linux command useful in converting a multi-line private key into a single line:

```
cat $privateKeyFile | awk -v ORS='\\n' '1'
```

Using PySDK:

```
t.systems.createUserCredential(systemId='tacc-sample-<userid>',
                             userName='<userid>', publicKey='<ssh_public_key>',
                             privateKey='<ssh_private_key>'))
```

Using CURL:

```
$ curl -X POST -H "content-type: application/json" -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/systems/credential/tacc-sample-<userid>/user/<userid> -d @cred_tmp.json
```

An optional attribute *loginUser* may be included in the request body in order to map the Tapis user to a username to be used when accessing the system. If the login user is not provided then there is no mapping and the Tapis user is always used when accessing the system. When a *loginUser* is provided the json would be similar to the following:

```
{
  "publicKey": "<ssh_public_key>",
  "privateKey": "<ssh_private_key>",
  "loginUser": "<linux_host_username>"
}
```

Note that credentials are stored in the Security Kernel. Only specific Tapis services are authorized to retrieve credentials.

Viewing Systems

Retrieving details for a system

To retrieve details for a specific system, such as the one above:

Note

See the section below on [Selecting](#) to find out how to control the amount of information returned.

Using PySDK:

```
t.systems.getSystem(systemId='tacc-sample-<userid>')
```

Using CURL:

```
$ curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/systems/tacc-sample-<userid>
```

The response should look similar to the following:

```
{
  "result": {
    "tenant": "dev",
    "id": "tacc-sample-<userid>",
  }
}
```

```

    "description": "My storage system",
    "systemType": "LINUX",
    "owner": "<userid>",
    "host": "tapis-vm.tacc.utexas.edu",
    "enabled": true,
    "effectiveUserId": "<userid>",
    "defaultAuthnMethod": "PKI_KEYS",
    "authnCredential": null,
    "rootDir": "/",
    "port": 22,
    "useProxy": false,
    "proxyHost": "",
    "proxyPort": -1,
    "dtnSystemId": null,
    "canExec": false,
    "canRunBatch": false,
    "enableCmdPrefix": false,
    "allowChildren": false,
    "jobRuntimes": [],
    "jobWorkingDir": null,
    "jobEnvVariables": [],
    "jobMaxJobs": 2147483647,
    "jobMaxJobsPerUser": 2147483647,
    "batchScheduler": null,
    "batchSchedulerProfile": null,
    "batchLogicalQueues": [],
    "batchDefaultLogicalQueue": null,
    "jobCapabilities": [],
    "tags": [],
    "notes": {},
    "uuid": "f83606bf-7a1a-4ff0-9953-dd732cc07ac0",
    "deleted": false,
    "created": "2021-04-26T18:45:40.771Z",
    "updated": "2021-04-26T18:45:40.771Z"
  },
  "status": "success",
  "message": "TAPIS_FOUND System found: tacc-sample-<userid>",
  "version": "0.0.1",
  "metadata": null
}

```

Note that `authnCredential` is *null*. Only specific Tapis services are authorized to retrieve credentials.

Retrieving details for all systems

To see the list of systems that you own:

Using PySDK:

```
t.systems.getSystems()
```

Using CURL:

```
$ curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/systems?select=allAttributes
```

The response should contain a list of items similar to the single listing shown above.

Note

See the sections below on [Searching](#), [Selecting](#), [Sorting](#) and [Limiting](#) to find out how to control the amount of information returned.

Child Systems

Creating Child Systems

A system that has *allowChildren* set to *true* allows for creation of child systems based on it. This ability provides a way to easily clone and manage systems based on existing systems. Child systems allow a user to set only a few fields, and use all other values from an existing parent system. This can reduce the difficulty in managing systems. It allows for all child systems to be updated when the parent is updated.

To create a child system, first ensure that the system intended to serve as the parent as *allowChildren* set to *true*. Next, create a local file (for example `child_system_example.json`) similar to the following:

```
{
  "id": "my-child-<userid>",
  "effectiveUserId": "${apiUserId}",
  "rootDir": "/home/<userid>"
}
```

Where *<userid>* is replaced with your username. Also ensure that the root directory path is correct. Now use the create child system REST endpoint to create the child system. Let's assume that the new child system will be a child of a parent system called *parent-system*.

Using PySDK:

```
import json
from tapipy.tapis import Tapis
t = Tapis(base_url='https://tacc.tapis.io', username='<userid>',
password='*****')
with open('child_system_example.json', 'r') as openfile:
    child_system = json.load(openfile)
t.systems.createChildSystem(parentId="parent-system",
**child_system)
```

Using CURL:

```
$ curl -X POST -H "content-type: application/json" -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/systems/parent-system/createChildSystem -d @child_system_example.json
```

These fields are maintained independently for child systems:

Attribute	Type	Example	Notes
id	String	ds1.storage.default	• Identifier for the system. URI safe, see RFC 3986. • <i>tenant</i> + <i>id</i> must be unique. • Allowed characters: Alphanumeric [0-9a-zA-Z] and special characters [-. _~].
owner	String	jdoe	• username of <i>owner</i>. • Variable references: <i>\${apiUserId}</i>. Resolved at create time. • By default this is the resolved value for <i>\${apiUserId}</i>.
enabled	boolean	FALSE	• Indicates if system currently enabled for use. • May be updated using the enable/disable endpoints. • By default this is <i>true</i>.
effectiveUserId	String	tg869834	• User to use when accessing the system. • May be a static string or a variable reference. • Variable references: <i>\${apiUserId}</i>, <i>\${owner}</i>

Attribute	Type	Example	Notes
rootDir	String	/home/\${apiUserId}	• On output variable reference will be resolved. • Required if <i>systemType</i> is LINUX or IRODS. • For LINUX or IRODS must begin with /. • Optional for S3 and GLOBUS. For S3 will typically not begin with /. • Variable references are resolved at create time. • Serves as effective root directory when listing or moving files. • May not be updated. Contact support to request a change. • Variable references: <i>\$</i> <i>{apiUserId}</i>, <i>\$</i> <i>{owner}</i>, <i>\$</i> <i>{tenant}</i>
deleted	boolean	FALSE	• Indicates if system has been deleted. • May be updated using the delete/undelete endpoints.
created	Timestamp	2020-06-19T15:10:43Z	• When the system was created. Maintained by service.
updated	Timestamp	2020-07-04T23:21:22Z	• When the system was last updated. Maintained by service.

During the creation of a child system, any of these fields may be specified except for created, updated and deleted. All other fields are taken from the parent system.

Updating a Child System

Updates are done just like any other system, however, only the following fields may be updated for a child system.

Attribute	Type	Example	Notes
effectiveUserId	String	tg869834	• User to use when accessing the system. • May be a static string or a variable reference. • Variable references: $\\${apiUserId}$, $\\${owner}$ • On output variable reference will be resolved.

Some other fields can be updated through special endpoints. For example deleted and enabled are updated through the endpoints for deleting, undeleting, enabling and disabling.

Child System Operations

Most operations other than update are the same for child systems as they are for parent systems. For more information see the appropriate section of the document for the operation.

- Delete - see [Deletion](#)
- Undelete - see [Deletion](#)
- Enable - see "enabled" in [System Attributes Table](#)
- Disable - see "enabled" in [System Attributes Table](#)

Unlinking a Child System from it's Parent System

A child system may be unlinked from it's parent. This is a permanent operation, and cannot be undone. This will make the child a standalone system with all of it's current settings. When the unlink happens any fields that had previously been linked to the parent will be copied to the child, and it will be as if the child was created as in independent system with those values.

If the owner of the child system wants to unlink the child from it's parent, the owner may use the *unlinkFromParent* endpoint.

Using PySDK:

```
import json
from tapipy.tapis import Tapis
t = Tapis(base_url='https://tacc.tapis.io', username='<userid>',
```



```
password='*****')
t.systems.unlinkFromParent(childSystemId="<child-system-id>")
```

Using CURL:

```
$ curl -X POST -H "content-type: application/json" -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/systems/<child-system-id>/unlinkFromParent
```

Replace *<child-system-id>* with the id of the child system.

The owner of a parent system can also decide to unlink child systems from the parent. In that case the parent system owner would use the *unlinkChildren* endpoint. The child systems to unlink may be specified in the request body. First create a json file (for example *children_to_unlink.json*):

```
{
  "childSystemIds":
  [
    "<child-system-1-id>",
    "<child-system-2-id>"
    ...
  ]
}
```

Using PySDK:

```
import json
from tapipy.tapis import Tapis
t = Tapis(base_url='https://tacc.tapis.io', username='<userid>',
password='*****')
with open('children_to_unlink.json', 'r') as openfile:
    children_to_unlink = json.load(openfile)
t.systems.unlinkChildren(parentSystemId="<parent-system-id>",
**children_to_unlink)
```

Using CURL:

```
$curl -X POST -H "content-type: application/json" -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/systems/<parent-system-id>/unlinkChildren -d @./children_to_unlink.json
```

Or all child systems using *all=True* (no json file required)

Using PySDK:

```
import json
from tapipy.tapis import Tapis
t = Tapis(base_url='https://tacc.tapis.io', username='<userid>',
password='*****')
t.systems.unlinkChildren(parentSystemId="<parent-system-id>",
all=True)
```

Using CURL:

```
$ curl -X POST -H "content-type: application/json" -H "X-Tapis-Token: $JWT" "https://tacc.tapis.io/v3/systems/<parent-system-id>/unlinkChildren?all=true"
```

Minimal Definition and Restrictions

When creating a system the required attributes are: *id*, *systemType*, *host*, *defaultAuthnMethod* and *canExec*. Depending on the type of system and specific values for certain attributes there are other requirements. The restrictions are:

- If *systemType* is S3 then *bucketName* is required and *canExec* must be false.
- If *systemType* is LINUX or IRODS then *rootDir* is required and must begin with */*.
- If *effectiveUserId* is *#{apiUserId}* (i.e. it is not static) then *authnCredential* may not be specified.
- If *canExec* is true then *jobWorkingDir* is required and *jobRuntimes* must have at least one entry.
- If *canRunBatch* is true then *batchScheduler* must be specified.
- If *canRunBatch* is true then *batchLogicalQueues* must have at least one item.
 - If *batchLogicalQueues* has more than one item then *batchLogicalDefaultQueue* must be specified.
 - If *batchLogicalQueues* has exactly one item then *batchLogicalDefaultQueue* is set to that item.

Permissions

The permissions model allows for fine grained access control of Tapis systems.

At system creation time the owner is given full access to the system. Permissions for other users may be granted and revoked through the systems API. Please note that grants and revokes through this service only impact the default role for the user. A user may still have access through permissions in another role. So even after revoking permissions through this service, when permissions are retrieved the access may still be listed. This indicates access has been granted via another role.

Permissions are specified as either *** for all permissions or some combination of the following specific permissions: (*"READ"*, *"MODIFY"*, *"EXECUTE"*). Specifying permissions in all lower case is also allowed. Having *MODIFY* implies *READ*.

Sharing

In addition to fine grained permissions support, Tapis also supports a higher level approach to granting access. This approach is known simply as *sharing*. The sharing API allows you to share a system with a set of users as well as share publicly with

all users in a tenant. Sharing provides READ+EXECUTE access. When the system has a dynamic *effectiveUserId*, sharing also allows for MODIFY access to all file paths for calls made through the Files service.

Note

Note that there is one other case when a system is treated as having a dynamic *effectiveUserId* in the context of sharing, even with a static *effectiveUserId*. This is when the system type is IRODS and the attribute *useProxy* is set to `true`. In this case the connection to the IRODS host is made using a special administrative account which then acts as the Tapis user. So please be aware that for this type of system sharing the system or a file path will allow for MODIFY access to files paths on that system.

The most common use case for sharing a system is to publicly share the system with all users in the tenant. This would allow any user to use the system for execution or storage when running an application.

Note

If a system has a dynamic *effectiveUserId* and has been shared publicly or with specific users, then those users will have Tapis permissions to operate on any files without explicitly sharing any paths through the Files service. This includes file listing, upload, download and delete.

If a system has a static *effectiveUserId*, then file paths will need to be explicitly shared using the Files service in order to allow users READ access. Having READ access allows users to list and download files.

These restrictions are in place in order to reduce the risks associated with sharing a system. With a dynamic *effectiveUserId* users are always logging in to the host as themselves. With a static *effectiveUserId* there is a privilege escalation security risk.

Warning

In the context of using a shared application to run a job, sharing a system (and hence all file paths on the system) will grant users READ and MODIFY access to the file paths, even for the case of a static *effectiveUserId*.

Note

Tapis permissions and sharing are independent of native permissions enforced by the underlying system host.

For more information on sharing please see [sharing](#)

Authentication Credentials

At system creation time the authentication credentials may be specified if the effective access user *effectiveUserId* is a specific user (such as a service account) and not a dynamic user (i.e. not equal to `${apiUserId}`).

If the effective access user is dynamic (i.e. equal to `${apiUserId}`) then authentication credentials for any user allowed to access the system must be registered in separate API calls. In this case the payload provided may contain the optional attribute `loginUser` which will be used to map the Tapis user to a username to be used when accessing the system. If the login user is not provided then there is no mapping and the Tapis user is always used when accessing the system.

Note that the Systems service does not store credentials. Credentials are persisted by the Security Kernel service and only specific Tapis services are authorized to retrieve credentials.

Also, note that there is support for only one set of credentials per user per system. Updating credentials overwrites previously registered data.

By default any credentials provided for LINUX and S3 type systems are verified. The query parameter `skipCredentialCheck=true` may be used to bypass the initial verification of credentials.

Use of PKI_KEYS as credentials

When using an ssh keypair as credentials there are several important points to keep in mind. As discussed above, the public key and private key must be encoded on a single line. This can sometimes be challenging. For example, copying and pasting may convert newline characters in a way that is not compatible with processing in Tapis. You may find the following linux command useful in converting a multi-line private key into a single line:

```
cat $privateKeyFile | awk -v ORS='\\n' '1'
```

When generating the keypair, do not use a passphrase. This can interfere with non-interactive use of the keypair.

Finally, please be aware that if the host has multi-factor authentication (MFA) enabled this may prevent Tapis from communicating with the host. Tapis does not currently support MFA.

If problems are encountered here are some suggestions on what to check:

- Public and private keys are each on one line in the json file. Newline characters in private key are properly encoded.
- Keypair does not have a passphrase
- Public key has been added to the `authorized_keys` file for the target user. File `~/.ssh/authorized_keys`
- File `~/.ssh/authorized_keys` has proper permissions.
- MFA is not enabled for the target host.

If problems persist you can also attempt to manually validate the keypair using a command similar to this:

```
ssh -i /tmp/my_private_key testuser@myhost.com
```

where `/tmp/my_private_key` contains the original multi-line private key. If everything is set up correctly and the keypair is valid you should be logged into the host without being prompted for a password.

Use of TMS_KEYS for credentials

Tapis supports the use of the Trust Manager System (TMS) for managing credentials. For more information on TMS refer to [TMS_Documentation](#).

Please note that your Tapis site installation must have been configured by the site administrator to support TMS. See [TMS_Config](#).

Also, any target hosts defined in Tapis systems must be configured to use the TMS KeyCmd program for ssh connections. Please refer to the TMS documentation for details.

The integration of Tapis with TMS allows users to have Tapis automatically create and use SSH keypairs rather than having to provide their own. In order to register TMS credentials for a system, begin by making sure the system is defined with *defaultAuthnMethod* set to TMS_KEYS. Then when creating credentials simply add the flag `createTmsKeys=true`.

For example, the CURL command to create TMS keys for a system might look as follows:

```
$ curl -X POST -H "content-type: application/json" -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/systems/credential/tms-test/user/<userid>?createTmsKeys=true -d @cred_tmp.json
```

Note that the request body may be empty.

Please note that the following restrictions apply:

- Tapis installation for your site must be configured to support the Trust Manager System (TMS).
- The host for the system must have the sshd configuration set up to use TMS.
- The *effectiveUserId* must be dynamic.
- Mapping of user using *loginUser* is not supported.

Registering Credentials for a Globus System

Registering credentials for a GLOBUS type system is a special case that involves multiple steps and is significantly different compared to registering other types of credentials. For a GLOBUS type system, the user will need to use the TOKEN authentication method and generate an `accessToken` and `refreshToken` using two special-purpose System service endpoints.

Please note that your Tapis site installation must have been configured by the site administrator to support Globus. See [Globus_Config](#).

Obtain Globus Authorization Code

The first step in generating Globus credentials is for the user to call the systems *authUrl* credential endpoint to obtain a Globus authorization code.

Using CURL, the request would look something like this:

```
$curl -H "X-Tapis-Token: $JWT" https://dev.tapis.io/v3/systems/credential/<system>/globus/authUrl
```

The response should look similar to the following. Note that for brevity and readability, only the result portion of the response is shown, the response has been split into multiple lines and various IDs are not filled in:

```
{
  "url": "https://auth.globus.org/v2/oauth2/authorize?
client_id=<client_id>

&redirect_uri=https%3A%2F%2Fauth.globus.org%2Fv2%2Fweb%2Fauth-code

&scope=openid+profile+email+urn%3Aglobus%3Aauth%3Ascope%3Atransfer.

&state=_default&response_type=code&code_challenge=<challenge_id>
    &code_challenge_method=S256&access_type=offline",
  "sessionId": "<session_id>"
}
```

The user should copy the url (as a single string, no line breaks) and make note of the session Id for later use. The user then visits the provided URL and is presented with a Globus logon page that will allow them to authenticate using one of thousands of supported identity providers, including through their existing organization using CILogon.

The user must use the following flow to obtain an authorization code:

1. Visit the provided URL and authenticate through Globus. After authentication, user is re-directed back to a Globus page showing the access being requested by Tapis.
2. Fill in a label for future reference and click *Allow* to authorize Tapis to access Globus on their behalf.
3. Copy the provided authorization code in preparation for the final step. Note that the code is valid for a short time (as of this writing it is valid for 10 minutes).

Exchange Authorization Code for Tokens

The final step is for the user to call the systems credential endpoint to exchange the authorization code and session ID for tokens which are stored by the Systems service in a credentials record.

Using CURL, the request would look something like this:

```
$curl -X POST -H "content-type: application/json" -H "X-Tapis-Token: $JWT"
```

```
https://dev.tapis.io/v3/systems/credential/<system>/user/  
<user>/globus/tokens/<authCode>/<sessionId>
```

The response should look similar to the following:

```
{  
  "result": null,  
  "status": "success",  
  "message": "SYSAPI_CRED_UPDATED Credential updated. ...",  
  "version": "1.3.1",  
  "commit": "619aa7ce",  
  "build": "2023-04-02T19:06:38Z",  
  "metadata": null  
}
```

At this point the user will have registered credentials for a Tapis system that can be used as a source or destination for Globus operations.

Registering Credentials for an AWS S3 System

Registering credentials for a Tapis S3 type system referencing an AWS bucket is a special case that is significantly different compared to registering other types of credentials. For such a system, the user will need to first manage the S3 bucket and IAM access tokens within AWS, then register the AWS access token with Tapis as a key pair.

Creating the S3 Bucket within AWS

The first step in registering S3 as a storage solution with Tapis is creating the S3 bucket. Tapis does not require any special considerations to be taken when creating the bucket, but you will need to keep track of the bucket's name.

Obtaining an Access Key Pair from AWS

Although creating an IAM user is not strictly necessary, it is highly recommended. You can create an access token for the root user by going to 'my security credentials' in the IAM console in AWS but this is not recommended for security reasons. Instead, it is recommended to create an IAM user for Tapis access. Make sure that when you create the user, you explicitly give it permissions to access S3. Refer to the [IAM documentation](#) for instructions on assigning permissions.

Take the following steps to obtain an access token pair for an IAM user:

1. In the IAM console in AWS, select the user who will be registered in the Tapis system.
2. Navigate to the 'Security Credentials' tab, and select 'Create access key'.
3. Go through the creation wizard, making sure that you save both the access key and the secret access key on the last page. You will need both of these in the next step to register the credentials with Tapis.

Create the S3 System in Tapis

Now that you have everything you need from AWS, create the Tapis system. Refer to the [Creating a System](#) section for instructions, making sure to use S3 for `systemType` and the name of the bucket for `host`.

The system definition should look something like this:

```
{
  "id": "demo-aws-s3-bucket",
  "description": "Demo S3 AWS acct.",
  "host": "tapisdemo-bucket.s3.amazonaws.com",
  "systemType": "S3",
  "effectiveUserId": "tapisdemo",
  "defaultAuthnMethod": "ACCESS_KEY",
  "bucketName": "tapisdemo-bucket",
  "rootDir": "",
  "canExec": false
}
```

Registering AWS Credentials with Tapis

Now that we have a bucket, AWS tokens, and a Tapis system, the last step is to register the AWS credentials with Tapis.

using CURL, we can do something like:

```
$curl -X POST -H "content-type:applications/json" -H "X-Tapis-Token: $JWT" \
  -d {
    "accessKey": "< AWS access key name >",
    "accessSecret": "< AWS access key secret >"
  } \
  https://dev.tapis.io/v3/systems/credential/<system>/user/<user>
```

Which should return a response similar to :

```
{
  "result": null,
  "status": "success",
  "message": "SYSAPI_CRED_UPDATED Credential updated. ...",
  "version": "1.8.2",
  "commit": "9e30ecbf",
  "build": "2025-02-25T14:24:36Z",
  "metadata": null
}
```


Runtime

Runtime environment supported by the system that may be used to run applications, such as docker, singularity or ZIP. Consists of the runtime type and version.

Logical Batch Queue

A queue that maps to a single HPC queue. Logical batch queues provide a uniform front end abstraction for an HPC queue. They also provide more features and flexibility than is typically provided by an HPC scheduler. Multiple logical queues may be defined for each HPC queue. If an HPC queue does not have a corresponding logical queue defined then a user will not be able use the Tapis system to directly submit a job via Tapis to that HPC queue.

Batch Scheduler Profile

The Systems service supports managing Tapis scheduler profiles. An HPC center often has certain conventions and restrictions around the use of batch schedulers. A scheduler profile resource can be defined to provide the Tapis Jobs service with additional site specific information to be used when executing applications using a scheduler. A scheduler profile contains information on options that should be hidden from the scheduler, the module load command to use and which modules should be loaded by default when running a job. Anyone in a tenant may create a scheduler profile for use by all users in the tenant. The owner of a profile or a tenant administrator may modify or delete a profile. A profile may be referenced in a system definition using the attribute *batchSchedulerProfile*. The profile to be used may also be set in the job submit request using the special scheduler option *--tapis-profile*. The value in the job submit request takes precedence over a value defined for the execution system.

For example, at TACC there is a certain module that must be loaded when running Slurm jobs via singularity. Also, use of the Slurm option *--mem* is prohibited. In support of this, most of the tenants at TACC make use of a profile similar to the following:

```
{
  "name": "tacc",
  "owner": "testuser1",
  "description": "Profile for TACC Slurm",
  "moduleLoads": [
    {
      "moduleLoadCommand": "module load",
      "modulesToLoad": ["tacc-singularity"]
    }
  ],
  "hiddenOptions": ["MEM"]
}
```

The *moduleLoads* array contains one or more entries. Each entry contains a *moduleLoadCommand*, which specifies the local command used to load each of the modules (in order) in its *modulesToLoad* list.

The *hiddenOptions* array identifies scheduler options that the local implementation prohibits. Options specified here will have the corresponding Slurm option suppressed. Supported options are "MEM" for *--mem* and "PARTITION" for *--partition*. Including an option in the array indicates that the corresponding Slurm option should never be passed through to Slurm.

Deletion

A system may be deleted and undeleted. Deletion means the system is marked as deleted and is no longer available for use. By default deleted systems will not be included in searches and operations on deleted systems will not be allowed. When listing systems the query parameter *showDeleted* may be used in order to include deleted systems in the results.

System Attributes Table

Attribute	Type	Example	Notes
tenant	String	designsafe	• Name of the tenant for which the system is defined. • <i>tenant + id</i> must be unique. • Determined by the service at system creation time.
id	String	ds1.storage.default	• Identifier for the system. URI safe, see RFC 3986. • <i>tenant + id</i> must be unique. • Allowed characters: Alphanumeric [0-9a-zA-Z] and special characters [-._~].
description	String	Default storage	• Description
systemType	enum	LINUX	• Type of system.

Attribute	Type	Example	Notes
			Types: LINUX, S3, IRODS, GLOBUS
owner	String	jdoe	- username of <i>owner</i>. - Variable references: <i>\$ {apiUserId}</i>. Resolved at create time. - By default this is the resolved value for <i>\$ {apiUserId}</i>.
host	String	data.tacc.utexas.edu	Host name, ip address, Globus endpoint ID or Globus collection ID.
enabled	boolean	FALSE	- Indicates if system currently enabled for use. - May be updated using the enable/disable endpoints. - By default this is <i>true</i>.
effectiveUserId	String	tg869834	- User to use when accessing the system. - May be a static string or a variable reference. - Variable references: <i>\$ {apiUserId}</i>, <i>\${owner}</i> - On output variable reference will be resolved.
defaultAuthnMethod	enum	PKI_KEYS	- How access authentication is handled by default. - Can be overridden as part of a request to get a system or credential. - Methods: PASSWORD, PKI_KEYS, ACCESS_KEY, TOKEN, TMS_KEYS - See table <i>Credential Attributes</i> below for more information.
authnCredntial	Credential		

Attribute	Type	Example	Notes
			• On input credentials to be stored in Security Kernel. • <i>effectiveUserId</i> must be static, either a string constant or <i>\${owner}</i>. • May not be specified if <i>effectiveUserId</i> is dynamic, i.e. <i>\${apiUserId}</i>. • On output contains credential for <i>effectiveUserId</i> and requested <i>authnMethod</i>. • Returned credential contains relevant information based on <i>authnMethod</i>. • Credentials may be updated using the systems credentials endpoint. • By default for LINUX the credentials are verified during create or update. • Use query parameter <i>skipCredentialCheck=true</i> to bypass initial verification. • See table <i>Credential Attributes</i> below for more information.
bucketName	String	tapis-ds1-jdoe	• Name of bucket for an S3 system. • Required if <i>systemType</i> is S3. • Variable references: <i>\${apiUserId}</i>, <i>\${owner}</i>, <i>\${tenant}</i>
rootDir	String	/home/\${apiUserId}	• Required if <i>systemType</i> is LINUX or IRODS. • For LINUX or IRODS must begin with <i>/</i>. • Optional for S3 and GLOBUS. For S3 will typically not begin with <i>/</i>. • Variable references are resolved at create time.

Attribute	Type	Example	Notes
			• Serves as effective root directory when listing or moving files. • May not be updated. Contact support to request a change. • Variable references: <i>\$ {apiUserId}</i>, <i>\$ {owner}</i>, <i>\$ {tenant}</i>
port	int	22	• Port number used to access the system
useProxy	boolean	TRUE	• Indicates if system should be accessed through a proxy. • Currently only supported for IRODS type systems. • Indicates if an IRODS proxy administrative user should be used. • <i>effectiveUserId</i> is the IRODS proxy admin user. • Tapis user making the request is the IRODS user who will be impersonated. • For this case <i>effectiveUserId</i> is considered dynamic in context of sharing.
proxyHost	String		• Name of proxy host. • Not currently supported. Please contact support if needed.
proxyPort	int		• Port number for <i>proxyHost</i> • Not currently supported. Please contact support if needed.
dtnSystemId	String	default.corral.dtn	• A system that can be used as a Data Transfer Node (DTN). Use is optional.

Attribute	Type	Example	Notes
			• This system and <i>dtnSystemId</i> must have the same <i>rootDir</i> and shared storage. • Used if job submission or application specify a DTN input or output directory.
canExec	boolean		• Indicates if system will be used to execute jobs.
canRunBatch	boolean		• Indicates if system supports running jobs using a batch scheduler. • By default this is <i>false</i>.
enableCmdPrefix	boolean		• Indicates if system allows a job submission request to specify a <i>cmdPrefix</i>. • By default this is <i>false</i>.
allowChildren	boolean		• Indicates if system supports creating child systems using this system as parent. • By default this is <i>false</i>.
jobRuntimes	[Runtime]		• List of runtime environments supported by the system. • At least one entry required if <i>canExec</i> is true. • Each Runtime specifies the Runtime type and version • Runtime type is required and must be one of: DOCKER, SINGULARITY, ZIP. • Runtime version is optional.
jobWorkingDir	String	workdir	• Parent directory from which a job is run.

Attribute	Type	Example	Notes
			• Relative to the effective root directory <i>rootDir</i>. • Required if <i>canExec</i> is true. • Variable references: <i>\${apiUserId}</i>, <i>\${owner}</i>, <i>\${tenant}</i>
jobEnvVariables	[KeyValuePair]		• Environment variables added to the shell environment in which the job is running. • Added to environment variables specified in job and application definitions. • Each entry has <i>key</i> (required) and <i>value</i> (optional) as well as other attributes. • See table <i>KeyValuePair Attributes</i> below for more information.
jobMaxJobs	int		• Max total number of jobs . • Set to -1 for unlimited.
jobMaxJobsPerUser	int		• Max total number of jobs associated with a specific user. • Set to -1 for unlimited.
batchScheduler	String	SLURM	• Type of scheduler used when running batch jobs. • Schedulers: SLURM
batchSchedulerProfile	String		• Default Tapis scheduler profile for batch jobs.
batchLogicalQueues	[LogicalQueue]		• List of logical queues available on the system. • Each logical queue maps to a single HPC queue.

Attribute	Type	Example	Notes
			- Multiple logical queues may be defined for each HPC queue. - See table <i>LogicalQueue Attributes</i> below for more information.
batchDefaultLogicalQueue	LogicalQueue		Default logical batch queue for the system.
tags	[String]		List of tags as simple strings.
notes	String	"{"	- Simple metadata in the form of a Json object. - Not used by Tapis.
uuid	UUID		Auto-generated by service.
deleted	boolean	FALSE	- Indicates if system has been deleted. - May be updated using the delete/undelete endpoints.
created	Timestamp	2020-06-19T15:10:43Z	When the system was created. Maintained by service.
updated	Timestamp	2020-07-04T23:21:22Z	When the system was last updated. Maintained by service.

Credential Attributes Table

Attribute	Type	Example	Notes
user	String	jsmith	Username associated with the credential.

Attribute	Type	Example	Notes
authnMethod	String	PKI_KEYS	- Indicates the authentication method associated with a retrieved credential. - When a credential is retrieved it is for a specific authentication method. - Methods: PASSWORD, PKI_KEYS, ACCESS_KEY, TOKEN, TMS_KEYS
loginUser	String		- Optional native username valid on the system. - May be used to map a Tapis user to a native login user.
password	String		Password for when authnMethod is PASSWORD. For LINUX and IRODS systems.
privateKey	String		Private key for when authnMethod is PKI_KEYS. For LINUX systems.
publicKey	String		Public key for when authnMethod is PKI_KEYS. For LINUX systems.
accessKey	String		Access key for when authnMethod is ACCESS_KEY. For S3 systems.
accessSecret	String		Access secret for when authnMethod is ACCESS_KEY. For S3 systems.
accessToken	String		Access token for when authnMethod is TOKEN. For GLOBUS systems.
refreshToken	String		Refresh token for when authnMethod is TOKEN. For GLOBUS systems.
tmsPrivateKey	String		

Attribute	Type	Example	Notes
			Private key for when authnMethod is TMS_KEYS. For LINUX systems.
tmsPublicKey	String		Public key for when authnMethod is TMS_KEYS. For LINUX systems.
tmsFingerprint	String		Fingerprint of public key for when authnMethod is TMS_KEYS. For LINUX systems.

KeyValuePair Attributes Table

Attribute	Type	Example	Notes
key	String	"INPUT_FILE"	Environment variable name. Required.
value	String	"/tmp/file.input"	Environment variable value
description	String		Description
inputMode	enum	REQUIRED	- Indicates how argument is to be treated when processing requests. - Modes: REQUIRED, FIXED, INCLUDE_ON_DEMAND, INCLUDE_BY_DEFAULT - Default is INCLUDE_BY_DEFAULT. ***** - REQUIRED: Must be provided in a job request or a job request. - FIXED: Not overridable in application or job request. - INCLUDE_ON_DEMAND: Included if referenced in a job request. - INCLUDE_BY_DEFAULT: Included unless <i>includeOnDemand</i> is set in a job request.
notes	String	"{}"	- Simple metadata in the form of a Json object. - Not used by Tapis.

LogicalQueue Attributes Table

Attribute	Type	Example	Notes
name	String	tapisNormal	• Name for logical queue. Typically will match or be a variant of HPC queue name.
hpcQueueName	String	normal	• Name of the HPC queue for which this logical queue is a front end.
maxJobs	int		• Maximum total number of jobs that can be queued or running in this queue.
maxJobsPerUser	int		• Maximum number of jobs associated with a specific user that can be queued.
minNodeCount	int		• Minimum number of nodes that can be requested when submitting a job to the queue.
maxNodeCount	int		• Maximum number of nodes that can be requested when submitting a job to the queue.
minCoresPerNode	int		• Minimum number of cores per node that can be requested when submitting a job. • Default is 1
maxCoresPerNode	int		• Maximum number of cores per node that can

Attribute	Type	Example	Notes
			be requested when submitting a job.
minMemoryMB	int		• Minimum memory in megabytes that can be requested when submitting a job. • Default is 0
maxMemoryMB	int		• Maximum memory in megabytes that can be requested when submitting a job. • Default is unlimited
minMinutes	int		• Minimum run time in minutes that can be requested when submitting a job. • Default is 0
maxMinutes	int		• Maximum run time in minutes that can be requested when submitting a job. • Default is unlimited
schedulerOptions	[Arg]		• Scheduler options passed to the HPC batch scheduler. • See table below for more information on <i>Arg</i> type entries. • For more information on <i>schedulerOptions</i> please see the chapter on <i>Jobs</i>

Arg Attributes Table

For more information on how the *inputMode* attribute is used during job processing, please see [JobsArgProcessing](#).

Attribute	Type	Example	Notes
name	String		- Identifying label associated with the argument. Required at creation time.
description	String		Optional description of the argument which may include usage, etc.
inputMode	enum		- Indicates how argument is to be treated when processing incoming requests. - Modes: REQUIRED, FIXED, INCLUDE_ON_DEMAND, INCLUDE_BY_DEFAULT - Default is INCLUDE_BY_DEFAULT. ***** - REQUIRED: Must be provided in a job request. - FIXED: Completely defined in the application and not overridden in request. - INCLUDE_ON_DEMAND: Included if referenced in a job request. - INCLUDE_BY_DEFAULT: Included unless <i>include=false</i> is specified in request.
arg	String		- Value for the argument Required at creation time.
notes	String	{"fieldType": "int"}	- Metadata in the form of a Json object, such as type, allowed values, etc. - Not used by Tapis.

Scheduler Profile Attributes Table

Attribute	Type	Example	Notes
name	String	tacc	Name. Required. <i>tenant + name</i> uniquely identifies the profile.
owner	String	jdoe	Tapis user that created the profile.
description	String		Description
moduleLoads	Array		

Attribute	Type	Example	Notes
			Each entry contains a module command and a list of modules to load.
hiddenOptions	String[]	["MEM"]	- List of locally prohibited scheduler options that should be filtered out. - Allowed values: MEM, PARTITION

Searching

The service provides a way for users to search for systems based on a list of search conditions provided either as query parameters for a GET call or a list of conditions in a request body for a POST call to a dedicated search endpoint.

Search using GET

To search when using a GET request to the systems endpoint a list of search conditions may be specified using a query parameter named `search`. Each search condition must be surrounded with parentheses, have three parts separated by the character `.` and be joined using the character `~`. All conditions are combined using logical AND. The general form for specifying the query parameter is as follows:

```
?
search=(<attribute_1>.<op_1>.<value_1>)~(<attribute_2>.<op_2>.<value_2>)~(<attribute_N>.<op_N>.<value_N>)
```

Attribute names are given in the table above and may be specified using Camel Case or Snake Case.

Supported operators: `eq neq gt gte lt lte in nin like nlike between nbetween`

Example CURL command to search for systems that have `Test` in the `id`, are of type `LINUX`, are using a port less than `1024` and have a default authentication method of either `PKI_KEYS` or `PASSWORD`:

```
$ curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/systems?
search="(id.like.*Test*)~(system_type.eq.LINUX)~(port.lt.1024)~(Def
```

Notes:

- For the `like` and `nlike` operators the wildcard character `*` matches zero or more characters and `!` matches exactly one character.
- For the `between` and `nbetween` operators the value must be a two item comma separated list of unquoted values.

- If there is only one condition the surrounding parentheses are optional.
- In a shell environment the character & separating query parameters must be escaped with a backslash.
- In a shell environment the query value must be surrounded by double quotes and the following characters must be escaped with a backslash in order to be properly interpreted by the shell:

◦ " \ `

- Attribute names may be specified using Camel Case or Snake Case.
- Following complex attributes not supported when searching:

```
◦ authnCredential jobRuntimes jobEnvVariables
  batchLogicalQueues notes
```

Dedicated Search Endpoint

The service provides the dedicated search endpoint `systems/search/systems` for specifying complex queries. Using a GET request to this endpoint provides functionality similar to above but with a different syntax. For more complex queries a POST request may be used with a request body specifying the search conditions using an SQL-like syntax.

Search using GET on Dedicated Endpoint

Sending a GET request to the search endpoint provides functionality very similar to that provided for the endpoint `systems` described above. A list of search conditions may be specified using a series of query parameters, one for each attribute. All conditions are combined using logical AND. The general form for specifying the query parameters is as follows:

```
?
<attribute_1>.<op_1>=<value_1>&<attribute_2>.<op_2>=<value_2>)& ...
&<attribute_N>.<op_N>=<value_N>
```

Attribute names are given in the table above and may be specified using Camel Case or Snake Case.

Supported operators: eq neq gt gte lt lte in nin like nlike between nbetween

Example CURL command to search for systems that have Test in the name, are of type LINUX, are using a port less than 1024 and have a default authentication method of either PKI_KEYS or PASSWORD:

```
$ curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/systems/
search/systems?
name.like=*Test*\&enabled.eq=true\&system_type.eq=LINUX\&DefaultAut
```

Notes:

- For the `like` and `nlike` operators the wildcard character `*` matches zero or more characters and `!` matches exactly one character.
- For the `between` and `nbetween` operators the value must be a two item comma separated list of unquoted values.
- In a shell environment the character `&` separating query parameters must be escaped with a backslash.
- Attribute names may be specified using Camel Case or Snake Case.
- Following complex attributes not supported when searching:
 - `authnCredential` `jobRuntimes` `jobEnvVariables` `batchLogicalQueues` `tags` `notes`

Search using POST on Dedicated Endpoint

More complex search queries are supported when sending a POST request to the endpoint `systems/search/systems`. For these requests the request body must contain json with a top level property name of `search`. The `search` property must contain an array of strings specifying the search criteria in an SQL-like syntax. The array of strings are concatenated to form the full search query. The full query must be in the form of an SQL-like `WHERE` clause. Note that not all SQL features are supported.

For example, to search for systems that are owned by `jdoe` and of type `LINUX` or owned by `jsmith` and using a port less than `1024` create a local file named `system_search.json` with following json:

```
{
  "search":
  [
    "(owner = 'jdoe' AND system_type = 'LINUX') OR",
    "(owner = 'jsmith' AND port < 1024)"
  ]
}
```

To execute the search use a CURL command similar to the following:

```
$ curl -X POST -H "content-type: application/json" -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/systems/search/systems -d @system_search.json
```

Notes:

- String values must be surrounded by single quotes.
- Values for `BETWEEN` must be surrounded by single quotes.
- Search query parameters as described above may not be used in conjunction with a POST request.
- SQL features not supported include:
 - `IS NULL` and `IS NOT NULL`
 - Arithmetic operations
 - Unary operators
 - Specifying escape character for `LIKE` operator

Map of SQL operators to Tapis operators

Sql Operator	Tapis Operator
=	eq
≠	neq
<	lt
<=	lte
>	gt
>=	gte
LIKE	like
NOT LIKE	nlike
BETWEEN	between
NOT BETWEEN	nbetween
IN	in
NOT IN	nin

Sort, Limit, Select and ListType

When a list of Systems is retrieved the service provides for sorting, filtering and limiting the results. By default, only resources owned by you will be included. The service provides a way for you to request that all resources accessible to you be included. This is determined by the query parameter *listType*.

When retrieving either a list of resources or a single resource the service also provides a way to *select* which fields (i.e. attributes) are included in the results. Sorting, limiting and attribute selection are supported using query parameters.

Selecting

When retrieving systems the fields (i.e. attributes) to be returned may be specified as a comma separated list using a query parameter named *select*. Attribute names may be given using Camel Case or Snake Case.

Notes:

- Special select keywords are supported: *allAttributes* and *summaryAttributes*
- Summary attributes include:
 - *id*, *systemType*, *owner*, *host*, *effectiveUserId*, *defaultAuthnMethod*, *canExec*
- By default all attributes are returned when retrieving a single resource via the endpoint *systems/<system_id>*.
- By default summary attributes are returned when retrieving a list of systems.
- Specifying nested attributes is not supported.

- The attribute `id` is always returned.

For example, to return only the attributes `host` and `effectiveUserId` the CURL command would look like this:

```
$ curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/systems?select=host,effectiveUserId
```

The response should look similar to the following:

```
{
  "result": [
    {
      "id": "CSys_CltSrchGet_011",
      "host": "hostCltSrchGet_011",
      "effectiveUserId": "effUserCltSrchGet_011"
    },
    {
      "id": "CSys_CltSrchGet_012",
      "host": "hostCltSrchGet_012",
      "effectiveUserId": "effUserCltSrchGet_012"
    },
    {
      "id": "CSys_CltSrchGet_013",
      "host": "hostCltSrchGet_013",
      "effectiveUserId": "effUserCltSrchGet_013"
    }
  ],
  "status": "success",
  "message": "TAPIS_FOUND Systems found: 12 systems",
  "version": "1.0.0",
  "metadata": {
    "recordCount": 3,
    "recordLimit": 100,
    "recordsSkipped": 0,
    "orderBy": null,
    "startAfter": null,
    "totalCount": -1
  }
}
```

Sorting

The query parameter for sorting is named `orderBy` and the value is the attribute name to sort on with an optional sort direction. The general format is `<attribute_name>(<dir>)`. The direction may be `asc` for ascending or `desc` for descending. The default direction is ascending.

Examples:

- `orderBy=id`
- `orderBy=id(asc)`
- `orderBy=name(desc),created`

- `orderBy=id(asc),created(desc)`

Limiting

Additional query parameters may be used in order to limit the number and starting point for results. This is useful for implementing paging. The query parameters are:

- `limit` - Limit number of items returned. For example `limit=10`.
 - Use 0 or less for unlimited.
 - Default is 100.
- `skip` - Number of items to skip. For example `skip=10`.
 - May not be used with `startAfter`.
 - Default is 0.
- `startAfter` - Where to start when sorting. For example `limit=10&orderBy=id(asc),created(desc)&startAfter=101`
 - May not be used with `skip`.
 - Must also specify `orderBy`.
 - The value of `startAfter` applies to the major `orderBy` field.
 - Condition is context dependent. For ascending the condition is value `> startAfter` and for descending the condition is value `< startAfter`.

When implementing paging it is recommend to always use `orderBy` and when possible use `limit+startAfter` rather than `limit+skip`. Sorting should always be included since returned results are not guaranteed to be in the same order for each call. The combination of `limit+startAfter` is preferred because `limit+skip` is more likely to result in inconsistent results as records are added and removed. Using `limit+startAfter` works best when the attribute has a natural sequential ordering such as when an attribute represents a timestamp or a sequential ID.

ListType

By default, you will only see the resources that you own. The query parameter *listType* allows you to see additional resources that are available to you.

Options:

OWNED

Include only items owned by you (Default)

SHARED_PUBLIC

Include only items shared publicly

ALL

Include all items you are authorized to view.

Tapis Responses

For requests that return a list of resources the response result object will contain the list of resource records that match the user's query and the response metadata object will contain information related to sorting and limiting.

The metadata object will contain the following information:

- `recordCount` - Actual number of records returned.
- `recordLimit` - The limit query parameter specified in the request. -1 if query parameter was not specified.
- `recordsSkipped` - The skip query parameter specified in the request. -1 if query parameter was not specified.
- `orderBy` - The orderBy query parameter specified in the request. Empty string if query parameter was not specified.
- `startAfter` - The startAfter query parameter specified in the request. Empty string if query parameter was not specified.
- `totalCount` - Total number of records that would have been returned without a limit query parameter being imposed. -1 if total count was not computed.

For performance reasons computation of `totalCount` is only determined on demand. This is controlled by the boolean query parameter `computeTotal`. By default `computeTotal` is *false*.

Example query and response:

Query:

```
$ curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/systems?limit=2&orderBy=id(desc)
```

Response:

```
{
  "result": [
    {
      "id": "testMin0",
      "systemType": "S3",
      "owner": "testuser",
      "host": "my.example.host",
      "defaultAccessMethod": "ACCESS_KEY",
      "canExec": false
    },
    {
      "id": "MinSystem1c",
      "systemType": "LINUX",
      "owner": "testuser",
      "defaultAccessMethod": "PASSWORD",
      "host": "data.tacc.utexas.edu",
      "canExec": true
    }
  ],
  "status": "success",
```

```
"message": "TAPIS_FOUND Systems found: 2 systems",
"version": "1.0.0",
"metadata": {
  "recordCount": 2,
  "recordLimit": 2,
  "recordsSkipped": 0,
  "orderBy": "id(desc)",
  "startAfter": null,
  "totalCount": -1
}
```

Sites, Tenancy and Authentication

Sites

Tapis supports geographically distributed deployments where different components are running in different data centers and managed by different institutions. These physically isolated installations of Tapis software are referred to as *sites*. There is a single *primary site* and zero or more *associate sites* within a Tapis installation.

Primary Site

The primary site in a Tapis installation runs a complete set of Tapis API services and all associated 3rd-party services, such as databases and message queues. The creation of new sites is coordinated through the primary site, and the primary site runs the unique instance of the Sites and Tenants API (see Tenants below) which maintain the complete registry of all sites and tenants in the installation.

The primary site of the main Tapis installation is hosted at the Texas Advanced Computing Center at the tapis.io domain.

Associate Sites

Associate sites are required to run the Tapis Security Kernel, a compliant Token Generator API, and one or more additional Tapis services. Each associate site is managed and operated by a separate, partner institution. For Tapis services not run at the associate site, the corresponding service at the primary site is used for requests. In this way, partner institutions can choose which Tapis services to run within their institution and leverage the primary site deployment for the rest.

Deployment

The official Tapis deployment tooling targets the Kubernetes container orchestration platform. The project maintains a set of deployment templates which can be used in conjunction with configuration files to deploy Tapis services. If your institution is interested in becoming a Tapis associate site please contact us.

Details about the current list of sites is available from the tenants API. For example, one can retrieve the full list of sites as follows:

With PySDK:

```
>>> t.tenants.list_sites()
```

With CURL:

```
$ curl https://admin.tapis.io/v3/sites
```

The response will look similar to the following (the response below is truncated for brevity):

```
[
  base_url: https://tapis.io
  primary: True
  services: ['systems', 'files', 'security', 'tokens', 'streams',
'authenticator', 'meta', 'actors']
  site_admin_tenant_id: admin
  site_id: tacc
  tenant_base_url_template: https://${tenant_id}.tapis.io]
. . .
]
```

Each site has a `site_id` as well as a list of Tapis services it provides and the tenant ID of the administrative tenant (`admin_tenant`) associated with it.

Tenants

Tapis is a *multi-tenant* platform, meaning that different projects (or *tenants*) can have logically isolated views of the Tapis objects (i.e., the systems, files, actors, etc.) they create for their project.

Each tenant is made up of the following:

1. A base URL with which to access the tenant; by default, the base URL takes the form `https://<tenant_id>.tapis.io` where `tenant_id` is a short, unique identifier for the tenant in the Tapis system. For example, `https://tacc.tapis.io` is the base URL for the `tacc` tenant.
2. An *authenticator* providing the rules for who can authenticate in the tenant.

Additionally, each tenant is "managed" by a site.

To see the current list of tenants registered with Tapis, we can use the tenants API.

With PySDK:

```
>>> t.tenants.list_tenants()
```

With CURL:

```
$ curl https://tacc.tapis.io/v3/tenants
```

The response will look similar to the following (the response below is truncated for brevity):

```
allowable_x_tenant_ids: ['tacc']
authenticator: https://tacc.tapis.io/v3/oauth2
base_url: https://tacc.tapis.io
create_time: Thu, 02 Jul 2020 23:45:16 GMT
description: Production tenant for all TACC users.
is_owned_by_associate_site: False
last_update_time: Thu, 02 Jul 2020 23:45:16 GMT
owner: CICSsupport@tacc.utexas.edu
public_key: -----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAz7rr5CsFM7rHMFs7uKIdcc:
fp5Q+5ktltoBTfIaVDrXGF4DiCuzLsuvTG5fGElKEPPcpNqaCzD8Y1v9r3tfkoPT3Bc
EnSg1N30j1x8ktJPwbReKprHGiedlqdyT6j58l/I+9ihR6ettkMVCq7Ho/
bsIrwM5gP0PjJRvaD5Flsze7P4gQT37D1c5nbLR+K6/T0QTiyQIDAQAB
-----END PUBLIC KEY-----
security_kernel: https://tacc.tapis.io/v3/security
service_ldap_connection_id: None
tenant_id: tacc
token_service: https://tacc.tapis.io/v3/tokens
user_ldap_connection_id: tacc-all,

allowable_x_tenant_ids: ['dev']
authenticator: https://dev.tapis.io/v3/oauth2
base_url: https://dev.tapis.io
create_time: Fri, 19 Jun 2020 20:36:38 GMT
description: The dev tenant.
is_owned_by_associate_site: False
last_update_time: Fri, 19 Jun 2020 20:36:38 GMT
owner: CICSsupport@tacc.utexas.edu
public_key: -----BEGIN PUBLIC KEY-----
```

. . .

Here we see the first two tenants registered in the Tapis framework, the tacc and dev tenants.

In general, the rules for authentication vary from tenant to tenant within Tapis. For example, in the tacc tenant, any user with a valid TACC account can authenticate and use the APIs. The dev tenant is a development sandbox with test accounts used by the core Tapis team.

This documentation focuses on the tacc tenant; however, much of what follows in the subsequent sections will be the same regardless of the tenant you are using.

Authentication

The default authenticator provided by the Tapis project is based on OAuth2, and this is the authentication mechanism in place for the tacc tenant. The OAuth2-based authentication services are available via the /v3/oauth2 endpoints.

OAuth uses different *grant type flows* for generating tokens in different situations. We do not provide a comprehensive guide to OAuth2; for that, we refer the reader to the [OAuth2 docs](#). Instead, we provide a guide to the two most common use cases for users: generating tokens for themselves using the *password* grant flow, and generating tokens on behalf of others in a web application using the *authorization code* grant flow.

In the PySDK examples that follow, we will tacitly assume the `tapipy.tapis.Tapis` object has been instantiated as the Python object `t`. There are several options in the Tapis constructor, but the basic options include `base_url` and `username`, for example:

```
>>> t = Tapis(base_url='https://tacc.tapis.io', username='jdoe')
```

Password Grant - Generating a Token For Yourself

The simplest case is that you want to generate a Tapis OAuth token for yourself; to do this you can use the *password* grant flow, providing your username and password.

Tapis v3 tries to make this process as easy as possible by providing a simplified version of the password grant flow that does not require an OAuth client (see the `oauth-clients-label` section).

With PySDK:

```
>>> t = Tapis(base_url='https://tacc.tapis.io',
username='apitest', password='abcd123')
>>> t.get_tokens()
```

With CURL:

```
> curl -H "Content-type: application/json" -d '{"username":
"apitest", "password": "abcde123", "grant_type": "password" }' \
https://tacc.tapis.io/v3/oauth2/tokens
```

In the PySDK, the access token is a first-class Python object stored on the Tapis object (the `t` in the examples above). We can inspect it

```
>>> t.access_token
```

```
access_token:
eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJqdGkiOiJmN2I5YjE5ZS02MDk5LTUzNShKcUiz3-t0JPaYtFksKb4Bit_aFE1HH_X_znnP2QkJaqc-xaRoMlQu26MN72TLJE0siIN3T38xXWBGDumHUYbvnNzT-7lk7AQU5MHSyCWx8IRDmT9X6eLJQ_cz1QqMAiI_cPgA9VVE22qDK3Lbz2pp9t0sm-l9XjE5y5Im8Y0B2p0ssPD0TjW20C5yngZ4-4jowDafboKlscog9ko-adrsVIjG_r-ccCUX3r8SVwQLypZFZAPKqbVzl8jt_mCi30W8AYwiaYGmH7INBbHI9h07kwJNFMuSyIclaims: {'jti': 'f7b9b19e-6099-4f82-a722-b610c5d0ba0c', 'iss': 'https://tacc.tapis.io/v3/tokens', 'sub': 'apitest@tacc', 'tapis/tenant_id': 'tacc', 'tapis/token_type': 'access', 'tapis/delegation': False, 'tapis/delegation_sub': None, 'tapis/username': 'apitest', 'tapis/account_type': 'user', 'exp':
```

```
1595099456, 'tapis/client_id': None, 'tapis/grant_type':  
'password'}  
expires_at: 2020-07-18 19:10:56+00:00  
expires_in: <function Tapis.set_access_token.<locals>._expires_in  
at 0x7f29e213c510>  
jti: f7b9b19e-6099-4f82-a722-b610c5d0ba0c  
original_ttl: 14400
```

What we see is that the `access_token.access_token` is a Python string representing a JSON Web Token (JWT). JWTs are cryptographically signed with the private key associated with the tenant, and anyone can validate the signature by using the corresponding public key associated with the tenant (see Tenants section above). The public key for each tenant is available from the Tenants API. The core Tapis services will validate the access token sent on a given API call using the public key associated with the tenant to verify the JWT signature.

Using a Token

In order to use an access token in an API request to Tapis, pass the token in as the value of the `X-Tapis-Token` header. The PySDK will automatically send the token via this header for you. In CURL examples used throughout this documentation, we assume the raw JWT string representing an access token (like the above) has been exported as a shell variable; i.e.,

```
$ export  
JWT=eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJqdGkiOiJmN2I5YjE5ZS02Ml  
zNshKcUiz3-t0JPaYtFksKb4Bit_aFE1HH_X_znnP2QkJaqc-  
xaRoMlQu26MN72TlJE0siIN3T38xXWBGDumHUYbvnNzT-7lk7AQU5MHSyCWx8IRDmT  
X6eLJQ_cz1QqMAiI_cPgA9VVE22qDK3Lbz2pp9t0sm-  
l9XjE5y5Im8Y0B2p0ssPD0TjW20C5yngZ4-4jowDafboKlscog9ko-adrsVIjG_r-  
ccCUX3r8SVwQLypZFZAPKqbVz18jt_mCi30W8AYwiaYGmH7INBbHI9h07kwJNFMuSy
```

With that variable set, we can use the `-H` flag with `curl` to set the `X-Tapis-Token` header as follows:

```
$ curl -H "X-Tapis-Token: $JWT" ....
```

Note also the *claims* associated with the access token. These claims provide information about the token, including the user it represents (`apitest` in the above example), the tenant it belongs to (`tacc` above) when it expires, etc. Tapis tokens always include the following standard claims:

Claim	Description	Example Value
sub	The subject of the token; the subject uniquely identifies the user in a Tapis installation. The format is user @ tenant	<code>apitest@tacc</code>
exp	The expiry associated with the token.	<code>1595099456</code>
jti		

Claim	Description	Example Value
	Unique identifier for the token.	f7b9b19e-6099-4f82-a722-b610c5d0ba0c
iss	The identifier (URL) of the issuer of the JWT. For Tapis, the issuer will be a Tokens API.	https://tacc.tapis.io/v3/tokens

Additional custom claims specific to Tapis are namespaced with `tapis/` at the beginning of the claim name. The authenticator for each tenant may optionally choose to support one or more of these additional claims. The following claims are encouraged and supported by the default OAuth2 Tapis authenticator.

Claim	Description	Example Value
tapis/tenant_id	The tenant of the subject.	tacc
tapis/username	The username of the subject.	apitest
tapis/token_type	Type of token: access or refresh	access
tapis/account_type	Type of account: user or service	user
tapis/delegation	Whether a delegation flow was used to generate this token. (true or false).	false
tapis/delegation_sub	For a delegation token, the subject who actually generated the token. In form user @ tenant	superuser@tacc
tapis/client_id	The id of the OAuth client used to generate the token.	tacc.CIC.tokenapp
tapis/grant_type	The grant type used to generate the token.	authorization_code

The authenticator for your tenant may include additional claims not listed here.

OAuth Clients

In order to use the more advanced OAuth2 flows, including any use of the authorization code grant type, the implicit grant type, and to generate refresh tokens with the password grant type, you must generate an OAuth2 *client*. Clients in OAuth2 represent applications (for example, a web or mobile application) that will interact with the OAuth2 server to generate tokens on behalf of one or more users. Clients are created and managed using the `/v3/oauth2/clients` endpoints.

Creating Clients

To create a client, make a POST request to the Clients API. All fields are optional; if you do not pass a `client_id` or `client_key` in the request, the clients API will generate random ones for you. In order to use the `authorize_code` and `implicit` grant types you will need to set the `callback_url` when registering your client (see `auth_code`). For a complete list of available parameters, see the API live-docs for [Clients](#).

With PySDK:

```
>>> t.authenticator.create_client(client_id='test',
callback_url='https://foo.example.com/oauth2/callback')
```

With CURL:

```
$ curl -H "X-Tapis-Token: $JWT" -H "Content-type: application/
json" -d '{"client_id": "test", "callback_url": "https://
foo.example.com/oauth2/callback"}' https://tacc.tapis.io/v3/
oauth2/clients
```

The response will be similar to

```
callback_url: https://foo.example.com/oauth2/callback
client_id: test
client_key: WQZlQlMox0ynW
create_time: Sat, 18 Jul 2020 19:09:47 GMT
description:
display_name: https://foo.example.com/oauth2/callback
last_update_time: Sat, 18 Jul 2020 19:09:47 GMT
owner: apitest
tenant_id: tacc
```

Listing Clients

With PySDK:

```
>>> t.authenticator.list_clients()
```

With CURL:

```
$ curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/oauth2/
clients
```

The response will be similar to

```
[
callback_url: https://foo.example.com/oauth2/callback
client_id: test
client_key: WQZlQlMox0ynW
create_time: Sat, 18 Jul 2020 19:09:47 GMT
description:
display_name: https://foo.example.com/oauth2/callback
last_update_time: Sat, 18 Jul 2020 19:09:47 GMT
```

```
owner: apitest
tenant_id: tacc]
```

Deleting Clients

You can also delete clients you are no longer using; just pass the `client_id` of the client to be deleted:

With PySDK:

```
>>> t.authenticator.delete_client(client_id='test')
```

With CURL:

```
$ curl -H "X-Tapis-Token: $JWT" -X DELETE https://tacc.tapis.io/v3/oauth2/clients/test
```

A null response is returned from a successful delete request.

Authorization Code Grant - Generating Tokens For Users

An important aspect of OAuth2 is that it enables applications to generate tokens on behalf of users without the applications needing to possess user credentials (i.e., passwords). In this section, we discuss using the OAuth2 *authorization code* grant type to do just that.

Assuming a Model-View-Controller (MVC) architecture, there are two controllers that must be written to support the authorization code grant type flow.

1. A controller to determine if the user already has a valid access token and direct them to the OAuth2 authorization server when they do not. This controller starts the authorization code process. To do so, it should:

- First inform the user that they will be asked to authenticate with their tenant username and password and then be asked to grant authorization to your client application.
- Redirect the user to the OAuth2 server's authorization URL. In the default Tapis authenticator, the authorization URL path is `/v3/oauth2/authorize`; for example, `https://tacc.tapis.io/v3/oauth2/authorize` in the `tacc` tenant.
- The redirect request should include the following query parameters:
 - `client_id`: the id of your client.
 - `redirect_uri`: the URI to redirect back to with the authorization code. This must match the `callback_url` parameter associated with your client.
 - `response_type`: should always have the value `code`.

1. A controller to process the authorization code returned and retrieve an access token on the user's behalf. This controller receives requests containing authorization codes from the OAuth2 server after the user has successfully

authenticated with said OAuth2 server, and it immediately exchanges the code for a token.

- Responds to GET requests to the URL defined in the `callback_url` parameter of your client.
- Retrieves the code query parameter from the request.
- Makes a POST request to the OAuth2 server's tokens endpoint to generate a token. In the default Tapis authenticator, the tokens URL path is `/v3/oauth2/tokens`; for example, `https://tacc.tapis.io/v3/oauth2/tokens` in the `tacc` tenant. The POST body must include the following parameters:
 - `code`: the code the controller just received in the request from the OAuth2 server.
 - `redirect_uri`: should be the same as the `callback_url` parameter of your client.
 - `grant_type`: should always have the value `authorization_code`.

Note that many popular web frameworks support OAuth2 flows with minimal custom coding required.

The final step to using the authorization code grant type is to register a client (see above) with a `callback_url` parameter equal to the URL within your web application where it will handle converting authorization codes into access tokens (i.e., controller 2 above).

Implicit Grant - Directly Returning Tokens For Users

As mentioned in the authorization code grant section, OAuth2 provides an extra wall of security where the application does not encounter credentials of the user (i.e., passwords). In this section, we discuss using the OAuth2 *implicit* grant type.

Just like the authorization code grant type, the implicit grant type retrieves an access token on behalf of the user to login. In addition, the implicit grant type requires a redirect to the authorization server.

The difference between the two grant types is that the access token is directly delivered through the browser, rather than a code that is later exchanged for the access token. This allows the implicit grant type to be easier to set up, but much less secure as it cannot authenticate the client. The typical use case for the implicit grant type would be for applications where the client secrets cannot be securely stored.

This is how to start the implicit flow:

- Redirect the user to the OAuth2 server's authorization URL. In the default Tapis authenticator, the authorization URL path is `/v3/oauth2/authorize`; for example, `https://tacc.tapis.io/v3/oauth2/authorize` in the `tacc` tenant.
- The redirect request should include the following query parameters:
 - `client_id`: the id of your client.
 - `redirect_uri`: the URI to redirect back to with the access token. This must match the `callback_url` parameter associated with your client.

- `response_type`: should always have the value `token`.

Once the user completes the login process through the authorization server, the server redirects the user to the `redirect_uri` along with the access token in the browser.

The final step is to collect the access token in the url and store it in the cookies.

The implicit grant type also requires a client (see above) with a `callback_url` parameter equal to the URL within your web application where it will handle storing the access tokens in the cookies.

Implicit Grant Tutorial

This section serves as a small tutorial to further understand how the implicit grant works and how to set it up.

The first step is to create an OAuth2 client as shown in the [OAuth Clients](#) section. While setting up the client, make sure to set the `callback_url` to the website that the user will be redirected to (`redirect_uri`) after successfully authenticating on the authorization server. The whole flow will not work if these two values are not identical. Additionally, make sure to remember the `client_id` as it will be needed later.

Note

The `callback_url` of the client created can be changed later on if needed, but the `client_id` cannot.

The next step is to redirect the user to the authorization server's website. The most common way is to create a button on the website that takes the user to the authorization server's website. Make sure to follow this specific formatting for the website url in order to prevent any errors from occurring:

```
https://{tenant}.tapis.io/v3/oauth2/login?client_id={client_id}&redirect_uri={redirect_uri}&response_type={response_type}
```

This is the formatting that will be used for all OAuth2 authentication. Everything in curly brackets needs to be replaced with the correct values to successfully set up the implicit flow. Make sure to drop the curly brackets when replacing them with their respective values.

- `{tenant}`: This is the tenant value. Examples are `tacc`, `scoped`, `dev`, etc.
- `{client_id}`: This is the client id for the client that was created in the first step.
- `{redirect_uri}`: This is the value that needs to be the same as the `callback_url`.

Important

The value that replaces this `{redirect_uri}` needs to be *URL Encoded*!! Use an online url encoder to help format the value if needed.

Example of a properly url encoded value:

Notifications

The Notifications service supports the publish-subscribe model for delivering information to interested parties. Currently only a Tapis service may directly create subscriptions and post events through this service. Users may subscribe to job related events by creating subscriptions through the Jobs service. For more information on creating subscriptions through the Jobs service, please see [Job Subscriptions](#)

Warning

Currently subscriptions for notifications may only be managed through the Tapis Jobs service.

Overview

In Tapis, a notification *event* represents an occurrence that may be of interest to other parties. The events are delivered asynchronously using a publish-subscribe model. Once an interested party has created a *subscription*, matching *events* are delivered to the interested party as part of a *notification* object. Deliveries are made via webhook or email.

Currently only services may directly create subscriptions and publish events.

The model for a notification event is based on the CloudEvent specification version 1.0. For more information about CloudEvents and the specification, please see <https://cloudevents.io> and <https://github.com/cloudevents/spec>.

Please note that although currently the Notifications service is only accessed by users through the Jobs service, this document discusses the general design and details of the service in order to provide information for future planned development.

Event Model

An event contains the attributes listed below. The attributes *source*, *type* and *timestamp* are required. Note that some attributes are maintained by Tapis, they are present when the event is part of a delivered notification. The attributes maintained by Tapis may be present when publishing an event, but they will be ignored.

Event attributes:

source

Context in which the event happened. For example, for a job related event originating from Tapis at the primary TACC site, the source would be `https://tapis.io/jobs`.

type

Type of event. Used for routing notifications. A series of 3 fields separated by the dot character. The first field is the service name, the second field is the category and the third field is the detail. For example, when a job transitions to the FINISHED state, the type is `jobs.JOB_NEW_STATUS.FINISHED`.

subject

Subject of event in context of service. Examples: job Id, system Id, file path, role name, etc.

timestamp

When the event happened.

data

Optional. Additional information associated with the event. For example, data specific to the service associated with the event.

seriesId

Optional. Id to be used for grouping events from the same tenant, source and subject. In a series, event order is preserved when sending out notifications. For example, the Jobs service (the source) sends out events with the job UUID as the subject and sets the seriesId to the job UUID. That way, a subscription can be created to follow (in order) all events of various types related to the job.

deleteSubscriptionsMatchingSubject

Boolean indicating that all subscriptions whose *subjectFilter* matches the *subject* of the event should be deleted once all notifications are delivered.

tenant

Tapis tenant associated with the event.

uuid

Tapis generated unique identifier for the event.

user

Tapis user associated with the event.

received

Tapis generated timestamp for when the event was received by the Notifications service.

seriesSeqCount

Tapis generated counter for seriesId. Can be used by notification receivers to track expected order. Notifications for events will be sent in order but may not be received in order.

Note that events are not persisted by the front end api service process. When events are received they are sent to a message broker for asynchronous processing. The back end process will persist events temporarily in order to support recovery.

Event Type

An event type represents a channel through which users and services receive events. Services and users create subscriptions with an event type filter in order to select the events delivered to them. The identifier for each event type must have three parts in the format **<service>.<category>.<detail>**. For example *jobs.JOB_NEW_STATUS.PENDING*, *apps.APP.UPDATE* or *files.OBJECT.DELETE*.

Subscription

A service can create a subscription for an event type in order to allow users and services to receive events. Two delivery methods are supported, WEBHOOK and EMAIL.

At a high level, a subscription contains the following information:

name

Optional short descriptive name. *owner+name* must be unique. Composed of alphanumeric characters and the following special characters: `-._~`. If not provided, the service will create one.

owner

A specific user set at create time. Default is *apiUserId*.

description

An optional more verbose description.

typeFilter

Filter to use when matching events. Matches against event type. Has three parts for matching **<service>.<category>.<detail>**. Each field may be a specific type or the wildcard character `*`.

subjectFilter

Filter to use when matching events. Matches against event subject. This may be a specific subject such as a job Id or the wildcard character, *.

deliveryTargets

List of targets to be used when delivering an event. Each target includes delivery method (EMAIL or WEBHOOK) and delivery address.

ttlMinutes

Time to live in minutes. Specified when subscription created. Default is one week from creation. A TTL of 0 or less indicates no expiration. May be updated through an API call. Attribute *expiry* is recomputed when this attribute is updated.

expiry

Time at which the subscription expires. Maintained by the service. Computed at create time and recomputed when attribute *ttlMinutes* is updated.

The attributes *typeFilter*, *subjectFilter* and *deliveryTargets* are required.

Subscription Name

For each owner the name must be unique and can be composed of alphanumeric characters and the following special characters: - . _ ~. If the attribute *name* is not provided, then the service will generate one using the template:

```
<jwtUser>~<owner>~<oboTenant>~<subjectFilter>~<random4>
```

For example:

```
jobs~testuser1~dev~jobs.JOB_NEW_STATUS.ALL~m4Kx
```

Note that when constructing the name:

- *subjectFilter* will be truncated to 40 characters
- If *subjectFilter* is the wildcard character *, it is replaced with the string ALL when constructing the name.
- The last 4 characters are generated at random from the set of alphanumeric characters including upper case, lower case and digits.

Delivery Target

Each subscription will contain a list of delivery targets for use in delivering events. The list must contain at least one item. WEBHOOK and EMAIL deliveries are supported.

A delivery target contains the following information:

- *deliveryMethod* - The type of delivery method: WEBHOOK, EMAIL
- *deliveryAddress* - URL for WEBHOOK or email address for EMAIL

Notification Model

A notification is an object encapsulating the information sent to a delivery target. It contains the following:

- *uuid* - Unique identifier for the notification.
- *event* - All information contained in the event. See above under the section Event Model.
- *eventUuid* - Unique identifier for the event.
- *tenant* - tenant associated with the notification.
- *subscriptionName* - Name of subscription associated with the notification.
- *deliveryTarget* - the delivery target
- *created* Timestamp for when the notification was created.

Example of a notification sent to a webhook:

```
{
  "uuid": "30d70395-d5e9-43a4-ae90-2306b6bb00d6",
  "tenant": "admin",
  "subscriptionName": "4d0abbce-5cec-4d6e-8065-cdc5b2777389",
  "eventUuid": "50cfb971-c4b3-4d33-89c3-2b0f56f16e19",
  "event": {
    "source": "notifications",
    "type": "notifications.test.begin",
    "subject": "4d0abbce-5cec-4d6e-8065-cdc5b2777389",
    "data": null,
    "seriesId": "4d0abbce-5cec-4d6e-8065-cdc5b2777389",
    "timestamp": "2023-09-15T14:47:50.287792699Z",
    "deleteSubscriptionsMatchingSubject": false,
    "tenant": "admin",
    "user": "notifications",
    "received": "2023-09-15T14:47:51.000",
    "uuid": "50cfb971-c4b3-4d33-89c3-2b0f56f16e19",
    "seriesSeqCount": 4
  },
  "deliveryTarget": {
    "deliveryMethod": "WEBHOOK",
    "deliveryAddress": "https://admin.develop.tapis.io/v3/
notifications/test/callback/4d0abbce-5cec-4d6e-8065-cdc5b2777389/"
  },
  "created": "2023-09-15T14:47:50.315188203Z"
}
```

Example of a notification sent to an email address:

```
{
  "uuid": "befe2475-58ad-4a5c-bcf2-593f04e49a20",
  "tenant": "dev",
  "subscriptionName": "jobs~testuser2~dev~ef9004c3-09d5-41d5-acd3-be7c9fd3daf6-007~cxh2",
  "eventUuid": "1d16202d-2248-4690-bcc9-a0134a4089cd",
  "event": {
    "source": "https://tapis.io/jobs",

```

```

    "type": "jobs.JOB_NEW_STATUS.FINISHED",
    "subject": "ef9004c3-09d5-41d5-acd3-be7c9fd3daf6-007",
    "data": "{\n  \"newJobStatus\": \"FINISHED\", \"oldJobStatus\":\n  \"ARCHIVING\", \"blockedCount\": 0, \"remoteJobId\":\n  \"35299a7d78f1591e395fdcec9dc6b1f3606be9f56f38453129b6ccc383ed9759\"\n  \"remoteJobId2\": null, \"remoteOutcome\": \"FINISHED\", \n  \"remoteResultInfo\": \"0\", \"remoteQueue\": null, \n  \"remoteSubmitted\": \"2023-09-15T15:11:18.354731067Z\", \n  \"remoteStarted\": null, \"remoteEnded\": null, \"jobName\": \"Tapis V3\n  smoketest job\", \"jobUuid\": \"ef9004c3-09d5-41d5-acd3-\n  be7c9fd3daf6-007\", \"jobOwner\": \"testuser2\", \"message\": \"The\n  job has transitioned to a new status: FINISHED. The previous job\n  status was ARCHIVING.\"}",
    "seriesId": "ef9004c3-09d5-41d5-acd3-be7c9fd3daf6-007",
    "timestamp": "2023-09-15T15:11:23.947827477Z",
    "deleteSubscriptionsMatchingSubject": true,
    "tenant": "dev",
    "user": "jobs",
    "received": "2023-09-15T14:47:51.000",
    "uuid": "1d16202d-2248-4690-bcc9-a0134a4089cd",
    "seriesSeqCount": -1
  },
  "deliveryTarget": {
    "deliveryMethod": "EMAIL",
    "deliveryAddress": "me@example.com"
  },
  "created": "2023-09-15T15:11:23.965413696Z"
}

```

Notification Delivery

Background

When events are published to the Notifications front end api service, they are initially placed on a message broker queue to be picked up asynchronously by a back end worker process known as the dispatch service. Currently RabbitMQ is used as the message broker.

The dispatch service reads events from the queue and assigns them to workers known as *delivery bucket managers*. Delivery bucket managers are threads that receive their assigned events from in-memory queues. The dispatch service assigns events to a bucket manager by taking a hash of the event *tenant*, *source*, *subject* and *seriesId*. The hash allows for distributing work among the bucket managers while ensuring that for a given *seriesId* the same bucket manager will process that series of events. This is how the service ensures that notifications for events in a series are sent out in order.

When a bucket manager worker receives an event to process, it first finds all matching subscriptions by querying a database. As discussed above, the matching is based on the *typeFilter* and *subjectFilter* defined in a subscription.

For each delivery target in each matching subscription, the worker creates a Notification object and persists it to a database. By persisting to a database we are able to support recovery and retries. The worker then begins the process of delivering the notifications.

Configuring for EMAIL Delivery

Supporting delivery by EMAIL involves configuring the Tapis Notifications service to use an SMTP relay. This must be done by the Tapis systems administrator. Parameters for the relay are set as environment variables to be picked up by the dispatcher service when it is started during a deployment. For more information on deployer configuration please see [Notifications_Email_Config](#).

Please note that deployer currently only supports template variables for TAPIS_MAIL_PROVIDER, TAPIS_SMTP_HOST and TAPIS_SMTP_PORT. Other environment variables must be set manually in the deployment.

The environment variables used to configure email delivery are:

TAPIS_MAIL_PROVIDER

Optional. Supported values: SMTP, LOG, NONE. Default is LOG. This should typically be set to SMTP. Setting to LOG results in the dispatcher generating a log message showing the email information. Setting to NONE results in delivery being a NO-OP.

TAPIS_SMTP_HOST

Required if provider is SMTP. Host to use as relay when sending email via SMTP.

TAPIS_SMTP_PORT

Optional. Port used when sending email using SMTP. Default is 25.

TAPIS_SMTP_FROM_NAME

Optional. Name for the email From: field. Default value is *Tapis Notifications Service*.

TAPIS_SMTP_FROM_ADDRESS

Optional. Address for the email From: field. Default value is *no-reply@nowhere.com*.

TAPIS_SMTP_AUTH

Optional. Boolean indicating if SMTP server requires a username and password. Default is *false*.

TAPIS_SMTP_USER

Required if TAPIS_SMTP_AUTH is *true*.

TAPIS_SMTP_PASSWORD

Required if TAPIS_SMTP_AUTH is *true*.

EMAIL Delivery

When the notification delivery method is of type EMAIL, the dispatch worker will send an email using SMTP.

The **To:** field for the email will be the notification delivery address.

The **From:** field for the email will depend on the configuration parameters, as discussed above in the section Configuring for EMAIL Delivery. By default this will be:

Tapis Notifications Service <no-reply@nowhere.com>

The **Subject:** of the email will have the following format:

Tapis v3 notification. Event type: <event_type> subject: <subject>

If the event has no *subject* then the email subject will not have the subject portion.

An example email subject for the case where the event contains a *subject* attribute:

Tapis v3 notification. Event type: jobs.JOB_NEW_STATUS.FINISHED
subject: 1451b0ef-c057-4177-acd5-51a4901acb07-007

The body of the email will contain the notification data as json. An example may be found above under the section Notification Model.

WEBHOOK Delivery

When the notification delivery method is of type WEBHOOK, the dispatch worker will deliver the notification using an HTTP POST request. The media type for the request will be *application/json* and the following header will be added: **User-Agent:** Tapis/v3.

The request body will be a json structure with the notification information. An example may be found above under the section Notification Model.

Tables

Subscription Attributes

Attribute	Type	Example	Notes
tenant	String	designsafe	• Name of the tenant associated with the subscription. • <i>tenant</i> + <i>owner</i> + <i>name</i> must be unique. • Determined by the service at creation time.
name	String	my-email-ntf-1	• Optional short descriptive name. • <i>tenant</i> + <i>owner</i> + <i>name</i> must be unique. • Allowed characters: Alphanumeric [0-9a-zA-Z] and <code>-._~</code>. • If not provided the service will create one.
owner	String	jdoe	• username of <i>owner</i>. • Variable references: <code>\${apiUserId}</code>. Resolved at create time. • By default this is the resolved value for <code>\${apiUserId}</code>.
description	String	My email	• Optional more verbose description. Maximum length of 2048 characters.
enabled	boolean	FALSE	• Indicates if subscription is active. • May be updated using the <code>enable/disable</code> endpoints. • By default this is <i>true</i>.
typeFilter	String	apps.APP.DELETE	• Filter to use when matching events. • Matches against event type. • Has three dot separated parts: <code><service>.<category>.<detail></code>.

Attribute	Type	Example	Notes
			Each part may be a specific type or the wildcard character *.
subjectFilter	String	<job-id>	- Filter to use when matching events. - Matches against event subject. - Can be specific for an exact match or the wildcard character *.
deliveryTargets	Object[]		- List of delivery targets to be used when delivering a matching event. - Must have at least one. - Each target includes delivery method and delivery address. - Delivery methods supported: WEBHOOK, EMAIL
ttlMinutes	int	60	- Time to live in minutes. Can be updated. - Service will compute expiry based on this attribute. - Default is one week from creation. - Value of 0 indicates no expiration.
expiry	Timestamp	2020-06-26T15:10:43Z	- Time at which the subscription expires and will be deleted. - Maintained by the service. - Computed at create time. - Recomputed when attribute <i>ttlMinutes</i> is updated.
uuid	UUID		Auto-generated by service.
created	Timestamp	2020-06-19T15:10:43Z	When the subscription was created. Maintained by service.
updated	Timestamp	2020-06-20T23:21:22Z	When the subscription was last updated. Maintained by service.

Event Attributes

Attribute	Type	Example	Notes
source	String	https://tapis.io/jobs	Context in which event happened.
type	String	jobs.JOB_NEW_STATUS.QUEUED	- Type of event. Used for routing notifications. - Pattern is <code><service>.<category>.<detail></code>
subject	String	<code><job-id></code>	- Subject of event in the context of the service. - Examples: job Id, app Id, file path, role name, etc.
timestamp	String	2020-06-19T15:10:43Z	When the event happened.
data	String		- Optional. Other data associated with the event. - For example, service specific information.
deleteSubscriptionsMatchingSubject	boolean		Delete subscriptions where subjectFilter matches subject
endSeries	boolean		Delete tracking data for the series.
seriesId	String	<code><job-id></code>	- Optional. Group events based on tenant,source,subject. - Preserves event order during notification delivery.
tenant	String	<code>tacc</code>	Tapis tenant associated with the event.
uuid	String		Tapis generated unique identifier.
user	String		

Attribute	Type	Example	Notes
			• Tapis user associated with the event.
received	String	2020-06-19T15:10:43Z	• When the event was received by Tapis.
seriesSeqCount	String		• Tapis generated counter for seriesId.

Notification Attributes

Attribute	Type	Notes
uuid	String	Unique identifier for the notification.
tenant	String	Tenant associated with the notification.
subscriptionName	String	Name of subscription associated with the notification.
eventUuid	String	Unique identifier for the event contained in the notification.
event	Object	Event that triggered the notification.
deliveryTarget	Object	The delivery target for the notification.
created	String	When the notification was created.

Delivery Target Attributes

Attribute	Type	Notes
deliveryMethod	enum	WEBHOOK or EMAIL
deliveryAddress	String	URL for WEBHOOK or email address for EMAIL

From: <https://tacc.tapis.io/v3/oauth2/login?>
To: <https%3A%2F%2Ftacc.tapis.io%2Fv3%2Foauth2%2Flogin%3F>

* {response_type}: This is the value that will be delivered through the browser and is dependent on the authorization type that is being used.

- For implicit, use token.
- For authorization code, use code.

Once the user has authenticated through the server, the user will automatically be redirected to the `redirect_uri` or `callback_url`. When redirected, the `access_token` (code for authorization code) will be returned in the url. Use a method to capture this value and store it in the cookies.

Important

Note that for both `access_token` and `code`, there are more parameters that are returned alongside these values through the url like `state`. Therefore, make sure to capture the value that is between the "`*access_token=*`" (or "`*code=*`") and "`*&state=*`".

Finally, since the user does not need to physically see the redirect page, store the token in the cookies and redirect the user one last time to the main page.

Note

For the `authorization code` grant type, everything in this tutorial is pretty much identical. However, there is one more final step of exchanging the code returned through the url with the token to complete the flow.

The Tapis Token Web Application

Tapis provides a graphical interface via a web application that enables users to generate tokens. The Tapis Web Application is available by default for any tenant using the default Tapis authenticator, including the `tacc` tenant. The Tapis Token Web Application serves as an example of an application using the authorization code grant type.

The Tapis Token Web Application and its source code are available at the following URLs:

- Token App (`tacc` tenant): <https://tacc.tapis.io/v3/oauth2/webapp>
- Token App source code: <https://github.com/tapis-project/authenticator>

Meta

Meta V3 is a REST API Microservice for MongoDB which provides server-side Data, Identity and Access Management for Web and Mobile applications.

Why Meta V3

Meta V2 functionality was built on MongoDB version 1.0 technology. Although limited, it brought the basic functions of a document store to Agave/Tapis platform. Some projects decided to use basic key/value storage to add metadata to Jobs, Apps, Systems and Files. Other projects stored complex document data to associate Jobs and other entities to add a richer information layer to their portals. The limited search functionality and imposed document structure created impediments to using MongoDB as projects had envisioned. Meta V3 removes these impediments.

Meta V3 is built on top of MongoDB version 4.2 technology. The REST API opens up the full functionality of MongoDB as a document store. and delivers MongoDB as a service so that projects are free to create metadata and documents in a fashion that fits their needs. Some of the Meta V3 advantages over Meta V2 include:

- Any valid MongoDB document structure can be used.
- If a search runs in MongoDB CLI, it should run from the API.
- Aggregations are available.
- Database, collection and document creation can be managed by tenant administrator.
- Performance is many times faster.

Migration from Meta V2 to V3

Migration can simply be accomplished by creating a new database with one or more collections for your project. The tenant administrator can request the initial permissions setup. Once your collection(s) are in place move the result and associatedIds into your new document model.

Overview

Meta V3 is:

A Stateless Microservice. With Meta V3 projects can focus on building Angular or other frontend applications, because most of the server-side logic necessary to manage database operations, authentication / authorization and related APIs is automatically handled, without the need to write any server-side code except for the UX/UI.

For example, to insert data into MongoDB a developer has to just create client-side JSON documents and then execute POST operations via HTTP to Meta V3. Other functions of a modern MongoDB installation like flexible schema, geoJson and aggregation pipelines ease the development process.

Every tenant will have access to at least one database where they can store and manage json documents. Documents are the trailing end of a nested hierarchy of data that begins with a database that houses one or more collections. The collections house json documents the structure of which is left up to the administrators of the tenant database.

Permissions for access to databases, collections and documents must be predefined before accessing those resources. The definitions for access are defined within the Security Kernel API of Tapis V3 and must be added by a tenant or service administrator. See the Permissions section below for some examples of permissions definitions and access to resources in the Meta V3 API.

Getting Started

Create a document

We have a database named MyTstDB and a collection name MyCollection. To add a json document to MyCollection, we can do the following:

With CURL:

```
$ curl -v -X POST -H "Content-Type:application/json" -H "X-Tapis-Token:$jwt" --data '{"name": "test document slt 7.21.2020-14:27","jimmyList": ["1","3"],"description": "new whatever",}' $BASE_URL/v3/meta/MyTstDB/MyCollection?basic=true
```

The response will have an empty response body with a status code of 201 "Created" unless the "basic" url query parameter is set to true. Setting the "basic" parameter to true will give a Tapis Basic response along with the "_id" of the newly created document. A more detailed discussion of autogenerated ids and specified ids can be found in the "Create Document" section of "Document Resources".

```
{
  "result": {
    "_id": "5f189316e37f7b5a692285f3"
```

```

    },
    "status": "201",
    "message": "Created",
    "version": "0.0.1"
}

```

List documents

Using our MyTstDb/MyCollection resources we can ask for a default list of documents in MongoDB default sorted order. The document we created earlier should be listed with a new "_id" field that was autogenerated by MongoDB.

With CURL:

```
$ curl -v -X GET -H "Content-Type:application/json" -H "X-Tapis-Token:$jwt" $BASE_URL/v3/meta/MyTstDB/MyCollection
```

The response will be an array of json documents from MyCollection :

```

[
  {
    "_id": {
      "$oid": "5f189316e37f7b5a692285f3"
    },
    "name": "test document slt 7.21.2020-14:27",
    "jimmyList": [
      "1",
      "3"
    ],
    "description": "new whatever",
    "_etag": {
      "$oid": "5f189316296c81742a6a3e4c"
    }
  },
  {
    "_id": {
      "$oid": "5f1892ece37f7b5a692285e9"
    },
    "name": "test document slt 7.21.2020-14:25",
    "jimmyList": [
      "1",
      "3"
    ],
    "description": "new whatever",
    "_etag": {
      "$oid": "5f1892ec296c81742a6a3e4b"
    }
  }
]

```

Get a document

If we know the "_id" of a created document, we can ask for it directly.

With CURL:

```
$ curl -v -X GET -H "Content-Type:application/json" -H "X-Tapis-Token:$jwt" $BASE_URL/v3/meta/MyTstDB/MyCollection/5f1892ece37f7b5a692285e9
```

The response will be a json document from MyCollection with the "_id" of 5f1892ece37f7b5a692285e9 :

```
{
  "_id": {
    "$oid": "5f1892ece37f7b5a692285e9"
  },
  "name": "test document slt 7.21.2020-14:25",
  "jimmyList": [
    "1",
    "3"
  ],
  "description": "new whatever",
  "_etag": {
    "$oid": "5f1892ec296c81742a6a3e4b"
  }
}
```

Find a document

We can pass a query parameter named "filter" and set the value to a json MongoDB query document. Let's find a document by a specific "name".

With CURL:

```
$ curl -v -X GET -H "Content-Type:application/json" -H "X-Tapis-Token:$jwt" --data-urlencode filter='{ "name": "test document slt 7.21.2020-14:25" }' $BASE_URL/v3/meta/MyTstDB/MyCollection
```

The response will be an array of json documents from MyCollection :

```
[
  {
    "_id": {
      "$oid": "5f1892ece37f7b5a692285e9"
    },
    "name": "test document slt 7.21.2020-14:25",
    "jimmyList": [
      "1",
      "3"
    ],
    "description": "new whatever",
  }
]
```



```
    "_etag": {  
      "$oid": "5f1892ec296c81742a6a3e4b"  
    }  
  }  
}
```

Resources

General resources

An unauthenticated Health check is included in the Meta V3 API to let any user know the current condition of the service.

Health Check

An unauthenticated request for the health status of Meta V3 API.

With pySDK operation:

```
$ t.meta.healthCheck()
```

With CURL:

```
$ curl -v -X GET -H "Content-Type:application/json" $BASE_URL/v3/  
meta/
```

The response will be a Basic Tapis response on health:

```
{  
  "result": "",  
  "status": "200",  
  "message": "OK",  
  "version": "0.0.1"  
}
```

Root resources

The Root resource space represents the root namespace for databases on the MongoDB host. All databases are located here. Requests to this space are limited to READ only for tenant administrators.

List DB Names

A request to the Root resource will list Database names found on the server. This request has been limited to those users with tenant administrative roles.

With pySDK operation:

```
$ t.meta.listDBNames()
```

With CURL:

```
$ curl -v -X GET -H "Content-Type:application/json" -H "X-Tapis-Token:$jwt" $BASE_URL/v3/meta/
```

The response will a json list of database names:

```
[
  "StreamsDevDB",
  "v1airr"
]
```

Database resources

The Database resource is the top level for many tenant projects. The resource maps directly to a MongoDB named database in the database server. Case matters for matching the name of the database and must be specified when making requests for collections or documents. Currently

List Collection Names

This request will return a list of collection names from the specified database {db}. The permissions for access to the database are set prior to access.

With pySDK operation:

```
$ t.meta.listCollectionNames(db='')
```

With CURL:

```
$ curl -v -X GET -H "Content-Type:application/json" -H "X-Tapis-Token:$jwt" -d '' $BASE_URL/v3/meta/{db}
```

Here is an example response:

```
[
  "streams_alerts_metadata",
  "streams_channel_metadata",
  "streams_instrument_index",
  "streams_project_metadata",
  "streams_templates_metadata",
  "tapiskapa-local"
]
```

Get DB Metadata

This request will return the metadata properties associated with the database. The core server generates an etag in the properties collection for a database that is necessary for future deletion.

With pySDK operation:

```
$ t.meta.getDBMetadata(db='')
```

With CURL:

```
$ curl -v -X GET -H "Content-Type:application/json" -H "X-Tapis-Token:$jwt" -d '' $BASE_URL/v3/meta/{db}/_meta
```

Here is an example response:

```
{
  "_id": "_meta",
  "_etag": { "$oid": "5ef6232b296c81742a6a3e02" }
}
```

Create DB

TODO: this implementation is not exposed. Creation of a database by tenant administrators is scheduled for inclusion in an administrative interface API in a future release.

This request will create a new named database in the MongoDB root space by a tenant or service administrator.

With pySDK operation:

```
$ t.meta.createDB(db='')
```

With CURL:

```
$ curl -v -X PUT -H "Content-Type:application/json" -H "X-Tapis-Token:$jwt" -d '' $BASE_URL/v3/meta/{db}
```

Here is an example response:

Status: 200 ok

Delete a DB TODO: this implementation is not exposed. Deletion of a database by tenant administrators is scheduled for inclusion in an administrative interface API in a future release.

This request will delete a named database in the MongoDB root space by a tenant or service administrator.

With pySDK operation:

```
$ t.meta.deleteDB(db='')
```

With CURL:

```
$ curl -v -X DELETE -H "Content-Type:application/json" -H "X-Tapis-Token:$jwt" -d '' $BASE_URL/v3/meta/{db}
```

Here is an example response:

Status: 204 Deleted DB

Collection Resources

The Collection resource allows requests for managing and querying json documents within a MongoDB collection.

Create a Collection

You can create a new collection of documents by specifying a collection name under a specific database. /v3/meta/{db}/{collection}

With pySDK operation:

```
$ t.meta.createCollection(db='',collection='')
```

With CURL:

```
$ curl -v -X PUT -H "Content-Type:application/json" -H "X-Tapis-Token:$jwt" $BASE_URL/v3/meta/{db}/{collection}
```

Here is an example response:

Status: 201 Collection created. An Etag header value is returned for collection identification

List Documents

A default number of documents found in the collection are returned in an array of documents.

With pySDK operation:

```
$ t.meta.listDocuments(db='',collection='',filter='')
```

With CURL:

```
$ curl -v -X GET -H "Content-Type:application/json" -H "X-Tapis-Token:$jwt" -d '' $BASE_URL/v3/meta/{db}/{collection}
```

The response will look like the following:

```
[
  {
    "_id": {
      "$oid": "5f1892ece37f7b5a692285e9"
    },
    "name": "test document slt 7.21.2020-14:25",
    "description": "new whatever",
    "_etag": {
      "$oid": "5f1892ec296c81742a6a3e4b"
    }
  },
  {
    "_id": {
      "$oid": "5f1892ece37f7b5a69228533"
    }
  }
]
```

```

    },
    "name": "test document slt 7.21.2020-14:25",
    "description": "new whatever",
    "_etag": {
      "$oid": "5f1892ec296c81742a6a3e444"
    }
  }
]

```

List Documents Large Query

A default number of documents found in the collection are returned in an array of documents.

With pySDK operation:

```

$
t.meta.submitLargeQuery(db='',collection='',page='',pagesize='',soi

```

With CURL:

```

$ curl -v -X GET -H "Content-Type:application/json" -H "X-Tapis-
Token:$jwt" -d @FILENAME '' $BASE_URL/v3/meta/{db}/{collection}/
_filter

```

The response will look like the following:

```

[
  {
    "_id": {
      "$oid": "5f1892ece37f7b5a692285e9"
    },
    "name": "test document slt 7.21.2020-14:25",
    "description": "new whatever",
    "_etag": {
      "$oid": "5f1892ec296c81742a6a3e4b"
    }
  },
  {
    "_id": {
      "$oid": "5f1892ece37f7b5a69228533"
    },
    "name": "test document slt 7.21.2020-14:25",
    "description": "new whatever",
    "_etag": {
      "$oid": "5f1892ec296c81742a6a3e444"
    }
  }
]

```

Delete a Collection

This administrative method is only available to tenant or meta administrators and requires an If-Match header parameter of the Etag for the collection. The Etag value, if not already known, can be retrieved from the "_meta" call for a collection.

With pySDK operation:

```
$ t.meta.deleteCollection(db='',collection='')
```

With CURL:

```
$ curl -v -X DELETE -H "Content-Type:application/json" -H "X-Tapis-Token:$jwt" $BASE_URL/v3/meta/{db}/{collection}
```

Here is an example response:

Status: 204 Deleted named collection from the database

Get Collection Size

You can find the given size or number of documents in a given collection by calling "_size" on a collection.

With pySDK operation:

```
$ t.meta.getCollectionSize(db='',collection='')
```

With CURL:

```
$ curl -v -X GET -H "Content-Type:application/json" -H "X-Tapis-Token:$jwt" $BASE_URL/v3/meta/{db}/{collection}/_size
```

Here is an example response:

Status: 200 ok

```
{  
    "_size": 1 }
```

Get Collection Metadata

You can find the metadata properties of a given collection by calling "_meta" on a collection. This would include the Etag value for a collection that is needed for deletion.

With pySDK operation:

```
$ t.meta.getCollectionMetadata(db='',collection='')
```

With CURL:

```
$ curl -v -X GET -H "Content-Type:application/json" -H "X-Tapis-Token:$jwt" $BASE_URL/v3/meta/{db}/{collection}/_meta
```

Here is an example response:

```
{
  "_id": "_meta",
  "_etag": {
    "$oid": "5f2b2b7a204ce7637579c85f"
  }
}
```

Document Resources

Document resources are json documents found in a collection. Reading, creating, deleting and updating documents along with batch processing make up the operations that can be applied to documents in a collection. There various ways to retrieve one or more documents from a collection, including using a filter query parameter and value in the form of a MongoDB query document. Batch addition of documents, as well as, batch updates based on queries is also allowed.

Create a Document

Creating a new document within a collection. Submitting a json document within the request body of a POST request will create a new document within the specified collection with a MongoDB autogenerated "_id". Batch document addition is possible by POSTing an array of new documents with a request body for the specified collection. The rules for "_id" creation operates the same way on multiple documents as they do with a single document.

The default representation returned is an empty response body along with a 201 Http status code "Created". However if an additional query parameter named "basic" is added with the value of "true" a basic Tapis response is returned along with the newly created "_id" of the document.

With pySDK operation:

```
$ t.meta.createDocument(db='',collection=' ',basic='
',request_body=' ')
```

With CURL:

```
$ curl -v -X POST -H "Content-Type:application/json" -H "X-Tapis-
Token:$jwt" -d '{"docName":"test doc"}' $BASE_URL/v3/meta/{db}/
{collection}
```

Here is an example response:

Status: 201 Document created

Multiple documents can be added to a collection by POSTing a json array of documents. The batch addition of documents only supports the default response.

With pySDK operation:

```
$ t.meta.createDocument(db='',collection='',basic='',  
request_body='')
```

With CURL:

```
$ curl -v -X POST -H "Content-Type:application/json" -H "X-Tapis-  
Token:$jwt" -d '[{"docName":"test doc1"}, {"docName":"test doc2"}]'  
$BASE_URL/v3/meta/{db}/{collection}
```

The response body will be empty:

Status: 201 Document created

Get a Document

Get a specific document by its "_id".

With pySDK operation:

```
$ t.meta.getDocument(db='',collection='',docId='')
```

With CURL:

```
$ curl -v -X GET -H "Content-Type:application/json" -H "X-Tapis-  
Token:$jwt" $BASE_URL/v3/meta/{db}/{collection}/{document_id}
```

The response will be the standard json response:

Status: 200 Document returned

```
{  
  
    "_id":}
```

Replace a Document

This call replaces an existing document identified by document id ("_id"), with the json supplied in the request body.

With pySDK operation:

```
$ t.meta.replaceDocument(db='',collection='',docId='')
```

With CURL:

```
$ curl -v -X PUT -H "Content-Type:application/json" -H "X-Tapis-  
Token:$jwt" -d '{"docName":"test doc another one"}' $BASE_URL/v3/  
meta/{db}/{collection}/{document_id}
```

Here is an example response:

Status: 201 document upserted

Modify a Document

This call will replace a portion of a document identified by document id ("_id") with the supplied json.

With pySDK operation:

```
$ t.meta.modifyDocument(db='',collection='',docId='')
```

With CURL:

```
$ curl -v -X PATCH -H "Content-Type:application/json" -H "X-Tapis-Token:$jwt" -d '{"docName":"test changed"}' $BASE_URL/v3/meta/{db}/{collection}/{document_id}
```

Here is an example response:

Status: 201 document modified

Delete Document

Deleting a document with a specific document id ("_id"), removes it from the collection.

With pySDK operation:

```
$ t.meta.deleteDocument(db='',collection='',docId='')
```

With CURL:

```
$ curl -v -X DELETE -H "Content-Type:application/json" -H "X-Tapis-Token:$jwt" -d '' $BASE_URL/v3/meta/{db}/{collection}/{document_id}
```

Here is an example response:

Status: 204 Document deleted

Index Resources

Indexes can help speed up queries of your collection and the API gives you the ability to define and manage your indexes. You can create an index for a collection, list indexes for a collection and delete an index. Indexes can't be updated they must be deleted and recreated.

List Indexes

List the indexes defined for a collection.

With pySDK operation:

```
$  
t.meta.listIndexes(db='database_name',collection='collection_name')
```

With CURL:

```
$ curl -v -X POST -H "Content-Type:application/json" -H "X-Tapis-Token:$jwt" $BASE_URL/v3/meta/{db}/{collection}/_indexes
```

Here is an example response:

```
[
{
  "v":2,
  "key":{
    "id":1
  }
  "_id":"_id_"
}
]
```

Create Index

Create a new Index with a new name. To create an index you have to specify the keys and the index options. Let's create an unique, sparse index on property qty and name our index "qtyIndex".

```
PUT /v3/meta/{db}/{collection}/_indexes/qtyIndex
```

```
{"keys": {"qty": 1}, "ops": {"unique": true, "sparse": true }}
```

With pySDK operation:

```
$ t.meta.createIndex(db='jupyterhub_v3_metadata',
collection='jupyterhub_dev', request_body={request_body},
indexName='test')
```

With CURL:

```
$ curl -v -X PUT -H "Content-Type:application/json" -H "X-Tapis-Token:$jwt" -d '{ "keys": <keys>, "ops": <options> }' $BASE_URL/v3/meta/{db}/{collection}/_indexes/{indexName}
```

Here is an example response:

Status: 201 Empty Response

Delete Index

Remove a named Index from the index list.

With pySDK operation:

```
$ t.meta.deleteIndex(db='jupyterhub_v3_metadata',
collection='jupyterhub_dev', indexName='test1')
```

With CURL:

```
$ curl -v -X DELETE -H "Content-Type:application/json" -H "X-Tapis-Token:$jwt" $BASE_URL/v3/meta/{db}/{collection}/_indexes/{indexName}
```

Here is an example response:

Status: 204 Deleted index from collection

Aggregation Resources

Aggregations operations process data records and return computed results. Aggregation operations group values from multiple documents together, and can perform a variety of operations on the grouped data to return a single result. Aggregations in the API are predefined and added to a collections properties. They may also be parameterized for use with multiple sets of inputs.

Create an Aggregation

Create an aggregation pipeline by adding the aggregation to the collection for future execution. The aggregation may have variables that are defined so that a future request may pass variable values for aggregation execution. See "Execute an Aggregation".

```
{ "aggrs" : [
  { "stages" : [ { "$match" : { "name" : { "$var" : "n" } } } ],
    { "$group" : { "_id" : "$name",
                  "avg_age" : { "$avg" : "$age" }
                } }
  ],
  "type" : "pipeline",
  "uri" : "example-pipeline"
}
```

Property	Mandatory	Description
type	yes	• for aggregation pipeline operations is "pipeline"
uri	yes	• specifies the URI when the operation is bound under the path /<db>/<collection>/_aggrs.
stages	yes	• the MongoDB aggregation pipeline stages.

For more information refer to <https://docs.mongodb.org/manual/core/aggregation-pipeline/>

With pySDK operation:

```
$ t.meta To Appear in Future Release
```

With CURL:

```
$ curl -v -X PUT -H "Content-Type:application/json" -H "X-Tapis-Token:$jwt" \
  -d '{ "aggrs" : [{ "stages" : [ { "$match" : { "name" :
{ "$var" : "n" } } }, { "$group" : { "_id" : "$name", "avg_age" :
{ "$avg" : "$age" } } } ],
      "type" : "pipeline", "uri" : "example-pipeline" } ] }'
$BASE_URL/v3/meta/{db}/{collection}
```

Here is an example response:

Status: 201 empty response body on created

Execute an Aggregation

TODO

With pySDK operation:

```
$ t.meta To Appear in Future Release
```

With CURL:

```
$ curl -v -X POST -H "Content-Type:application/json" -H "X-Tapis-Token:$jwt" -d '' $BASE_URL/v3/meta/
```

Here is an example response:

TODO

Delete an Aggregation

TODO

With pySDK operation:

```
$ t.meta To Appear in Future Release
```

With CURL:

```
$ curl -v -X POST -H "Content-Type:application/json" -H "X-Tapis-Token:$jwt" -d '' $BASE_URL/v3/meta/
```

Here is an example response:

Status: 204 Empty response body on delete

Jobs

Introduction to Jobs

The Tapis v3 Jobs service is specialized to run containerized applications on any host that supports container runtimes. Currently, Docker, Singularity and ZIP containers are supported. The Jobs service uses the Systems, Apps, Files and Security Kernel services to launch, monitor and manage jobs.

API Endpoints

The following table lists the available API endpoints for Jobs. All UrlPaths shown start with /v3/jobs. The unauthenticated health and ready check APIs do not require a Tapis JWT in the request header.

Name	Method	UrlPath
Submit	POST	/submit
Resubmit	POST	/{{JobUUID}}/resubmit
List	GET	/list
Search	GET	/search
Search	POST	/search
Get	GET	/{{JobUUID}}
Get Status	GET	/{{JobUUID}}/status
Get History	GET	/{{JobUUID}}/history
Get Output list	GET	/{{JobUUID}}/output/list/{{outputPath}}
Download Output	GET	/{{JobUUID}}/output/download/{{outputPath}}
Resubmit Request	GET	/{{JobUUID}}/resubmit_request
Cancel	POST	/{{JobUUID}}/cancel
Hide	POST	/{{JobUUID}}/hide
Unhide	POST	/{{JobUUID}}/unhide
SendEvent	POST	/{{JobUUID}}/sendEvent
PatchAnnotation	PATCH	/{{JobUUID}}/annotation
UpdateAnnotation	PUT	/{{JobUUID}}/annotation

Name	Method	UrlPath
Get Subscriptions	GET	/subscribe/{JobUUID}
Create Subscription	POST	/subscribe/{JobUUID}
Delete Subscriptions	DELETE	/subscribe/{JobUUID}
Post Share	POST	/share/{JobUUID}
Get Share	GET	/share/{JobUUID}
Delete Share	DELETE	/share/{JobUUID}/share/{user}
Health Check	GET	/healthcheck
Ready Check	GET	/readycheck

Job Processing Overview

Before discussing the details of how to construct a job request, we describe the overall lifecycle of a job. When a job request is received as the payload of a POST call, the following steps are taken:

1. **Request authorization** - The tenant, owner, and user values from the request and Tapis JWT are used to authorize access to the application, execution system and, if specified, archive system.
2. **Request validation** - Request values are checked for missing, conflicting or improper values; all paths are assigned; required paths are created on the execution system; and macro substitution is performed to finalize all job parameters.
3. **Job creation** - A Tapis job object is written to the database.
4. **Job queuing** - The Tapis job is placed on an internal queue serviced by one or more Job Worker processes.
5. **Response** - The initial Job object is sent back to the caller in the response. This ends the synchronous portion of job submission.

Once a response to the submission request is sent to the caller, job processing proceeds asynchronously. Job worker processes read jobs from their work queues. The number of workers and queues is limited only by hardware resource constraints. Each job is assigned a worker thread. This thread shepherds a job through its lifecycle until the job completes, fails or becomes blocked due to a transient resource constraint. The job lifecycle is reflected in the [Job Status](#) and generally progresses as follows:

- a) Stage inputs to execution system (PROCESSING_INPUTS, STAGING_INPUTS)
- b) Stage application artifacts to execution system (STAGING_JOB)
- c) Queue or invoke job on execution system (SUBMITTING_JOB)
- d) Monitor job until it terminates (QUEUED, RUNNING)
- e) Collect job exit code (RUNNING)
- f) Archive job output (ARCHIVING)

Simple Job Submission Example

The POST payload for the simplest job submission request looks like this:

```
{
  "name": "myJob"
  "appId": "myApp"
  "appVersion": "1.0"
}
```

In this example, all input and output directories are either specified in the *myApp* definition or are assigned their default values. Currently, the execution system on which an application runs must be specified in either the application definition or job request. Our example assumes that *myApp* assigns the execution system. Future versions of the Jobs service may support dynamic execution system selection.

An archive system can also be specified in the application or job request; the default is for it to be the same as the execution system.

The Job Submission Request

A job submission request must contain the name, appId and appVersion values as shown in the [Simple Job Submission Example](#). Those values are marked *Required* in the list below, a list of all possible values allowed in a submission request. If a parameter has a default value, that value is also shown.

In addition, some parameters can inherit their values from the application or system definitions as discussed in [Parameter Precedence](#). These parameters are marked *Inherit*. Parameters that merge inherited values (rather than override them) are marked *InheritMerge*.

Parameters that do not need to be set are marked *Not Required*. Finally, parameters that allow macro substitution are marked *MacroEnabled* (see [Macro Substitution](#) for details).

name

The user chosen name of the job. *MacroEnabled, Required*.

appId

The Tapis application to execute. *Required*.

appVersion

The version of the application to execute. *Required*.

jobType

A job's type can be either FORK or BATCH.

owner

User ID under which the job runs. Administrators can designate a user other than themselves.

tenant

Tenant of job owner. Default is the tenant of the job owner.

description

Human readable job description. *MacroEnabled, Not Required*

archiveOnAppError

Deprecated. Use archiveMode. Whether archiving should proceed even when the application reports an error. Default is *true*.

archiveMode

Indicates if outputs should be archived after running the job. Default is *ALWAYS*.

execSystemId

Tapis execution system ID. *Inherit*.

execSystemExecDir

Directory into which application assets are staged. *Inherit*, see [Directories](#) for default.

execSystemInputDir

Directory into which input files are staged. *Inherit*, see [Directories](#) for default.

execSystemOutputDir

Directory into which the application writes its output. *Inherit*, see [Directories](#) for default.

execSystemLogicalQueue

Tapis-defined queue that corresponds to a batch queue on the execution system. *Inherit* when applicable.

archiveSystemId

Tapis archive system ID. *Inherit*, defaults to *execSystemId*.

archiveSystemDir

Directory into which output files are archived after application execution. *Inherit*, see [Directories](#) for default.

nodeCount

Number of nodes required for application execution. *Inherit*, default is 1.

coresPerNode

Number of cores to use on each node. *Inherit*, default is 1.

memoryMB

Megabytes of memory to use on each node. *Inherit*, default is 100.

maxMinutes

Maximum number of minutes allowed for job execution. *Inherit*, default is 10.

fileInputs

Input files that need to be staged for the application. *InheritMerge*.

fileInputArrays

Arrays of input files that need to be staged for the application. *InheritMerge*.

parameterSet

Runtime parameters organized by category. *Inherit*.

subscriptions

Subscribe to the job's events. *InheritMerge*.

tags

An array of user-chosen strings that are associated with a job. *InheritMerge*.

notes

A JSON object containing any user-chosen data. *Inherit*.

isMpi

Indicates whether this job is an MPI job. *Inherit*, default is false.

mpiCmd

Specify the MPI launch command. Conflicts with cmdPrefix if isMpi is set. *Inherit*.

cmdPrefix

String prepended to the application invocation command. Conflicts with mpiCmd if isMpi is set. *Inherit*.

notes

Optional JSON object containing arbitrary user data, maximum length 65536 bytes. *Inherit*.

The following subsections discuss the meaning and usage of each of the parameters available in a job request. The [schema](#) and its referenced [library](#) comprise the actual JSON schema definition for job requests.

Parameter Precedence

The runtime environment of a Tapis job is determined by values in system definitions, the app definition and the job request, in low to high precedence order as listed. Generally speaking, for values that can be assigned in multiple definitions, the values in job requests override those in app definitions, which override those in system definitions. There are special cases, however, where the values from different definitions are merged.

See the jobs/apps/systems parameter [matrix](#) for a detailed description of how each parameter is handled.

Job Type

An execution system can run jobs using a batch scheduler (e.g. Slurm) or a native runtime (e.g., Docker or Singularity) or both. Users specify how to run a job using the *jobType* parameter. This can be set to "BATCH" to use a batch scheduler or "FORK" to use a native runtime. The *jobType* can also be specified in application definitions. The final value assigned to the *jobType* of a job is calculated as follows:

1. If the user specifies *jobType* in the job request, use it.
2. Otherwise, if the application defines the *jobType*, use it.
3. Otherwise, if the execution system specifies *canRunBatch* as true set *jobType* to BATCH.
4. Otherwise, set *jobType* to FORK.

Directories

The execution and archive system directories are calculated before the submission response is sent. This calculation can include the use of macro definitions that get replaced by values at submission request time. The [Macro Substitution](#) section discusses what macro definitions are available and how substitution works. In this section, we document the default directory assignments which may include macro definitions.

Directory Definitions

The directories assigned when a system is defined:

rootDir – the effective root of the file system when accessed through this Tapis system.
jobWorkingDir – the default directory for files used or created during job execution.

The directories assigned in application definitions and/or in a job submission requests:

```
execSystemExecDir
execSystemInputDir
execSystemOutputDir
dtnSystemInputDir
dtnSystemOutputDir
archiveSystemDir
```

Directory Assignments

The `rootDir` and `jobWorkingDir` are always assigned upon system creation, so they are available for use as macros when assigning directories in applications or job submission requests.

When a job request is submitted, each of the job's four execution and archive system directories are assigned as follows:

1. If the job submission request assigns the directory, that value is used. Otherwise,
2. If the application definition assigns the directory, that value is used. Otherwise,
3. The default values shown below are assigned:

```
execSystemExecDir:    ${JobWorkingDir}/jobs/${JobUUID}
execSystemInputDir:   ${JobWorkingDir}/jobs/${JobUUID}
execSystemOutputDir:  ${JobWorkingDir}/jobs/${JobUUID}/output
archiveSystemDir:     /jobs/${JobUUID}/archive           (if
archiveSystemId is set)
```

FileInputs

The *fileInputs* in Applications definitions are merged with those in job submission requests to produce the complete list of inputs to be staged for a job. The following rules govern how job inputs are calculated.

1. The effective inputs to a job are the combined inputs from the application and job request.
2. Only named inputs are allowed in application definitions.
3. Application defined inputs are either REQUIRED, OPTIONAL or FIXED.
4. Applications can restrict the number and definitions of inputs (*strictFileInputs=true*).
5. Anonymous (unnamed) inputs can be specified in the job request unless prohibited by the application definition (*strictFileInputs=true*).
6. Job request inputs override values set in the application except for FIXED inputs.
7. The *tapislocal* URL scheme specifies in-place inputs for which transfers are not performed.

The `fileInputs` array in job requests contains elements that conform to the following JSON schema.

```

"JobFileInput": {
  "$comment": "Used to specify file inputs on Jobs submission
requests",
  "type": "object",
  "properties": {
    "name": { "type": "string", "minLength": 1,
"maxLength": 80 },
    "description": { "type": "string", "minLength": 1,
"maxLength": 8096 },
    "envKey": { "type": "string", "minLength": 1},
    "autoMountLocal": { "type": "boolean"},
    "sourceUrl": { "type": "string", "minLength": 1,
"format": "uri"},
    "targetPath": { "type": "string", "minLength": 0},
    "notes": { "type": "string", "minLength": 0}
  },
  "additionalProperties": false
}

```

JobFileInputs can be named or unnamed. When the *name* field is assigned, Jobs will look for an input with the same name in the application definition (all application inputs are named). When a match is found, values from the AppFileInput are merged into unassigned fields in the JobFileInput.

The *name* must start with an alphabetic character or an underscore (`_`) followed by zero or more alphanumeric or underscore characters. If the name does not match one of the input names defined in the application, then the application must have *strictFileInputs=false*. If the name matches an input name defined in the application, then the application's *inputMode* must be REQUIRED or OPTIONAL. An error occurs if the *inputMode* is FIXED and there is a name match--job inputs cannot override FIXED application inputs.

The *envKey* provides an easy way to insert the *targetPath* into the runtime environment of an application under a user-specified label. The *envKey* string is used as the name of an environment variable that Jobs makes accessible to executing applications. The environment variable's value is the *targetPath*. *envKey* can only contain alphanumeric characters and underscore (`_`) and its first character cannot be a number.

The optional *notes* field can contain any valid user-specified JSON object.

Except for in-place inputs discussed below, the *sourceUrl* is the location from which data are copied to the *targetPath*. In Posix systems the *sourceUrl* can reference a file or a directory. When a directory is specified, the complete directory subtree is copied.

Any URL protocol accepted by the Tapis [Files](#) service can be used in a *sourceUrl*. The most common protocols used are `tapis`, `http`, and `https`. The standard `tapis` URL format is `tapis://<tapis-system>/<path>`; please see the [Files](#) service for the complete list of supported protocols.

The *targetPath* is the location to which data are copied from the *sourceUrl*. The target is rooted at the *execSystemInputDir* except, possibly, when `HOST_EVAL()` is used, in which case it is still relative to the execution system's rootDir.

A `JobFileInput` object is **complete** when its *sourceUrl* and *targetPath* are assigned; this provides the minimal information needed to effect a transfer. If only the *sourceUrl* is set, Jobs will use the simple directory or file name from the URL to automatically assign the *targetPath*. Specifying a *targetPath* as "*" results in the same automatic assignment. Whether assigned by the user or Jobs, all job inputs that are not in-place and do not use the `HOST_EVAL()` function are copied into the *execSystemInputDir* subtree.

After application inputs are added to or merged with job request inputs, all complete `JobFileInput` objects are designated for staging. Incomplete objects are ignored only if they were specified as `OPTIONAL` in the application definition. Otherwise, an incomplete input object causes the job request to be rejected.

In-Place Inputs (tapislocal)

Job inputs already present on an execution system do not need to be transferred, yet users may still want to declare them for documentation purposes or to control how they are mounted into containers. It's common, for example, for large data sets that cannot reasonably be copied to be mounted directly onto execution systems. The Jobs and Applications services provide custom syntax that allows such input to be declared, but instructs the Jobs service to **not** copy that input.

Tapis introduces a new URL scheme, *tapislocal*, that is only recognized by the Applications and Jobs services. Here are example URLs:

```
tapislocal://exec.tapis/home/bud/mymri.dcm  
tapislocal://exec.tapis/corral/repl/shared
```

Like the *tapis* scheme and all common schemes (`https`, `sftp`, etc.), the first segment following the double slashes designates a host. For *tapislocal*, the host is always the literal **exec.tapis**, which serves as a placeholder for a job's execution system. The remainder of the URL is the path on the Tapis system. All paths on Tapis systems, including those using the `HOST_EVAL()` function and the *tapislocal* URL, are rooted at the Tapis system's `rootDir`.

A *tapislocal* URL can only appear in the *sourceUrl* field of `AppFileInput` and `JobFileInput` parameters.

The *tapislocal* scheme indicates to Jobs that a filepath already exists on the execution system and, therefore, does not require data transfer during job execution. If *targetPath* is "*", the Jobs service will assign the target path inside the container to be the last segment of the *tapislocal* URL path (`/mymri.dcm` and `/shared` in the examples above).

In container systems that require the explicit mounting of host filepaths, such as Docker, the Jobs service can mount the filepath into the container. Both application definitions and job requests support the *autoMountLocal* boolean parameter. This parameter is true by default, which causes Jobs to automatically mount the filepath into containers. Setting *autoMountLocal* to false allows the user complete control over mounting using a *containerArgs* parameter.

FileInputArrays

The *fileInputArrays* parameter provides an alternative syntax for specifying inputs in [Applications](#) and job requests. This syntax is convenient for specifying multiple inputs destined for the same target directory, an I/O pattern sometimes referred to as *scatter-gather*. Generally, input arrays support the same semantics as [FileInputs](#) with some restrictions.

The *fileInputArrays* parameter in job requests contains elements that conform to the following JSON schema.

```
"JobFileInputArray": {
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "name": { "type": "string", "minLength": 1, "maxLength":
80 },
    "description": { "type": "string", "minLength": 1,
"maxLength": 8096},
    "envKey": { "type": "string", "minLength": 1},
    "sourceUrls": { "type": ["array", "null"],
      "items": { "type": "string", "format":
"uri", "minLength": 1 } },
    "targetDir": { "type": "string", "minLength": 1 },
    "notes": { "type": "string", "minLength": 0 }
  }
}
```

A *fileInputArrays* parameter is an array of *JobFileInputArray* objects, each of which contains an array of *sourceUrls* and a single *targetDir*. One restriction is that *tapislocal* URLs cannot appear in *sourceUrls* fields.

An application's *fileInputArrays* are added to or merged with those in a job request following the same rules established for *fileInputs* in the previous section. In particular, when names match, the *sourceUrls* defined in a job request override (i.e., completely replace) those defined in an application. After merging, each *JobFileInputArray* must have a non-empty *sourceUrls* array. See [FileInputs](#) and [Applications](#) for related information.

Each *sourceUrls* entry is a location from which data is copied to the *targetDir*. In Posix systems each URL can reference a file or a directory. In the latter case, the complete directory subtree is transferred. All URLs recognized by the Tapis [Files](#) service can be used (*tapislocal* is not recognized by Files).

The *targetDir* is the directory into which all *sourceUrls* are copied. The *targetDir* is always rooted at the *ExecSystemInputDir* and if *targetDir* is "*" or not specified, then it is assigned *ExecSystemInputDir*. The simple name of each *sourceUrls* entry is the destination name used in *targetDir*. Use different *JobFileInputArrays* with different *targetDir*'s if name conflicts between *sourceUrls* entries exist.

The *envKey* provides an easy way to insert the *targetDir* into the runtime environment of an application under a user-specified label. The *envKey* string is used as the name of an environment variable that Jobs makes accessible to executing applications. The environment variable's value is the *targetDir*. *envKey* can only contain alphanumerics and underscore (`_`) and its first character cannot be a number.

The optional *notes* field can contain any valid user-specified JSON object.

ParameterSet

The job *parameterSet* argument is comprised of these objects:

Name	JSON Schema Type	Description
appArgs	JobArgSpec array	Arguments passed to user's application
containerArgs	JobArgSpec array	Arguments passed to container runtime
schedulerOptions	JobArgSpec array	Arguments passed to HPC batch scheduler
envVariables	KeyValuePair array	Environment variables injected into application container
archiveFilter	object	File archiving selector
logConfig	LogConfig	User-specified stdout and stderr redirection

Each of these objects can be specified in Tapis application definitions and/or in job submission requests. In addition, the execution system can also specify environment variable settings.

appArgs

Specify one or more command line arguments for the user application using the *appArgs* parameter. Arguments specified in the application definition are appended to those in the submission request.

containerArgs

Specify one or more command line arguments for the container runtime using the *containerArgs* parameter. Arguments specified in the application definition are appended to those in the submission request.

schedulerOptions

Specify HPC batch scheduler arguments for the container runtime using the *schedulerOptions* parameter. Arguments specified in the system *logicalQueue* and the application are merged with those in the job submission request. The arguments for each scheduler are passed using that scheduler's conventions.

Tapis defines a special scheduler option, **--tapis-profile**, to support local scheduler conventions. Data centers sometimes customize their schedulers or restrict how those schedulers can be used. The Systems service manages *SchedulerProfile* resources that are separate from any system definition, but can be referenced from system definitions. The Jobs service uses directives contained in profiles to tailor application execution to local requirements.

As an example, below is the JSON input used to create the TACC scheduler profile.

The *moduleLoads* array contains one or more objects. Each object contains a *moduleLoadCommand*, which specifies the local command used to load each of the modules (in order) in its *modulesToLoad* list.

The *hiddenOptions* array identifies scheduler options that the local implementation prohibits. Options specified here will have the corresponding Slurm option suppressed. Supported options are "MEM" for *--mem* and "PARTITION" for *--partition*. Including an option in the array indicates that the corresponding Slurm option should never be passed through to Slurm.

```
{
  "name": "TACC",
  "owner": "user1",
  "description": "Test profile for TACC Slurm",
  "moduleLoads": [
    {
      "moduleLoadCommand": "module load",
      "modulesToLoad": ["tacc-singularity"]
    }
  ],
  "hiddenOptions": ["MEM"]
}
```

Scheduler-Specific Processing

Jobs will perform macro-substitution on Slurm scheduler options *--job-name* or *-J*. This substitution allows Slurm job names to be dynamically generated before submitting them.

envVariables

Specify key-value pairs that will be injected as environment variables into the application's container when it is launched. Key-value pairs specified in the execution system definition, application definition, and job submission request are aggregated using precedence ordering (system < app < request) to resolve conflicts.

archiveFilter

The *archiveFilter* conforms to this JSON schema:

```
"archiveFilter": {
  "type": "object",
  "properties": {
```



```

    "includes": {"type": "array", "items": {"type": "string",
"minLength": 1}, "uniqueItems": true},
    "excludes": {"type": "array", "items": {"type": "string",
"minLength": 1}, "uniqueItems": true},
    "includeLaunchFiles": {"type": "boolean"}
  },
  "additionalProperties": false
}

```

An *archiveFilter* can be specified in the application definition and/or the job submission request. The *includes* and *excludes* arrays are merged by appending entries from the application definition to those in the submission request.

The *excludes* filter is applied first, so it takes precedence over *includes*. If *excludes* is empty, then no output file or directory will be explicitly excluded from archiving. If *includes* is empty, then all files in *execSystemOutputDir* will be archived unless explicitly excluded. If *includes* is not empty, then only files and directories that match an entry and not explicitly excluded will be archived.

Each *includes* and *excludes* entry is a string, a string with wildcards or a regular expression. Entries represent directories or files. The wildcard semantics are that of *glob ()*, which is commonly used on the command line. *Tapis* implements Java *glob_ semantics*. To filter using a regular expression, construct the pattern using Java *regex_ semantics* and then preface it with *REGEX:** (case sensitive). Here are examples of globs and regular expressions that could appear in a filter:

```

"myfile.*"
"*2021-*--events.log"
"REGEX:^(\\p{IsAlphabetic}\\p{IsDigit}_\\.\\-]+$"
"REGEX:\\s+"

```

When *includeLaunchFiles* is true (the default), then the script (*tapisjob.sh*) and environment (*tapisjob.env*) files that *Tapis* generates in the *execSystemExecDir* are also archived. These launch files provide valuable information about how a job was configured and launched, so archiving them can help with debugging and improve reproducibility. Since these files may contain application secrets, such database passwords or other credentials, care must be taken to not expose private data through archiving.

If no filtering is specified at all, then all files in *execSystemOutputDir* and the launch files are archived.

logConfig Spec

A LogConfig can be supplied in the job submission request and/or in the application definition, with the former overriding the latter when both are supplied. In supported runtimes (currently Singularity and ZIP), the *logConfig* parameter can be used to redirect the application container's stdout and stderr to user-specified files.

MPI and Related Support

On many systems, running Message Passing Interface (MPI) jobs is simply a matter of launching programs that have been configured or compiled with the proper MPI libraries. Most of the work in employing MPI involves parallelizing program logic and specifying the correct libraries for the target execution system. Once that is done, a command such as *mpirun* (or on TACC systems, *ibrun*) is passed the program's pathname and arguments to kick off parallel execution.

Tapis's *mpiCmd* parameter lets users set the MPI launch command in a system definition, application definition and/or job submission request (lowest to highest priority). For example, if *mpiCmd=mpirun*, then the string "mpirun " will be prepended to the command normally used to execute the application. Some MPI launchers have their own parameters, for instance, *mpiCmd=ibrun -n 4* requests 4 MPI tasks.

The *isMpi* parameter is specified in an application definition and/or job request to toggle MPI launching on or off. This switch allows the same system to run both MPI and non-MPI jobs depending on the needs of particular jobs or applications. The *isMpi* default is false, so this switch must be explicitly turned on to run an MPI job. When turned on, *isMpi* requires *cmdMpi* be assigned in the system, application and/or job request.

The *cmdPrefix* parameter provides generalized support for launchers and is available in application definitions and job submission requests. Like *mpiCmd*, a *cmdPrefix* value is simply prepended to a program's pathname and arguments. Being more general, *cmdPrefix* could specify an MPI launcher, but it's not supported in system definitions and does not have a toggle to control usage.

mpiCmd and *cmdPrefix* are mutually exclusive; so if *isMpi* is true, then *cmdPrefix* must not be set.

Subscriptions

Users can subscribe to job execution events. Subscriptions specified in the application definition and those specified in the job request are merged to produce a job's initial subscription list. New subscriptions can be added while a job is running, but not after the job has terminated. A job's subscriptions can be listed and deleted. Only job owners or tenant administrators can subscribe to a job, see the [subscription APIs](#) for details.

When creating a subscription the *ttlminutes* parameter can specify up to 4 weeks. If the parameter is not specified or if it's set to 0, a default value of 1 week is used.

Subscribers are notified of job events by the [Notifications](#) service. Currently, only email and webhook delivery methods are supported. The event types to which users can subscribe are:

Event Type	Description
JOB_NEW_STATUS	When the job transitions to a new status
JOB_INPUT_TRANSACTION_ID	When an input file staging request is made

Event Type	Description
JOB_ARCHIVE_TRANSACTION_ID	When an archive file transfer request is made
JOB_SUBSCRIPTION	When a change to the job's subscriptions is made
JOB_SHARE_EVENT	When a job resource has been shared or unshared
JOB_ERROR_MESSAGE	When the job experienced an error
JOB_USER_EVENT	When a user sends the job a custom event
ALL	When any of the above occur

All event types other than JOB_USER_EVENT are generated by Tapis. See [Notification Messages](#) for a description of what Jobs returns for each of the Tapis-generated event.

A JOB_USER_EVENT contains a user-specified payload that targets an active job by using the job's UUID. The sender must be in the same tenant as the job to inject custom events into the job's event stream. The event payload must contain a JSON key named *eventData* with a string value of at least 1 character and no more than 16,384 characters. The string can be unstructured or structured (such as a JSON object) as determined by the sender. The payload can optionally contain an *eventDetail* key with a string value of no more than 64 characters. This key is used to further categorize events and, if not provided, will default to "DEFAULT". User events are always added to the job history and notifications are sent to subscribers interested in those events.

Request Validation

Jobs detects invalid request input early to streamline application debugging and to guard against improper execution. In particular, Tapis usually double quotes strings that contain *dangerous* characters when those strings will appear on a command line. The set of command line dangerous characters is { **&**, **>**, **<**, **|**, **`**, **;** }, all of which have special meaning when issued in a shell.

String validation on job submission requests is as follows:

- The request must conform to the job submission JSON [schema](#).
- Strings **rejected** if they contain *ISO control characters* (0x00-0x1F and 0x7f-0x9f, which include tab, new line and carriage return):
 - job name
 - environment variables
 - system profile name
 - scheduler options
 - container arguments
 - application arguments
 - path names
 - URLs
- Strings **rejected** if they contain *dangerous characters*:
 - scheduler option names
 - container argument names

- application arguments
- environment variable keys
- mpi command
- system ids
- queue names
- log file names
- job owner
- job tenant
- Strings **double quoted** if they contain *space characters* or *dangerous characters*:
 - path names
 - scheduler options
 - container arguments
 - environment variable values

Note

Application arguments can contain spaces. Tapis, however, does not automatically double quote these arguments since applications define their own command line conventions. *Therefore, it is the responsibility of the job submitter to use single quotes or escaped double quotes (\") if so desired.*

Shared Components

JobArgSpec

Simple argument strings can be specified in the system *logicalQueue* and the application (AppArgSpec) as well as a job submission request (JobArgSpec). These argument strings are passed to specific components in the runtime system, such as the batch scheduler (schedulerOptions), the container runtime (containerArgs) or the user's application (appArgs). Note for the system *logicalQueue* only schedulerOptions may be defined.

The following rules govern how job arguments are calculated.

1. All argument in application definitions must be named.
2. Application arguments are either REQUIRED, FIXED or one of two optional types.
3. Anonymous (unnamed) argument can be specified in job requests.
4. Job request argument override values set in the application except for FIXED arguments.
5. The final argument ordering is the same as the order specified in the definitions, with application arguments preceding those from the job request. Application arguments maintain their place even when overridden in the job request.
6. The notes field can be any JSON object, i.e., JSON that begins with a brace (" {").

We define a **complete** AppArgSpec as one that has a non-empty name and arg value. We define a **complete** JobArgSpec as one that has a non-empty arg value. A JobArgSpec with the same name as an AppArgSpec inherits from the application and may override the AppArgSpec values.

This is the JSON schema used to define runtime arguments in [ParameterSet](#).

```
"JobArgSpec": {
  "$comment": "Used to specify parameters on Jobs submission requests",
  "type": "object",
  "properties": {
    "name": { "type": "string", "minLength": 1,
"maxLength": 80 },
    "description": { "type": "string", "minLength": 1,
"maxLength": 8096 },
    "include": { "type": "boolean" },
    "arg": { "type": "string", "minLength": 1},
    "notes": { "type": "object" }
  },
  "required": ["arg"],
  "additionalProperties": false
}
```

As mentioned, the JobArgSpec is used in conjunction with the AppArgSpec defined in [Applications](#). Arguments in application definitions are merged into job request arguments using the same name matching algorithm as in [FileInputs](#).

The *name* identifies the input argument. If present, the name must start with an alphabetic character or an underscore (`_`) followed by zero or more alphanumeric or underscore characters.

The *description* is used to convey usage information to job requester. If both application and request descriptions are provided, then the request description is appended as a separate paragraph to the application description.

The required *arg* value is an arbitrary string and is used as-is. If this argument's name matches that of an application argument, this *arg* value overrides the application's value except when *inputMode*=*FIXED* in the application.

The *include* field applies only on named arguments that are also defined in the application definition with *inputMode* INCLUDE_ON_DEMAND or INCLUDE_BY_DEFAULT; this parameter is ignored on all other inputModes. Argument inclusion is discussed in greater detail in following subsection.

Argument Processing

[Applications](#) use their AppArgSpecs to pass default values to job requests. The AppArgSpec's *inputMode* determines how to handle arguments during job processing. An *inputMode* field can have these values:

REQUIRED

The argument must be provided for the job to run. If an arg value is not specified in the application definition, then it must be specified in the job request. When provided in both, the job request arg value overrides the one in application.

FIXED

The argument is completely defined in the application and not overridable in a job request.

INCLUDE_ON_DEMAND

The argument, if complete, will only be included in the final argument list constructed by Jobs if it's explicitly referenced and included in the Job request. This is the default value.

INCLUDE_BY_DEFAULT

The argument, if complete, will automatically be included in the final argument list constructed by Jobs unless explicitly excluded in the Job request.

The truth table below defines how the AppArgSpec's *inputMode* and JobArgSpec's *include* settings interact to determine whether an argument is accepted or ignored during job processing.

AppArgSpec <i>inputMode</i>	JobArgSpec <i>include</i>	Meaning
INCLUDE_ON_DEMAND	True	include arg
INCLUDE_ON_DEMAND	False	exclude arg
INCLUDE_ON_DEMAND	undefined	include arg
INCLUDE_BY_DEFAULT	True	include arg
INCLUDE_BY_DEFAULT	False	exclude arg
INCLUDE_BY_DEFAULT	undefined	include arg

The JobArgSpec *include* value has no effect on REQUIRED or FIXED arguments. In the cases where the value does apply, not specifying *include* in a named JobArgSpec that matches an AppArgSpec is effectively the same as setting *include=True*. By setting *include=False*, a JobArgSpec can exclude any INCLUDE_ON_DEMAND or INCLUDE_BY_DEFAULT arguments.

KeyValuePair

The JSON schema for defining key-value pairs of strings in various ParameterSet components is below.

```
"KeyValuePair": {
  "$comment": "A simple key-value pair",
  "type": "object",
  "properties": {
    "key": { "type": "string", "minLength": 1 },
    "value": { "type": "string", "minLength": 0 },
    "description": { "type": "string", "minLength": 1 },
```

```

"maxLength": 8096},
    "include": { "type": "boolean" },
    "notes": { "type": "object" }
  },
  "required": ["key", "value"],
  "additionalProperties": false
}

```

Both the *key* and *value* are required, though the *value* can be an empty string. Descriptions are optional but if present must contain 1 or more characters.

The *include* field applies only to attributes that are defined in application and system definitions with an *inputMode* of `INCLUDE_ON_DEMAND` or `INCLUDE_BY_DEFAULT`; this parameter is ignored for all other inputModes. May be used in a job submit request to exclude attributes that have *inputMode=INCLUDE_BY_DEFAULT*. The *notes* field can be any JSON object, i.e., JSON that begins with a brace (`{`).

LogConfig

The JSON schema for used to redirect stdout and stderr to named file(s) in supported runtimes.

```

"logConfig": {
  "$comment": "Log file redirection and customization in
supported runtimes",
  "type": "object",
  "required": [ "stdoutFilename", "stderrFilename" ],
  "additionalProperties": false,
  "properties": {
    "stdoutFilename": {"type": "string", "minLength": 1},
    "stderrFilename": {"type": "string", "minLength": 1}
  }
}

```

Currently, only the Singularity (Apptainer) and ZIP runtimes are supported. For the Docker runtime, applications can redirect their *stdout* and *stderr* streams to a file (or files) in the *execSystemOutputDir* using an environment variable listed in the next section. Jobs will treat this redirected output like all other application generated output for later retrieval.

When specified, both file name fields must be explicitly assigned, though they can be assigned to the same file. If a *logConfig* object is not specified, or in runtimes where it is not supported, then both stdout and stderr are directed to the default **tapisjob.out** file in the job's output directory. Output files, even when *logConfig* is used, are always relative to the *ExecSystemOutputDir* (see [Directory Definitions](#)).

Job Execution

Environment Variables

The following standard environment variables are passed into each application container run by Tapis as long as they have been assigned a value.

- `_tapisAppId` – Tapis app ID
- `_tapisAppVersion` – Tapis app version
- `_tapisArchiveOnAppError` – true means archive even if the app returns a non-zero exit code
- `_tapisArchiveMode` – ALWAYS, SKIP_ON_FAIL or NEVER
- `_tapisArchiveSystemDir` – the archive system directory on which app output is archived
- `_tapisArchiveSystemId` – Tapis system used for archiving app output
- `_tapisCoresPerNode` – number of cores used per node by app
- `_tapisDtnSystemId` – the Data Transfer Node system ID
- `_tapisDtnSystemInputDir` – the directory on the DTN to which input files will be transferred during staging
- `_tapisDtnSystemOutputDir` – the directory on the DTN from which output files will be transferred during archiving
- `_tapisDynamicExecSystem` – true if dynamic system selection was used
- `_tapisEffectiveUserId` – the user ID under which the app runs
- `_tapisExecSystemExecDir` – the exec system directory where app artifacts are staged
- `_tapisExecSystemHPCQueue` – the actual batch queue name on an HPC host
- `_tapisExecSystemId` – the Tapis system where the app runs
- `_tapisExecSystemInputDir` – the exec system directory where input files are staged
- `_tapisExecSystemLogicalQueue` – the Tapis queue definition that specifies an HPC queue
- `_tapisExecSystemOutputDir` – the exec system directory where the app writes its output
- `_tapisStdoutFilename` – Name of file to use for stdout.
- `_tapisStderrFilename` – Name of file to use for stderr.
- `_tapisJobCreateDate` – ISO 8601 date, example: 2021-04-26Z
- `_tapisJobCreateTime` – ISO 8601 time, example: 18:44:55.544145884Z
- `_tapisJobCreateTimestamp` – ISO 8601 timestamp, example: 2021-04-26T18:44:55.544145884Z
- `_tapisJobName` – the user-chosen name of the Tapis job
- `_tapisJobOwner` – the Tapis job's owner
- `_tapisJobUUID` – the UUID of the Tapis job
- `_tapisJobWorkingDir` – exec system directory that the app should use for temporary files
- `_tapisMaxMinutes` – the maximum number of minutes allowed for the job to run
- `_tapisMemoryMB` – the memory required per node by the app
- `_tapisNodes` – the number of nodes on which the app runs
- `_tapisStderrFilename` – the file into which the application's

`stderr` is piped
`_tapisStdoutFilename` – the file into which the application's `stdout` is piped
`_tapisSysBatchScheduler` – the HPC scheduler on the execution system
`_tapisSysBucketName` – an object store bucket name
`_tapisSysHost` – the IP address or DNS name of the exec system
`_tapisSysRootDir` – the root directory on the exec system
`_tapisTenant` – the tenant in which the job runs

Macro Substitution

Tapis defines macros or template variables that get replaced with actual values at well-defined points during job creation. The act of replacing a macro with a value is often called macro substitution or macro expansion. The complete list of Tapis macros can be found at [JobTemplateVariables](#).

There is a close relationship between these macro definitions and the Tapis environment variables just discussed: Macros that have values assigned are passed as environment variables into application containers. This makes macros used during job creation available to applications at runtime.

Most macro definitions are *ground* definitions because their values do not depend on any other macros. On the other hand, *derived* macro definitions can include other macro definitions. For example, in [Directory Assignments](#) the default input file directory is constructed with two macro definitions:

```
execSystemInputDir = ${JobWorkingDir}/jobs/${JobUUID}
```

Macro values are referenced using the `${Macro-name}` notation. Since derived macro definitions reference other macros, there is the possibility of circular references. Tapis detects these errors and aborts job creation.

Below is the complete, ordered list of derived macros. Each macro in the list can be defined using any ground macro and any macro that precedes it in the list. Result are undefined if a derived macro references a macro that follows it in the derived list.

1. JobName
2. JobWorkingDir
3. ExecSystemInputDir
4. ExecSystemExecDir
5. ExecSystemOutputDir
6. ArchiveSystemDir
7. SchedulerOptions
8. AppArgs
9. ContainerArgs
10. LogConfig

Finally, macro substitution is applied to the job *description* field, whether the description is specified in an application or a submission request.

Macro Functions

Directory assignments in systems, applications and job requests can also use the **HOST_EVAL(\$var)** function at the beginning of their path assignments. This function dynamically extracts the named environment variable's value from an execution or archive host *at the time the job request is made*. Specifically, the environment variable's value is retrieved by logging into the host as the Job owner and issuing "echo \$var". The example in [Data Transfer Nodes](#) uses this function.

To increase application portability, an optional default value can be passed into the **HOST_EVAL** function. The function's complete signature with the optional path parameter is:

```
| HOST_EVAL($VAR, path)
```

If the environment variable VAR does not exist on the host, then the literal path parameter is returned by the function. This added flexibility allows applications to run in different environments, such as on TACC HPC systems that automatically expose certain environment variables and VMs that might not. If the environment variable does not exist and no optional path parameter is provided, the job fails due to invalid input.

Job Status

The list below contains all possible states of a Tapis job, which are indicated in the *status* field of a job record. The initial state is PENDING. Terminal states are FINISHED, CANCELLED and FAILED. The BLOCKED state indicates that the job is recovering from a resource constraint, network problem or other transient problem. When the problem clears, the job will restart from the state in which blocking occurred. :

PENDING – Job processing beginning
PROCESSING_INPUTS – Identifying input files for staging
STAGING_INPUTS – Transferring job input data to execution system
STAGING_JOB – Staging runtime assets to execution system
SUBMITTING_JOB – Submitting job to execution system
QUEUED – Job queued to execution system queue (when jobType is BATCH)
RUNNING – Job running on execution system
ARCHIVING – Transferring job output to archive system
BLOCKED – Job blocked
PAUSED – Job processing suspended
FINISHED – Job completed successfully
CANCELLED – Job execution intentionally stopped
FAILED – Job failed

Normal processing of a successfully executing job proceeds as follows:

PENDING->PROCESSING_INPUTS->STAGING_INPUTS->STAGING_JOB->
SUBMITTING_JOB->
QUEUED->RUNNING->ARCHIVING->FINISHED

Notification Messages

Notifications are the messages sent to subscribers who have registered interest in certain job events. In this section, we specify the messages sent to subscribers for event generated by the Jobs service. See [Subscriptions](#) for an introduction to the different event types, including user generated events (type JOB_USER_EVENT), which are not discussed here.

For events generated by the Jobs service, the *data* field in notification messages received by subscribers contains a JSON object that always include these fields:

- *jobName* - the user-specified job name
- *jobOwner* - the user who submitted the job
- *jobUuid* - the unique job ID
- *message* - a human readable message

Each of the Job event types also include additional fields as shown:

Job Event Type	Additional Fields
JOB_NEW_STATUS	newJobStatus, oldJobStatus
JOB_INPUT_TRANSACTION_ID	transferStatus, transactionId
JOB_ARCHIVE_TRANSACTION_ID	transferStatus, transactionId
JOB_SUBSCRIPTION	action, numSubscriptions
JOB_SHARE_EVENT	resourceType, shareType, grantee, grantor
JOB_ERROR_MESSAGE	jobStatus

Additionally, when either of these conditions hold:

1. JOB_NEW_STATUS messages indicate a **terminal** *newJobStatus*, or
2. JOB_ERROR_MESSAGE messages have *eventDetail* = "FINAL_MESSAGE",

then the following additional fields are included in the notification:

- *blockedCount* - the number of times the job blocked (JSON number)
- *condition* - a code that describes the job's final disposition (see [ConditionCodes](#))
- *remoteJobId* - execution system job id (ex: pid, slurm id, docker hash, etc.)
- *remoteJobId2* - execution system auxilliary id associated with a job
- *remoteOutcome* - FINISHED, FAILED, FAILED_SKIP_ARCHIVE
- *remoteResultInfo* - application exit code
- *remoteQueue* - execution system scheduler queue
- *remoteSubmitted* - time job was submitted on remote system
- *remoteStarted* - time job started running on remote system
- *remoteEnded* - time job stopped running on remote system

Job terminal statuses are FINISHED, CANCELLED and FAILED.

Job Condition Codes on Termination

When a job terminates, in addition to being assigned a terminal status, a job is assigned one of the following condition codes. :

CANCELLED_BY_USER - Job cancelled by user
JOB_ARCHIVING_FAILED - Job error while archiving outputs
JOB_DATABASE_ERROR - Jobs is unable to access its database
JOB_EXECUTION_MONITORING_ERROR - An error occurred during application monitoring
JOB_EXECUTION_MONITORING_ERROR_TIMEOUT - Continuous monitor errors exceeded allowed time
JOB_EXECUTION_MONITORING_TIMEOUT - Job time expired during execution monitoring
JOB_FILES_SERVICE_ERROR - An error involving the Files service occurred
JOB_INTERNAL_ERROR - Jobs service internal error
JOB_INVALID_DEFINITION - Invalid job record
JOB_LAUNCH_FAILURE - Tapis is unable to launch job
JOB_QUEUE_MONITORING_ERROR - An error occurred during application queue monitoring
JOB_RECOVERY_FAILURE - Tapis unable to recover job
JOB_RECOVERY_TIMEOUT - Tapis recovery time expired
JOB_REMOTE_ACCESS_ERROR - Jobs could not access a resource on a remote system
JOB_REMOTE_OUTCOME_ERROR - User application returned a non-zero exit code
JOB_UNABLE_TO_STAGE_INPUTS - Unable to stage application input files
JOB_UNABLE_TO_STAGE_JOB - Unable to stage application assets
JOB_TRANSFER_FAILED_OR_CANCELLED - A file transfer failed or was cancelled
JOB_TRANSFER_MONITORING_TIMEOUT - Jobs transfer monitoring expired
NORMAL_COMPLETION - Job completed normally
SCHEDULER_CANCELLED - Batch scheduler cancelled job
SCHEDULER_DEADLINE - Batch scheduler deadline exceeded
SCHEDULER_OUT_OF_MEMORY - Batch scheduler out of memory error
SCHEDULER_STOPPED - Batch scheduler stopped job
SCHEDULER_TIMEOUT - Batch scheduler timed out job
SCHEDULER_TERMINATED - Batch scheduler terminated job

Codes that begin with "JOB" indicate Jobs service conditions. Those that begin with "SCHEDULER" indicate a condition reported by a batch scheduler. The CANCELLED_BY_USER condition results from direct user action. Jobs that successfully execute are assigned the code NORMAL_COMPLETION.

Data Transfer Nodes

Introduction

The Jobs, Applications, Files and Systems services cooperate to allow the configuration and execution of jobs. In this section, we discuss the use of a high throughput **Data Transfer Node** (DTN) to stage job inputs and archive outputs. DTNs are used to optimize transfer speeds and to allow nodes to be separately optimized for either storage or job execution. For example, users often stage input data to long-term storage (the DTN), such as *cloud.corral* at TACC, and then copy that data to a fast, temporary file system, such as a "scratch" file system, when running jobs. DTNs are also used when specialized transfer software, such as GLOBUS, is required on nodes to receive data, but those nodes are not appropriate to run jobs. In this case, the DTN receives the data from a remote location and then another transfer copies them from the DTN to an execution system.

The DTN usage pattern is effective when:

1. The DTN has high performance network and storage capabilities,
2. the target system for data transfers and the DTN system have shared storage, and
3. all file inputs or all file outputs are to be staged through the DTN.

In this situation, the data transfers performed by Jobs benefit from the DTN's high performance I/O capabilities, while applications can still access their input from systems optimized for execution.

Configuring Tapis for DTN Usage

In order to run a job and use a DTN for file input staging or output archiving, the execution system and the application must be properly configured. There are also a few rules to follow and requirements that must be satisfied.

Configuring the Execution and DTN Systems

The execution system must have the attribute *dtnSystemId* set to the Tapis system that can be used as a DTN. Note that use of the DTN is optional. Use of a DTN is triggered by the setting of *dtnSystemInputDir* or *dtnSystemOutputDir* in either the job submission request or the application definition.

As listed below under *Rules and Requirements*, the execution system and the DTN system must have the same *rootDir*, must have shared storage, and the *rootDir* must be part of the shared storage.

Configuring the Application

There are two relevant attributes in an Application definition, *dtnSystemInputDir* and *dtnSystemOutputDir*. By default, these are set to the special value *tapis_not_set*. This value indicates that, by default, a DTN system will not be used for file inputs or archive outputs. In order to trigger the use of a DTN during either file input staging or archive output, these values must be set:

dtnSystemInputDir

Directory relative to *rootDir* to which input files will be transferred prior to launching the application.

dtnSystemOutputDir

Directory relative to *rootDir* from which output files will be transferred during the archiving phase.

Note that these attributes may be overridden by the job submission request. Please see the next section.

Constructing the Job Submission Request

In a job submission request, the values for *dtnSystemInputDir* or *dtnSystemOutputDir* can be set in order to override the values defined by the application. They can be set to *!tapis_not_set* to turn off DTN usage or set to an alternate path.

DTN Rules and Requirements

Certain rules and requirements need to be observed to correctly use a DTN.

- **Rule:** All file inputs or archive outputs will go through the DTN.
- **Rule:** Any HOST_EVAL expressions or macro substitutions (such as *effectiveUserId*) used in *dtnSystemInputDir* or *dtnSystemOutputDir* will be evaluated in the context of the execution system.
- **Rule:** Each job should use its own *dtnSystemInputDir* or *dtnSystemOutputDir* to avoid data from different jobs being mixed, overwritten or deleted before being used.
- **Requirement:** The execution system and the DTN system must have shared storage.
 - For example, there could be an NFS mount on the execution system pointing to the DTN host, or the DTN may be a Globus endpoint running on the execution host.
 - Note that for an HPC cluster it could be that the shared storage is set up on login nodes and not on compute nodes. This case is supported since the Tapis Jobs service typically uses a login node when executing an application.
- **Requirement:** Restrictions on *rootDir*
 - The **rootDir** for the execution system and the DTN system **must be same**. This is validated by the Systems service when the execution system is created or updated.
 - The **rootDir** must be part of the shared storage.
 - The absolute path to the shared storage must be the same on both the execution and DTN systems. This is because:
 - **Rule:** When the Tapis Jobs service moves data to or from the DTN system, it assumes no path translation is needed.
- **Requirement:** For any file inputs defined for the job, Tapis must support file transfers from the input source to the DTN system.
 - For example, if the source for the file input is a system of type GLOBUS, then the DTN must also be of type GLOBUS.

- **Requirement:** Credentials must be registered for the *effectiveUserId* for each system involved.
- **Requirement:** For each system involved, the *effectiveUserId* for the system must have appropriate Tapis and native file system access for the paths defined.
 - For the execution and DTN systems, the *effectiveUserId* must have READ/ WRITE access for both *dtnSystemInputDir* and *dtnSystemOutputDir*.

Modified Jobs Service Behavior

When an application runs and the execution system specifies a DTN, the Jobs service will behave as follows:

During staging of input files

If the execution system specifies a DTN and *dtnSystemInputDir* is set:

- The source systems and paths defined in FileInputs are unchanged.
- The target system for the transfer request sent to the Files service will be the DTN as specified in the execution system definition.
- The target path will be *\${DtnSystemInputDir}*.
- The staged input files will be *moved* from *\${DtnSystemInputDir}* to *\${ExecSystemInputDir}* on the execution system.
 - Note that the fully resolved absolute path to *\${DtnSystemInputDir}* must be **the same** on both the execution system and the DTN system. If this is not true the operation may fail.
 - The *move* operation deletes the files from *\${DtnSystemInputDir}*.

During archiving of output files

If the execution system specifies a DTN and *dtnSystemOutputDir* is set:

- The output files generated on the execution system are *moved* from *\${ExecSystemOutputDir}* to *\${DtnSystemOutputDir}*
 - Note that the fully resolved absolute path to *\${DtnSystemOutputDir}* must be **the same** on both the execution system and the DTN system. If this is not true the operation may fail.
 - The *move* operation deletes the files from *\${ExecSystemOutputDir}*.
- The source system for the transfer request sent to the Files service will be the DTN as specified in the execution system definition.
- The source path will be *\${DtnSystemOutputDir}*.
- The target system and path as defined in the archive definition is unchanged.

Example DTN Configuration and Usage

We provide the examples below to illustrate DTN configuration and usage.

DTN system definition:

```
{
  "id":"example-dtn",
  "host":"dtn.host.example.com",
  "systemType":"LINUX",
```

```

    "effectiveUserId": "testuser1",
    "defaultAuthnMethod": "PKI_KEYS",
    "rootDir": "/",
    "canExec": false
}

```

Execution system definition:

```

{
  "id": "example-exec-with-dtn",
  "host": "exec.host.example.com",
  "systemType": "LINUX",
  "effectiveUserId": "testuser1",
  "defaultAuthnMethod": "PKI_KEYS",
  "rootDir": "/",
  "canExec": true
  "canRunBatch": true,
  "jobRuntimes": [ { "runtimeType": "SINGULARITY" } ],
  "jobWorkingDir": "HOST_EVAL($SCRATCH)/dtn_test/jobs",
  "dtnSystemId": "example-dtn",
  "batchScheduler": "SLURM",
  "batchLogicalQueues": [
    {
      "name": "tapisNormal",
      "hpcQueueName": "skx-normal",
      "maxJobs": 50,
      "maxJobsPerUser": 10,
      "minNodeCount": 1,
      "maxNodeCount": 16,
      "minCoresPerNode": 1,
      "maxCoresPerNode": 68,
      "minMemoryMB": 1,
      "maxMemoryMB": 16384,
      "minMinutes": 1,
      "maxMinutes": 60
    }
  ],
  "batchDefaultLogicalQueue": "tapisNormal",
  "batchSchedulerProfile": "tacc"
}

```

Application definition:

```

{
  "id": "example-app-with-dtn",
  "version": "1",
  "containerImage": "example/example-app:1",
  "jobType": "BATCH",
  "runtime": "SINGULARITY",
  "jobAttributes": {
    "description": "Transfer file and verify transfer",
    "execSystemId": "example-exec-with-dtn",
    "execSystemExecDir": "${JobWorkingDir}/${JobUUID}",
    "execSystemInputDir": "${JobWorkingDir}/${JobUUID}/inputs",

```



```

    "execSystemOutputDir": "${JobWorkingDir}/${JobUUID}/output",
    "dtnSystemInputDir": "/shared/storage/stage_inputs/${JobUUID}",
    "dtnSystemOutputDir": "/shared/storage/stage_outputs/${JobUUID}",
    "archiveSystemId": "archive-storage-system",
    "archiveSystemDir": "/archive/storage/${JobUUID}",
    "fileInputs": [
      {
        "name": "file1",
        "description": "A text file",
        "inputMode": "REQUIRED",
        "sourceUrl": "tapis://example-data-repo/data/file1.txt",
        "targetPath": "file1.txt"
      }
    ]
  }
}

```

Job submission request:

```

{
  "name": "Tapis V3 example job using a DTN",
  "appId": "example-app-with-dtn",
  "appVersion": "0.0.1",
  "parameterSet": {
    "schedulerOptions": [ { "arg": "-A EXAMPLE-ALLOCATION" } ]
  }
}

```

Note that the execution system specifies the DTN system and the application specifies the DTN directories:

dtnSystemId

The Tapis system to use as a DTN.

dtnSystemInputDir

Directory relative to *rootDir* to which input files will be transferred prior to launching the application.

dtnSystemOutputDir

Directory relative to *rootDir* from which output files will be transferred during the archiving phase.

Container Runtimes

The Tapis v3 Jobs service supports Docker, Singularity and ZIP containers run natively (i.e., not run using a batch scheduler like Slurm). In general, Jobs launches an application's container on a remote system, monitors the container's execution,

and captures the application's exit code after it terminates. Jobs uses SSH to connect to the execution system to issue Docker, Singularity or native operating system commands.

To launch a job, the Jobs service creates a bash script, **tapisjob.sh**, with the runtime-specific commands needed to execute the container. This script references **tapisjob.env**, a file Jobs creates to pass environment variables to application containers. Both files are staged in the job's `execSystemExecDir` and, by default, are archived with job output on the archive system. See [archiveFilter](#) to override this default behavior, especially if archives will be shared and the scripts pass sensitive information into containers.

Docker

To launch a Docker container, the Jobs service will SSH to the target host and issue a command using this template:

```
docker run [docker options] image[:tag|@digest] [application args]
```

1. docker options: (optional) user-specified arguments passed to docker
2. image: (required) user-specified docker application image
3. application arguments: (optional) user-specified command line arguments passed to the application

The docker [run-command](#) options `--cidfile`, `-d`, `-e`, `--env`, `--name`, `--rm`, and `--user` are reserved for use by Tapis. Most other Docker options are available to the user. The Jobs service implements these calling conventions:

1. The container name is set to the job UUID.
2. The container's user is set to the user ID used to establish the SSH session.
3. The container ID file is specified as `<JobUUID>.cid` in the `execSystemExecDir`, i.e., the directory from which the container is launched.
4. The container is removed after execution using the `-rm` option or by calling `docker rm`.

Logging

Logging should be considered up front when defining Tapis applications to run under Docker. Since Jobs removes Docker containers after they execute, the container's log is lost under the default Docker [logging](#) configuration. Typically, Docker pipes `stdout` and `stderr` to the container's log, which requires the application to take deliberate steps to preserve these outputs.

An application can maintain control over its log output by logging to a file outside of the container. The application can do this by redirecting `stdout` and `stderr` or by explicitly writing to a file. As discussed in [dir-definitions](#), the application always has read/write access to the host's `execSystemOutputDir`, which is mounted at `/TapisOutput` in the container (see next section).

On the other hand, applications can run on machines where the default Docker log driver is configured to write to files or services outside of containers. In addition, Tapis passes any user-specified *log-driver* and *log-opts* options to *docker run*, so all customizations supported by Docker are possible.

Volume Mounts

In addition to the above conventions, bind mounts are used to mount the execution system's standard Tapis directories at the same locations in every application container.

```
execSystemExecDir    on host is mounted at /TapisExec in the
container.
execSystemInputDir   on host is mounted at /TapisInput in the
container.
execSystemOutputDir  on host is mounted at /TapisOutput in the
container.
```

Singularity

Tapis uses *singularity run* to launch a Singularity container.

Warning

Please note that use of the SINGULARITY_START runtime option to use *singularity instance start* for launching a container has been removed. If you have a need for this option please contact Tapis support (cicsupport@tacc.utexas.edu) about an enhancement request.

Singularity Launching

Jobs supports running containers on the remote system using *singularity run*. To launch a container, the Jobs service will SSH to the target host and issue a command using this template:

```
nohup singularity run [singularity options] <image id>
[application arguments] > tapisjob.out 2>&1 &
```

where:

1. nohup: allows the background process to continue running even if the SSH session ends.
2. *singularity options*: (optional) user-specified arguments passed to *singularity run*.
3. *image id*: (required) user-specified singularity application image.
4. *application arguments*: (optional) user-specified command line arguments passed to the application.
5. *redirection*: stdout and stderr are redirected to **tapisjob.out** in the job's output directory.

The singularity `--env` option is reserved for use by Tapis. Users can specify the environment variables to be injected into their application containers via the `envVariables` parameter. Most other singularity options are available to users.

Jobs will use the PID returned when issuing the background command to monitor the container's execution. Jobs determines that the application has terminated when the PID is no longer in use by the operating system.

Jobs uses the same **TapisJob.exitcode** file convention introduced above to attain the application's exit code (if the file exists).

Singularity Monitoring

The Tapis approach to launching the singularity container allows SSH sessions between Jobs and execution hosts to end without interrupting container execution. For proper monitoring, however, the application image must be appropriately constructed. Specifically, the runscript should be defined in the image.

Since Jobs monitors container execution by querying the operating system using the PID obtained at launch time, the initially launched program should be the last part of the application to terminate. The program specified in the image script can spawn any number of processes (and threads), but it should not exit before those processes complete.

Optional Exit Code Convention

Applications are not required to support the **TapisJob.exitcode** file convention as described above, but it is the only way in which Jobs can report the application specified exit status to the user.

ZIP

Standard archive files, such as zip and compressed tar, can be treated as a type of custom *application image* and launched in a runtime environment defined by Tapis. This *ZIP runtime environment* maximizes application flexibility while retaining much of the reproducibility and automation benefits of Tapis.

To define a ZIP application, specify "ZIP" as the *runtime* parameter in an application definition. The ZIP runtime works with either BATCH or FORK job types. For the ZIP runtime, *containerImage* must be an absolute path or a URL in a format supported by the Files service, such as URLs using the http, https or tapis protocols. The Applications service validates the *containerImage* attribute when an application is created or updated.

An application's archive file can contain scripts, binary executables, libraries and any other data the application developer needs---Tapis does not restrict content. The archive must be in zip format or any format readable by tar. In order to run correctly, the application must:

1. Have a designated executable program for Tapis to launch,
2. guarantee that the executable code is compatible with the target system's runtime,

3. guarantee that for FORK jobs the launched program continues to run until the application completes,
4. include in the archive all application dependencies not present on the target system.

There are also a number of conventions that ZIP applications should observe:

1. FORK jobs should write their exit code to *`\${execSystemOutputDir}/tapisjob.exitcode`*.
2. Except during testing, allow Tapis to remove downloaded archive files from the execution system after job completion.

The following sections describe the ZIP runtime processing during each phase of a job's lifecycle.

Staging Inputs

Inputs are staged in exactly the same way as in other runtime environments.

Staging Application Assets

ZIP jobs package executable code and data in an archive file that either already exists on the execution system or gets downloaded as part of job execution. The archive file is unpacked into the *execSystemExecDir*.

The archive file's location is specified in the *containerImage* attribute of the application. The location may either be (1) an absolute path on the execution system or (2) a URL. When an absolute path is used, the application archive already resides on the execution system. When a URLs is used, the archive is downloaded using one of following protocols: *http://*, *https://* and *tapis://*. The destination is always *execSystemExecDir* when URLs are used.

Application archives are always unpacked into *execSystemExecDir* whether the archive already existed on the system or was downloaded to the system. Jobs uses either *unzip* or *tar* as shown below. :

```
unzip <pathToArchiveFile>
tar -xf <pathToArchiveFile>
```

The *<pathToArchiveFile>* is the absolute path to the archive file, such as */path/to/archive/app.zip* or */path/to/archive/app.tgz*. The *<pathToArchiveFile>* must be present and accessible on the execution system. When an application archive is already present on the system (the non-URL case), the archive is not be copied, but its contents are extracted directly into the *execSystemExecDir*.

Note that the version of tar distributed with typical Linux distributions can unpack a number of compression formats, including *gzip*, *bzip2* and *xz*, but **not** *zip*. When an archive file name uses the *.zip* suffix, Tapis assumes *zip* formatting is being used. Tapis checks that the *unzip* command is available on the execution system and, if not, the job aborts.

Launching the Application

Once an application archive is unpacked, Jobs creates its own launch script, *tapisjob.sh*, and places it in the *execSystemExecDir*. From that directory, *tapisjob.sh* calls the user-provided executable. This executable is either *./tapisjob_app.sh*, if it exists, or an executable named in the *./tapisjob.manifest* file. The manifest file has these characteristics:

- The manifest file is optional.
- If *tapisjob_app.sh* does not exist then *tapisjob.manifest* must exist at the top-level of the archive and must specify the key-value pair:
tapisjob_executable=<relative path to executable file>.
 - The *tapisjob_executable* value is a path relative to *execSystemExecDir*.
 - The *tapisjob_executable* value cannot contain a double dot *".."* or semicolon *","*.
- If both *tapisjob_app.sh* and *tapisjob.manifest* exist, then *tapisjob_app.sh* will be used.
- If Jobs is not able to determine an executable to run then the job will fail.
- The application can put other information in *tapisjob.manifest*, such as build number, application version, etc.
 - The format of all entries in the manifest is: *<key>=<value>*, each on its own line.
 - Tapis ignores additional entries in the manifest.

Other than *tapisjob.manifest* and *tapisjob_app.sh*, no user-defined files in the top level archive directory should have a name that starts with "tapisjob". All such file names are reserved by Tapis and may be overwritten by the Jobs service. Specifically, the Jobs service creates files named *tapisjob.sh*, *tapisjob.env* and several temporary files that start with "tapisjob".

Environment and Executable Output

ZIP applications can expect to run in an environment that closely matches other Tapis runtimes. In particular, Jobs exports all user-specified and Tapis-generated environment variables in the SSH terminal from which it launches the application's executable. These are the same variables available to applications in other runtime environments. Similarly, *appArgs*, *containerArgs* and *schedulerOptions* semantics are unchanged.

Jobs also sets up the redirection of stdout and stderr using the *LogConfig* parameter as it does in other environments. Both streams are directed to *execSystemOutputDir/tapisjob.out* by default.

Monitoring the Application

For ZIP BATCH jobs, Tapis monitors the slurm job just as it does for other runtime types.

For ZIP FORK jobs, Tapis monitors the PID of the process launched by *tapisjob.sh*. When that PID goes away, Jobs assumes application processing has completed.

Note

It is the application developer's responsibility to terminate the executable launched by Tapis ONLY AFTER all other processes spawned by the application have terminated.

After a ZIP FORK job completes, Jobs looks for a file named **tapisjob.exitcode** in the *execSystemOutputDir*. If present, it is assumed to contain a single line with the application's exit code. A non-zero code is interpreted as a failure. If the file is not found, Jobs assumes success and reports an exit code of zero.

As with other runtimes, files created by Tapis during job execution are archived if *includeLaunchFiles* is true (the default). These files include *tapisjob.sh* and *tapisjob.env*.

Archiving Application Output

ZIP job outputs are archived using the same source and target specifications as other runtime types.

For ZIP jobs that specify the *containerImage* with an absolute path, the application archive file is never removed.

For ZIP jobs that specify the *containerImage* with a URL, Jobs removes the downloaded archive file by default in order to conserve storage. To prevent this automatic clean up, a new *containerArgs* flag, **--tapis-zip-save**, is defined. The flag takes no value and can be specified in the *containerArgs* list in the application definition or in the job submission request. If this argument is present, then the application's archive file will be left in the *execSystemExecDir*.

Usage Notes

To minimize ZIP archive size, jobs that invoke singularity containers may want to pre-position any large SIF files in shared directories on execution systems.

If *isMPI=true*, then Jobs will insert the MPI run command on the command line as usual. This is true for both BATCH and FORK job types. If the user-designated executable program launches MPI jobs itself, setting *isMPI=false* (the default) prevents Jobs from making conflicting MPI calls.

Job Monitoring

The Jobs service monitors a job during the two states where a job typically spends most of its time: QUEUED and RUNNING.

Note that if the monitoring process encounters errors for more than one hour then the job will transition to the FAILED state and the job completion condition code will be set to `JOB_EXECUTION_MONITORING_ERROR_TIMEOUT`.

Monitoring QUEUED State

Once a job enters the QUEUED state Tapis begins monitoring to detect when it becomes active and should be transitioned to the RUNNING state.

If the *jobType* is FORK then it will immediately transition to the RUNNING state because the remote application process will have been started during the SUBMITTING_JOB phase.

For BATCH jobs Tapis will monitor the remote status of the job at regular intervals to determine if it should be transitioned to the RUNNING state. The intervals for monitoring are based on a stepwise backoff policy as described below. The monitoring will be done indefinitely, allowing the job to remain in the QUEUED state as long as necessary.

Monitoring RUNNING State

Once a job enters the RUNNING state Tapis begins monitoring it to detect when it has finished and should be transitioned to the ARCHIVING state. The intervals for monitoring are based on a stepwise backoff policy as described below.

Monitoring will not continue indefinitely, however. If the remote batch scheduler times out the job or Tapis detects that the job runtime exceeds the *maxMinutes* setting, then Tapis will transition the job to the FAILED state and the job completion condition code will be set to either `SCHEDULER_TIMEOUT` for the former case or `JOB_EXECUTION_MONITORING_TIMEOUT` for the latter case.

Stepwise Backoff Monitoring Policy

The polling intervals for job monitoring are based on a stepwise backoff policy. The table below shows the number of iterations and the polling interval (in seconds) for each step.

Number of Iterations	Polling Interval (seconds)
1	1
5	10
10	60

Number of Iterations	Polling Interval (seconds)
100	180
100	300
Unlimited	600

Querying Jobs

Get Jobs list

With PySDK:

```
$ t.jobs.getJobList(limit=2,
orderBy='lastUpdated(desc),name(asc)', computeTotal=True)
```

With CURL:

```
$ curl -H "X-Tapis-Token:$jwt" $BASE_URL/v3/jobs/list?
limit=2&orderBy=lastUpdated(desc),name(asc)&computeTotal=true
```

The response will look something like the following:

```
{
"result": [
  {
    "uuid": "731b65f4-43e9-4a7a-b3a0-68644b53c1cb-007",
    "name": "SyRunSleepSecondsNoIPFiles-2",
    "owner": "testuser2",
    "appId": "SyRunSleepSecondsNoIPFiles-2",
    "created": "2021-07-21T19:56:02.163984Z",
    "status": "FINISHED",
    "remoteStarted": "2021-07-21T19:56:18.628448Z",
    "ended": "2021-07-21T19:56:52.637554Z",
    "tenant": "dev",
    "execSystemId": "tapisv3-exec2",
    "archiveSystemId": "tapisv3-exec2",
    "appVersion": "0.0.1",
    "lastUpdated": "2021-07-21T19:56:52.637554Z"
  },
  {
    "uuid": "79dfaba5-bfb4-4c6d-a198-643bda211dbf-007",
    "name": "SlurmSleepSeconds",
    "owner": "testuser2",
    "appId": "SlurmSleepSecondsVM",
```

```

        "created": "2021-07-21T19:16:02.019916Z",
        "status": "FINISHED",
        "remoteStarted": "2021-07-21T19:16:35.102868Z",
        "ended": "2021-07-21T19:16:57.909940Z",
        "tenant": "dev",
        "execSystemId": "tapisv3-exec2-slurm",
        "archiveSystemId": "tapisv3-exec2-slurm",
        "appVersion": "0.0.1",
        "lastUpdated": "2021-07-21T19:16:57.909940Z"
    }
],
"status": "success",
"message": "JOBS_LIST_RETRIVED Jobs list for the user testuser2 in
the tenant dev retrived.",
"version": "1.0.0-rc1",
"metadata": {
    "recordCount": 2,
    "recordLimit": 2,
    "recordsSkipped": 0,
    "orderBy": "lastUpdated(desc),name(asc)",
    "startAfter": null,
    "totalCount": 1799
}
}

```

Get Job Details

With PySDK:

```

$
t.jobs.getJob(jobUuid='ba34f946-8a18-44c4-9b25-19e21dfadf69-007')

```

With CURL:

```

$ curl -H "X-Tapis-Token:$jwt" $BASE_URL/v3/jobs/
ba34f946-8a18-44c4-9b25-19e21dfadf69-007

```

The response will look something like the following:

```

{
  "result": {
    id: 9594
    name: SleepSeconds
    owner: testuser2
    tenant: dev
    description: Transfer files and sleep for a specified amount of
time
    status: FINISHED
    condition: NORMAL_COMPLETION
    lastMessage: Setting job status to FINISHED.
    created: 2024-02-21T17:18:01.774368Z
    ended: 2024-02-21T17:18:49.182976Z
    lastUpdated: 2024-02-21T17:18:49.182976Z
  }
}

```

```
uuid: aa83fce9-074c-47fb-a8a0-a9ad216093bb-007
appId: SleepSeconds
appVersion: 0.0.1
archiveOnAppError: true
archiveMode: ALWAYS
dynamicExecSystem: false
execSystemId: tapisv3-exec2
execSystemExecDir: /workdir/jobs/aa83fce9-074c-47fb-a8a0-a9ad216093bb-007
execSystemInputDir: /workdir/jobs/aa83fce9-074c-47fb-a8a0-a9ad216093bb-007
execSystemOutputDir: /workdir/jobs/aa83fce9-074c-47fb-a8a0-a9ad216093bb-007/output
execSystemLogicalQueue: null
archiveSystemId: tapisv3-exec
archiveSystemDir: /jobs/aa83fce9-074c-47fb-a8a0-a9ad216093bb-007/archive
dtnSystemId: null
dtnSystemInputDir: null
dtnSystemOutputDir: null
nodeCount: 1
coresPerNode: 1
memoryMB: 100
maxMinutes: 240
fileInputs: [{"name": "empty", "notes": "{}", "envKey": null, "optional": true, "sourceUrl": "tapis://tapisv3-exec/jobs/input/empty.txt", "targetPath": "empty.txt", "description": "An empty file", "autoMountLocal": true, "srcSharedAppCtx": "", "destSharedAppCtx": ""}, {"name": "file1", "notes": "{}", "envKey": null, "optional": true, "sourceUrl": "tapis://tapisv3-exec/jobs/input/file1.txt", "targetPath": "file1.txt", "description": "A random text file", "autoMountLocal": true, "srcSharedAppCtx": "", "destSharedAppCtx": ""}, {"name": "file2", "notes": "{}", "envKey": null, "optional": true, "sourceUrl": "tapis://tapisv3-exec/jobs/input/file2.txt", "targetPath": "file2.txt", "description": "Another random text file", "autoMountLocal": true, "srcSharedAppCtx": "", "destSharedAppCtx": ""}]
parameterSet: {"appArgs": [], "logConfig": {"stderrFilename": "tapisjob.out", "stdoutFilename": "tapisjob.out"}, "envVariables": [{"key": "_tapisAppId", "notes": null, "value": "SleepSeconds", "include": null, "description": null}, {"key": "_tapisAppVersion", "notes": null, "value": "0.0.1", "include": null, "description": null}, {"key": "_tapisArchiveOnAppError", "notes": null, "value": "true", "include": null, "description": null}, {"key": "_tapisArchiveSystemDir", "notes": null, "value": "/jobs/aa83fce9-074c-47fb-a8a0-a9ad216093bb-007/archive", "include": null, "description": null}, {"key": "_tapisArchiveSystemId", "notes": null, "value": "tapisv3-exec", "include": null, "description": null}, {"key": "_tapisCoresPerNode", "notes": null, "value": "1", "include": null, "description": null}, {"key": "_tapisDynamicExecSystem", "notes": null, "value": "false", "include": null, "description": null}, {"key": "_tapisEffectiveUserId", "notes": null, "value": "testuser2",
```

```
"include": null, "description": null}, {"key":
"_tapisExecSystemExecDir", "notes": null, "value": "/workdir/jobs/
aa83fce9-074c-47fb-a8a0-a9ad216093bb-007", "include": null,
"description": null}, {"key": "_tapisExecSystemId", "notes": null,
"value": "tapisv3-exec2", "include": null, "description": null},
{"key": "_tapisExecSystemInputDir", "notes": null, "value": "/"
workdir/jobs/aa83fce9-074c-47fb-a8a0-a9ad216093bb-007", "include":
null, "description": null}, {"key": "_tapisExecSystemOutputDir",
"notes": null, "value": "/workdir/jobs/aa83fce9-074c-47fb-a8a0-
a9ad216093bb-007/output", "include": null, "description": null},
{"key": "_tapisJobCreateDate", "notes": null, "value":
"2024-02-21Z", "include": null, "description": null}, {"key":
"_tapisJobCreateTime", "notes": null, "value":
"17:18:01.774367559Z", "include": null, "description": null},
{"key": "_tapisJobCreateTimestamp", "notes": null, "value":
"2024-02-21T17:18:01.774367559Z", "include": null, "description":
null}, {"key": "_tapisJobName", "notes": null, "value":
"SleepSeconds", "include": null, "description": null}, {"key":
"_tapisJobOwner", "notes": null, "value": "testuser2", "include":
null, "description": null}, {"key": "_tapisJobUUID", "notes":
null, "value": "aa83fce9-074c-47fb-a8a0-a9ad216093bb-007",
"include": null, "description": null}, {"key":
"_tapisJobWorkingDir", "notes": null, "value": "/workdir",
"include": null, "description": null}, {"key": "_tapisMaxMinutes",
"notes": null, "value": "240", "include": null, "description":
null}, {"key": "_tapisMemoryMB", "notes": null, "value": "100",
"include": null, "description": null}, {"key": "_tapisNodes",
"notes": null, "value": "1", "include": null, "description":
null}, {"key": "_tapisStderrFilename", "notes": null, "value":
"tapisjob.out", "include": null, "description": null}, {"key":
"_tapisStdoutFilename", "notes": null, "value": "tapisjob.out",
"include": null, "description": null}, {"key": "_tapisSysHost",
"notes": null, "value": "129.114.35.53", "include": null,
"description": null}, {"key": "_tapisSysRootDir", "notes": null,
"value": "/home/testuser2", "include": null, "description": null},
{"key": "_tapisTenant", "notes": null, "value": "dev", "include":
null, "description": null}, {"key": "JOBS_PARMS", "notes": "{}",
"value": "15", "include": true, "description": "nothing
important"}, {"key": "MAIN_CLASS", "notes": "{}", "value":
"edu.utexas.tacc.testapps.tapis.SleepSeconds", "include": null,
"description": ""}], "archiveFilter": {"excludes": [], "includes":
["Sleep*"], "includeLaunchFiles": true}, "containerArgs": [],
"schedulerOptions": []
  subscriptions: []
  blockedCount: 0
  remoteJobId:
1e35edb11ee05bacf3da7cf0fedc55cfb6e616e15982df5b6d5ba69a44029351
  remoteJobId2: null
  remoteOutcome: FINISHED
  remoteResultInfo: 0
  remoteQueue: null
  remoteSubmitted: null
  remoteStarted: 2024-02-21T17:18:21.478768Z
  remoteEnded: 2024-02-21T17:18:43.465360Z
```

```

remoteSubmitRetries: 0
remoteChecksSuccess: 3
remoteChecksFailed: 0
remoteLastStatusCheck: 2024-02-21T17:18:43.463098Z
inputTransactionId: 4e47d395-50c5-4715-808e-3317f555e85f
inputCorrelationId: c62e5e7f-1142-47d6-bd85-cabfb9c8dc6b
archiveTransactionId: d20479da-50da-4ebd-be28-2680722a87d1
archiveCorrelationId: 2a70d8b9-fa5a-4cfd-a425-d095f69d8dfd
stageAppTransactionId: null
stageAppCorrelationId: null
dtnInputTransactionId: null
dtnInputCorrelationId: null
dtnOutputTransactionId: null
dtnOutputCorrelationId: null
tapisQueue: tapis.jobq.submit.DefaultQueue
visible: true
createdby: testuser2
createdbyTenant: dev
tags: [sleep, test]
jobType: FORK
mpiCmd: null
cmdPrefix: null
sharedAppCtx:
sharedAppCtxAttribs: []
notes: {"obj": {"f1": "v1", "f2": 44.0}, "array": ["x", "y",
"z"], "count": 27.0, "fname": "bud", "happy": true, "lname":
"jones"}
mpi: null
"status": "success",
"message": "JOBS_RETRIEVED Job
ba34f946-8a18-44c4-9b25-19e21dfadf69-007 retrieved.",
"version": "1.0.0-rc1",
"metadata": null
}

```

Understanding Job Timestamps

Timestamps are assigned as a job progresses, so they won't always have a value. All job timestamps are in Coordinated Universal Time (UTC) and have the following meanings:

1. *created*: When the job request has been accepted by the Jobs front-end.
2. *ended*: When the job reached a terminal state (FINISHED, FAILED or CANCELLED).
3. *lastUpdated*: The last time any part of the job record was updated.
4. *remoteSubmitted*: When the job was submitted to a scheduler on the execution system.
5. *remoteStarted*: When Jobs detected that the job began running on the execution system.
6. *remoteEnded*: When Job detected that the job stopped running on the executions system.

The *remoteSubmitted* timestamp indicates when batch jobs are submitted to the batch scheduler. Forked jobs, however, are not submitted to a scheduler, so in these cases *remoteSubmitted* indicates when application invocation began. Job monitoring typically involves polling an execution system to determine a job's state. The precision of the *remoteStarted* and *remoteEnded* timestamps is a function of the polling frequency, which changes based on Job service configuration and policies.

Get Job Status

With PySDK:

```
$  
t.jobs.getJobStatus(jobUuid='ba34f946-8a18-44c4-9b25-19e21dfadf69-007')
```

With CURL:

```
$ curl -H "X-Tapis-Token:$jwt" $BASE_URL/v3/jobs/  
ba34f946-8a18-44c4-9b25-19e21dfadf69-007/status
```

The response will look something like the following:

```
{  
  "result": {  
    "status": "FINISHED",  
    "condition": "NORMAL_COMPLETION"  
  },  
  "status": "success",  
  "message": "JOBS_STATUS_RETRIEVED Status of the Job  
ba34f946-8a18-44c4-9b25-19e21dfadf69-007 retrieved.",  
  "version": "1.0.0-rc1",  
  "metadata": null  
}
```

Get Job History

With PySDK:

```
$  
t.jobs.getJobHistory(jobUuid='ba34f946-8a18-44c4-9b25-19e21dfadf69-007')
```

With CURL:

```
$ curl -H "X-Tapis-Token:$jwt" $BASE_URL/v3/jobs/  
ba34f946-8a18-44c4-9b25-19e21dfadf69-007/history
```

The response will look something like the following:

```
{  
  "result": [  
    {  
      "event": "JOB_NEW_STATUS",  
      "created": "2021-07-12T23:56:02.365996Z",  
    }  
  ]  
}
```

```

        "jobStatus": "PENDING",
        "description": "The job has transitioned to a new status:
PENDING.",
        "transferTaskUuid": null,
        "transferSummary": {}
    },
    {
        "event": "JOB_NEW_STATUS",
        "created": "2021-07-12T23:56:02.799166Z",
        "jobStatus": "PROCESSING_INPUTS",
        "description": "The job has transitioned to a new status:
PROCESSING_INPUTS. The previous job status was PENDING.",
        "transferTaskUuid": null,
        "transferSummary": {}
    },
    {
        "event": "JOB_NEW_STATUS",
        "created": "2021-07-12T23:56:10.203007Z",
        "jobStatus": "STAGING_INPUTS",
        "description": "The job has transitioned to a new status:
STAGING_INPUTS. The previous job status was PROCESSING_INPUTS.",
        "transferTaskUuid": null,
        "transferSummary": {}
    },
    {
        "event": "JOB_NEW_STATUS",
        "created": "2021-07-12T23:56:10.226013Z",
        "jobStatus": "STAGING_JOB",
        "description": "The job has transitioned to a new status:
STAGING_JOB. The previous job status was STAGING_INPUTS.",
        "transferTaskUuid": null,
        "transferSummary": {}
    },
    {
        "event": "JOB_NEW_STATUS",
        "created": "2021-07-12T23:56:20.720637Z",
        "jobStatus": "SUBMITTING_JOB",
        "description": "The job has transitioned to a new status:
SUBMITTING_JOB. The previous job status was STAGING_JOB.",
        "transferTaskUuid": null,
        "transferSummary": {}
    },
    {
        "event": "JOB_NEW_STATUS",
        "created": "2021-07-12T23:56:20.888569Z",
        "jobStatus": "QUEUED",
        "description": "The job has transitioned to a new status:
QUEUED. The previous job status was SUBMITTING_JOB.",
        "transferTaskUuid": null,
        "transferSummary": {}
    },
    {
        "event": "JOB_NEW_STATUS",
        "created": "2021-07-12T23:56:20.902511Z",

```

```

        "jobStatus": "RUNNING",
        "description": "The job has transitioned to a new status:
RUNNING. The previous job status was QUEUED.",
        "transferTaskUuid": null,
        "transferSummary": {}
    },
    {
        "event": "JOB_NEW_STATUS",
        "created": "2021-07-12T23:56:42.427492Z",
        "jobStatus": "ARCHIVING",
        "description": "The job has transitioned to a new status:
ARCHIVING. The previous job status was RUNNING.",
        "transferTaskUuid": null,
        "transferSummary": {}
    },
    {
        "event": "JOB_NEW_STATUS",
        "created": "2021-07-12T23:56:55.966883Z",
        "jobStatus": "FINISHED",
        "description": "The job has transitioned to a new status:
FINISHED. The previous job status was ARCHIVING.",
        "transferTaskUuid": null,
        "transferSummary": {}
    }
],
"status": "success",
"message": "JOBS_HISTORY_RETRIEVED Job
ba34f946-8a18-44c4-9b25-19e21dfadf69-007 history retrieved for
user testuser2 tenant dev",
"version": "1.0.0-rc1",
"metadata": {
    "recordCount": 9,
    "recordLimit": 100,
    "recordsSkipped": 0,
    "orderBy": null,
    "startAfter": null,
    "totalCount": -1
}
}

```

Get Job Output Listing

With PySDK:

```

$
t.jobs.getJobOutputList(jobUuid='ba34f946-8a18-44c4-9b25-19e21dfadf69-007',
outputPath='/')

```

With CURL:

```

$ curl -H "X-Tapis-Token:$jwt" $BASE_URL/v3/jobs/
ba34f946-8a18-44c4-9b25-19e21dfadf69-007/output/list/

```


The response will look something like the following:

```
{
  "result": [
    {
      "mimeType": null,
      "type": "file",
      "owner": "1003",
      "group": "1003",
      "nativePermissions": "rw-rw-r--",
      "uri": "tapis://dev/tapisv3-exec/jobs/
ba34f946-8a18-44c4-9b25-19e21dfadf69-007/archive/
SleepSeconds.out",
      "lastModified": "2021-07-12T23:56:54Z",
      "name": "SleepSeconds.out",
      "path": "/jobs/ba34f946-8a18-44c4-9b25-19e21dfadf69-007/
archive/SleepSeconds.out",
      "size": 3538
    },
    {
      "mimeType": null,
      "type": "file",
      "owner": "1003",
      "group": "1003",
      "nativePermissions": "rw-rw-r--",
      "uri": "tapis://dev/tapisv3-exec/jobs/
ba34f946-8a18-44c4-9b25-19e21dfadf69-007/archive/tapisjob.env",
      "lastModified": "2021-07-12T23:56:53Z",
      "name": "tapisjob.env",
      "path": "/jobs/ba34f946-8a18-44c4-9b25-19e21dfadf69-007/
archive/tapisjob.env",
      "size": 1051
    },
    {
      "mimeType": null,
      "type": "file",
      "owner": "1003",
      "group": "1003",
      "nativePermissions": "rw-rw-r--",
      "uri": "tapis://dev/tapisv3-exec/jobs/
ba34f946-8a18-44c4-9b25-19e21dfadf69-007/archive/
tapisjob.exitcode",
      "lastModified": "2021-07-12T23:56:54Z",
      "name": "tapisjob.exitcode",
      "path": "/jobs/ba34f946-8a18-44c4-9b25-19e21dfadf69-007/
archive/tapisjob.exitcode",
      "size": 1
    },
    {
      "mimeType": null,
      "type": "file",
      "owner": "1003",
      "group": "1003",
      "nativePermissions": "rw-rw-r--",
```

```

        "uri": "tapis://dev/tapisv3-exec/jobs/
ba34f946-8a18-44c4-9b25-19e21dfadf69-007/archive/tapisjob.out",
        "lastModified": "2021-07-12T23:56:54Z",
        "name": "tapisjob.out",
        "path": "/jobs/ba34f946-8a18-44c4-9b25-19e21dfadf69-007/
archive/tapisjob.out",
        "size": 3566
    },
    {
        "mimeType": "application/x-shar",
        "type": "file",
        "owner": "1003",
        "group": "1003",
        "nativePermissions": "rw-rw-r--",
        "uri": "tapis://dev/tapisv3-exec/jobs/
ba34f946-8a18-44c4-9b25-19e21dfadf69-007/archive/tapisjob.sh",
        "lastModified": "2021-07-12T23:56:54Z",
        "name": "tapisjob.sh",
        "path": "/jobs/ba34f946-8a18-44c4-9b25-19e21dfadf69-007/
archive/tapisjob.sh",
        "size": 979
    }
],
"status": "success",
"message": "JOBS_OUTPUT_FILES_LIST_RETRIEVED Job
ba34f946-8a18-44c4-9b25-19e21dfadf69-007 output files list
retrieved for the user testuser2 in the tenant dev.",
"version": "1.0.0-rc1",
"metadata": {
    "recordCount": 5,
    "recordLimit": 100,
    "recordsSkipped": 0,
    "orderBy": null,
    "startAfter": null,
    "totalCount": 0
}
}

```

The Job output list API retrieves job's output files list for a previously submitted job by its UUID. By default, the job must be in a terminal state (FINISHED or FAILED or CANCELLED) for the API to list the job's output files . There is a query parameter `allowIfRunning` set to false by default. If `allowIfRunning=true`, the API returns the job output files list even if the job is not in the terminal state. Note that if a file is being written, still the file is listed.

Get Job Output Download

With PySDK:

```

$
t.jobs.getJobOutputDownload(jobUuid='ba34f946-8a18-44c4-9b25-19e21c
outputPath='/')

```

With CURL:

```
$ curl -H "X-Tapis-Token:$jwt" $BASE_URL/v3/jobs/
ba34f946-8a18-44c4-9b25-19e21dfadf69-007/output/download/ --output
joboutput.zip
```

All the files in the the requested outputPath get downloaded in a zip file.

The Jobs output download API retrieves the job's output files for a previously submitted job by its UUID. By default, the job must be in a terminal state (FINISHED or FAILED or CANCELLED) for the API to download the job's output files. There is a query parameter allowIfRunning set to false by default. If allowIfRunning=true, the API allows downloading the job output files even if the job is not in the terminal state. Note that if a file is being written at the time of the request, the file is still downloaded with the current content.

Dedicated Search Endpoint

The jobs service provides dedicated search end-points to query jobs based on different conditions. The GET end-point allows to specify the query in the query parameters while the POST end-point allows complex queries in the request body using SQL-like syntax.

Search using GET on Dedicated Endpoint

With CURL:

```
$ curl -H "X-Tapis-Token:$jwt" $BASE_URL/v3/jobs/search?
limit=2&status.eq=FINISHED&created.between=2021-07-01,2021-07-21&oi
```

The response will look something like the following:

```
{
  "result": [
    {
      "uuid": "79234b2a-0995-4632-956e-b940d10607ba-007",
      "name": "SyRunSleepSecondsNoIPFiles-2",
      "owner": "testuser2",
      "appId": "SyRunSleepSecondsNoIPFiles-2",
      "created": "2021-07-20T23:56:02.616Z",
      "status": "FINISHED",
      "remoteStarted": "2021-07-20T23:56:20.368Z",
      "ended": "2021-07-20T23:56:54.409Z",
      "tenant": "dev",
      "execSystemId": "tapisv3-exec2",
      "archiveSystemId": "tapisv3-exec",
      "appVersion": "0.0.1",
      "lastUpdated": "2021-07-20T23:56:54.409Z"
    },
    {
```

```

        "uuid": "432f7018-070d-41c3-ba0e-a685f7f11e5c-007",
        "name": "SlurmSleepSeconds",
        "owner": "testuser2",
        "appId": "SlurmSleepSecondsVM",
        "created": "2021-07-20T23:16:01.629Z",
        "status": "FINISHED",
        "remoteStarted": "2021-07-20T23:16:24.781Z",
        "ended": "2021-07-20T23:16:58.745Z",
        "tenant": "dev",
        "execSystemId": "tapisv3-exec2-slurm",
        "archiveSystemId": "tapisv3-exec",
        "appVersion": "0.0.1",
        "lastUpdated": "2021-07-20T23:16:58.745Z"
    },
    ],
    "status": "success",
    "message": "JOBS_SEARCH_RESULT_LIST_RETRIEVED Jobs search list
for the user testuser2 in the tenant dev retrieved.",
    "version": "1.0.0-rc1",
    "metadata": {
        "recordCount": 2,
        "recordLimit": 2,
        "recordsSkipped": 0,
        "orderBy": "lastUpdated(desc),name(asc)",
        "startAfter": null,
        "totalCount": 246
    }
}

```

Search using POST on Dedicated Endpoint

A user can make complex queries to Jobs service by specifying SQL-like syntax in the request body to the end-point /v3/jobs/search. An example request body in json format is shown below:

```

{
  "search":
  [
    "(status = 'FINISHED' AND name = 'SleepSeconds') ",
    " OR (tags IN ('test'))"
  ]
}

```

With cURL:

```

curl --location '$BASE_URL/v3/jobs/search?
listType=ALL_JOBS&limit=2&computeTotal=True&select=name%2Ctags%2Cst
\
--header 'X-Tapis-Token: $jwt' \
--header 'Content-Type: application/json' \
--data '{
  "search":

```

```
[
  "(status = '\\'FINISHED\\' AND name = '\\'SleepSeconds\\')
",
  " OR (tags IN (\\'test\\'))"
]
}'
```

The response looks like this:

```
{
  "result": [
    {
      "name": "SleepSecondsLoadTest",
      "status": "FINISHED",
      "appId": "SleepSeconds-Load",
      "tags": [
        "sleep",
        "test"
      ],
      "uuid": "e17edea6-33f8-441c-867c-d9d23509dd55-007"
    },
    {
      "name": "SleepSecondsLoadTest",
      "status": "FINISHED",
      "appId": "SleepSeconds-Load",
      "tags": [
        "sleep",
        "test"
      ],
      "uuid": "a3a539a9-c0e5-4a02-82c3-0dfbcada47f9-007"
    }
  ],
  "status": "success",
  "message": "JOBS_SEARCH_RESULT_LIST_RETRIEVED Jobs search list for the user testuser2 in the tenant dev retrieved. ",
  "version": "1.3.0",
  "commit": "ee1b3342",
  "build": "2023-03-01T15:42:55Z",
  "metadata": {
    "recordCount": 2,
    "recordLimit": 2,
    "recordsSkipped": 0,
    "orderBy": null,
    "startAfter": null,
    "totalCount": 27345
  }
}
```

Job Actions

Job Cancel

A previously submitted job not in terminal state can be cancelled by its UUID.

With PySDK:

```
$ t.jobs.cancelJob(jobUuid='19b06299-4e7c-4b27-ae77-2258e9dc4734-007')
```

With CURL:

```
$ curl -X POST -H "X-Tapis-Token:$jwt" $BASE_URL/v3/jobs/19b06299-4e7c-4b27-ae77-2258e9dc4734-007/cancel
```

The response will look something like the following:

```
{
  "result": {
    "message": "JOBS_JOB_CANCEL_ACCEPTED Request to cancel job 19b06299-4e7c-4b27-ae77-2258e9dc4734-007 has been accepted. ",
  },
  "status": "success",
  "message": "JOBS_JOB_CANCEL_ACCEPTED_DETAILS Request to cancel job 19b06299-4e7c-4b27-ae77-2258e9dc4734-007 has been accepted. If the job is in a terminal state, the request will have no effect. If the job is transitioning between active and blocked states, another cancel request may need to be sent.",
  "version": "1.2.1",
  "metadata": null
}
```

Hide Job

With PySDK:

```
$ t.jobs.hideJob(jobUuid='19b06299-4e7c-4b27-ae77-2258e9dc4734-007')
```

With CURL:

```
$ curl -X POST -H "X-Tapis-Token:$jwt" $BASE_URL/v3/jobs/19b06299-4e7c-4b27-ae77-2258e9dc4734-007/hide
```

The response will look something like the following:

```
{
  "result": {
    "message": "JOBS_JOB_CHANGED_VISIBILITY Job 19b06299-4e7c-4b27-ae77-2258e9dc4734-007 has been changed to
```

```
hidden."
},
"status": "success",
"message": "JOBS_JOB_CHANGED_VISIBILITY Job 19b06299-4e7c-4b27-ae77-2258e9dc4734-007 has been changed to hidden.",
"version": "1.2.1",
"metadata": null
}
```

Unhide Job

With PySDK:

```
$ t.jobs.unhideJob(jobUuid='19b06299-4e7c-4b27-ae77-2258e9dc4734-007')
```

With CURL:

```
$ curl -X POST -H "X-Tapis-Token:$jwt" $BASE_URL/v3/jobs/19b06299-4e7c-4b27-ae77-2258e9dc4734-007/unhide
```

The response will look something like the following:

```
{
  "result": {
    "message": "JOBS_JOB_CHANGED_VISIBILITY Job 19b06299-4e7c-4b27-ae77-2258e9dc4734-007 has been changed to unhidden."
  },
  "status": "success",
  "message": "JOBS_JOB_CHANGED_VISIBILITY Job 19b06299-4e7c-4b27-ae77-2258e9dc4734-007 has been changed to unhidden.",
  "version": "1.2.1",
  "metadata": null
}
```

Put Job Annotations

Replace the tags and notes of a specified job with the provided tags and notes. If no tags or notes are not provided at all, the current tags or notes are not changed. If empty tags ([]) or notes ({}) are provided, the current tags or notes are removed. The total size of notes and the total size of all tags of a job cannot exceed 128KB each.

With PySDK:

```
$ job_annotation=t.jobs.putJobAnnotations(jobUuid=resubmitted_job, tags=[
    "A"
],
notes= {
    "a": [
        "1"
    ]
})
```

```

        ],
        "b": {
            "c": "label1"
        },
        "c": 1
    }
)

```

With CURL:

```

$ curl -X PUT -H "X-Tapis-Token:$jwt" $BASE_URL/v3/jobs/6269c8f1-
a89b-40ae-86ff-6674125b563a-007/annotations -d '{
    "tags": [
        "A"
    ],
    "notes": {
        "a": [
            "1"
        ],
        "b": {
            "c": "label1"
        },
        "c": 1
    }
}'

```

The response will look something like the following:

```

{
    "result": {
        "id": 47099,
        "uuid": "6269c8f1-a89b-40ae-86ff-6674125b563a-007",
        "tags": [
            "A"
        ],
        "notes": {
            "a": [
                "1"
            ],
            "b": {
                "c": "label1"
            },
            "c": 1
        }
    },
    "status": "success",
    "message": "JOBS_JOB_ANNOTATION_UPDATED Job 6269c8f1-
a89b-40ae-86ff-6674125b563a-007 annotation updated for user
testuser7 in tenant dev.",
    "version": "1.9.1",
    "commit": "037fab4a",
    "build": "2025-10-09T18:13:04Z",
}

```



```

    "metadata": null
}

```

Patch Job Annotations

Patch the tags and notes of a specified job with the provided tags and notes. If no tags or notes are provided, the current tags and notes are not changed. If empty tags ([]) or notes ({}) are provided, the current tags or notes are not changed. Only the new tags provided are added to the current tags list of a job, in a case-sensitive manner. Only the top-level keys in notes provided are added or updated in the current notes of a job. The total size of notes and the total size of all tags of a job cannot exceed 128KB each.

With PySDK:

```

$ t.jobs.patchJobAnnotations(jobUuid=resubmitted_job, tags=[
    "A",
    "a",
    "B",
    "b",
    ""
], notes={
    "a": ["2", ""],
    "b": 2,
    "c": "c",
    "d": {}
})

```

With CURL:

```

$ curl -X PATCH -H "X-Tapis-Token:$jwt" $BASE_URL/v3/jobs/
6269c8f1-a89b-40ae-86ff-6674125b563a-007/annotation -d '{
    "tags": [
        "A",
        "a",
        "B",
        "b",
        ""
    ],
    "notes": {
        "a": ["2", ""],
        "b": 2,
        "c": "c",
        "d": {}
    }
}'

```

The response will look something like the following:

```

{
    "result": {
        "id": 47099,
        "uuid": "6269c8f1-a89b-40ae-86ff-6674125b563a-007",

```

```

        "tags": [
            "",
            "A",
            "B",
            "a",
            "b"
        ],
        "notes": {
            "a": [
                "2",
                ""
            ],
            "b": 2,
            "c": "c",
            "d": {}
        }
    },
    "status": "success",
    "message": "JOBS_JOB_ANNOTATION_PATCHED Job 6269c8f1-
a89b-40ae-86ff-6674125b563a-007 annotation updated for user
testuser7 in tenant dev.",
    "version": "1.9.1",
    "commit": "037fab4a",
    "build": "2025-10-09T18:13:04Z",
    "metadata": null
}

```

Job Sharing

Share a Job

A previously submitted job can be shared with a user in the same tenant. Job resources that can be shared are: JOB_HISTORY, JOB_RESUBMIT_REQUEST, JOB_OUTPUT. Currently only READ permission on the resources are allowed.

With PySDK:

```

$ t.jobs.shareJob(jobUuid='ccec730b-22ad-4088-a87e-
bb8cfb2ab2e6-007',

jobResource=["JOB_HISTORY","JOB_RESUBMIT_REQUEST","JOB_OUTPUT"],
jobPermission='READ',
grantee='testuser6')

```

With CURL:

```

$ curl -X POST -H "X-Tapis-Token:$jwt" $BASE_URL/v3/jobs/
ccec730b-22ad-4088-a87e-bb8cfb2ab2e6-007/share -d '{

```

```
"jobResource": ["JOB_HISTORY", "JOB_RESUBMIT_REQUEST",  
"JOB_OUTPUT"], "jobPermission": "READ", "grantee": "testuser6"}'
```

The response will look something like the following:

```
{  
  "result": {  
    "message": "JOBS_JOB_SHARED The job ccec730b-22ad-4088-a87e-  
bb8cfb2ab2e6-007 resource is shared by testuser2 to testuser6 in  
tenant dev"  
  },  
  "status": "success",  
  "message": "JOBS_JOB_SHARED The job ccec730b-22ad-4088-a87e-  
bb8cfb2ab2e6-007 resource is shared by testuser2 to testuser6 in  
tenant dev",  
  "version": "1.2.1",  
  "metadata": null  
}
```

Get Job Share Information

With PySDK:

```
$ t.jobs.getJobShare(jobUuid='ccec730b-22ad-4088-a87e-  
bb8cfb2ab2e6-007')
```

With CURL:

```
$ curl -H "X-Tapis-Token:$jwt" $BASE_URL/v3/jobs/  
ccec730b-22ad-4088-a87e-bb8cfb2ab2e6-007/share
```

The response will look something like the following:

```
{  
  "result": [  
    {  
      "tenant": "dev",  
      "createdby": "testuser2",  
      "jobUuid": "ccec730b-22ad-4088-a87e-bb8cfb2ab2e6-007",  
      "grantee": "testuser5",  
      "jobResource": "JOB_HISTORY",  
      "jobPermission": "READ",  
      "created": "2022-06-16T14:53:31.899199Z"  
    },  
    {  
      "tenant": "dev",  
      "createdby": "testuser2",  
      "jobUuid": "ccec730b-22ad-4088-a87e-bb8cfb2ab2e6-007",  
      "grantee": "testuser5",  
      "jobResource": "JOB_OUTPUT",  
      "jobPermission": "READ",  
      "created": "2022-06-16T14:53:32.004831Z"  
    }  
  ],  
}
```

```

{
  "tenant": "dev",
  "createdby": "testuser2",
  "jobUuid": "ccec730b-22ad-4088-a87e-bb8cfb2ab2e6-007",
  "grantee": "testuser6",
  "jobResource": "JOB_HISTORY",
  "jobPermission": "READ",
  "created": "2022-06-16T17:17:50.981844Z"
},
{
  "tenant": "dev",
  "createdby": "testuser2",
  "jobUuid": "ccec730b-22ad-4088-a87e-bb8cfb2ab2e6-007",
  "grantee": "testuser6",
  "jobResource": "JOB_RESUBMIT_REQUEST",
  "jobPermission": "READ",
  "created": "2022-06-16T17:17:51.059726Z"
},
{
  "tenant": "dev",
  "createdby": "testuser2",
  "jobUuid": "ccec730b-22ad-4088-a87e-bb8cfb2ab2e6-007",
  "grantee": "testuser6",
  "jobResource": "JOB_OUTPUT",
  "jobPermission": "READ",
  "created": "2022-07-14T19:57:15.838019Z"
}
],
"status": "success",
"message": "JOBS_JOB_SHARE_INFO_RETRIEVED Share information
retrieved for the job ccec730b-22ad-4088-a87e-bb8cfb2ab2e6-007 for
user testuser2 in tenant dev.",
"version": "1.2.1",
"metadata": {
  "recordCount": 5,
  "recordLimit": 100,
  "recordsSkipped": 0,
  "orderBy": null,
  "startAfter": null,
  "totalCount": 5
}
}

```

Unshare Job

With PySDK:

```
$ t.jobs.deleteJobShare(jobUuid='ccec730b-22ad-4088-a87e-
bb8cfb2ab2e6-007', user='testuser6')
```

With CURL:

```
$ curl -X DELETE -H "X-Tapis-Token:$jwt" $BASE_URL/v3/jobs/
ccec730b-22ad-4088-a87e-bb8cfb2ab2e6-007/share/testuser6
```

The response will look something like this:

```
{
  "result": {
    "message": "JOBS_JOB_UNSHARED The job ccec730b-22ad-4088-a87e-
bb8cfb2ab2e6-007 resource is unshared by testuser2 to testuser6 in
tenant dev"
  },
  "status": "success",
  "message": "JOBS_JOB_UNSHARED The job ccec730b-22ad-4088-a87e-
bb8cfb2ab2e6-007 resource is unshared by testuser2 to testuser6 in
tenant dev",
  "version": "1.2.1",
  "metadata": null
}
```

List Shared Jobs

The query parameter `listType=SHARED_JOBS` in the jobs list end-point allows to list all shared jobs for a user, say `testuser6`, using `testuser6`'s JWT. Note `testuser6` is not the owner of the jobs listed. Default value for `listType` is `MY_JOBS` and to list all jobs including both shared and `testuser6` owned job, `listType=ALL_JOBS`.

With PySDK:

```
$ t.jobs.getJobList(listType='SHARED_JOBS')
```

With CURL:

```
$ curl -H "X-Tapis-Token:$jwt" $BASE_URL/v3/jobs/list?
listType=SHARED_JOBS
```

The response will look something like this:

```
{
  "result": [
    {
      "uuid": "ccec730b-22ad-4088-a87e-bb8cfb2ab2e6-007",
      "name": "SleepSeconds",
      "owner": "testuser2",
      "appId": "SleepSeconds",
      "created": "2021-04-09T02:47:57.760Z",
      "status": "FINISHED",
      "remoteStarted": "2021-04-09T02:48:08.946Z",
      "ended": "2021-04-09T02:48:16.669Z",
      "tenant": "dev",
      "execSystemId": "tapisv3-exec2",
      "archiveSystemId": "tapisv3-exec",
      "appVersion": "0.0.1",
      "lastUpdated": "2021-04-09T02:48:16.669Z"
    }
  ]
}
```

```

    },
  ],
  "status": "success",
  "message": "JOBS_LIST_RETRIVED Jobs list for the user testuser6
in the tenant dev retrived.",
  "version": "1.2.1",
  "metadata": {
    "recordCount": 1,
    "recordLimit": 100,
    "recordsSkipped": 0,
    "orderBy": null,
    "startAfter": null,
    "totalCount": -1
  }
}

```

Job Search on Shared Job

Job search on a list of jobs can be performed using the query parameter listType=SHARED_JOBS in the search end-point.

In the following example, we use testuser5 JWT to do job search on list of jobs shared with testuser5.

With CURL:

```

$ curl -H "X-Tapis-Token:$jwt" '$BASE_URL/v3/jobs/search?
listType=SHARED_JOBS&name.eq=SleepSeconds&created.between=2022-07-01&

```

The response will look something like this:

```

{
  "result": [
    {
      "uuid": "3b9cb514-4962-44e8-a851-55e933e558c0-007",
      "name": "SleepSeconds",
      "owner": "testuser2",
      "appId": "SleepSeconds",
      "created": "2022-07-05T21:34:11.355Z",
      "status": "FINISHED",
      "remoteStarted": "2022-07-05T21:34:48.776Z",
      "ended": "2022-07-05T21:35:31.808Z",
      "tenant": "dev",
      "execSystemId": "tapisv3-exec2",
      "archiveSystemId": "tapisv3-exec",
      "appVersion": "0.0.1",
      "lastUpdated": "2022-07-05T21:35:31.808Z"
    },
    {
      "uuid": "eacde4c4-1d93-4393-b220-63aea509b32c-007",
      "name": "SleepSeconds",
      "owner": "testuser2",
      "appId": "SleepSeconds",

```

```

        "created": "2022-07-05T21:43:05.371Z",
        "status": "FINISHED",
        "remoteStarted": "2022-07-05T21:43:44.899Z",
        "ended": "2022-07-05T21:44:27.509Z",
        "tenant": "dev",
        "execSystemId": "tapisv3-exec2",
        "archiveSystemId": "tapisv3-exec",
        "appVersion": "0.0.1",
        "lastUpdated": "2022-07-05T21:44:27.509Z"
    },
    ],
    "status": "success",
    "message": "JOBS_SEARCH_RESULT_LIST_RETRIEVED Jobs search list for
the user testuser5 in the tenant dev retrieved.",
    "version": "1.2.1",
    "metadata": {
        "recordCount": 2,
        "recordLimit": 100,
        "recordsSkipped": 0,
        "orderBy": null,
        "startAfter": null,
        "totalCount": -1
    }
}

```

Share Job Output

As shown in the previous example of share a job, job output resource can be shared with a user, say testuser5, in the same tenant. This includes testuser5 can do job output listing and job output download with its own JWT even though its not the owner of the job.

Share Job History

As shown in the previous example of share a job, job history can be shared with a user, say testuser5, in the same tenant. This includes testuser5 can get job's history, status and job's detail information for jobs that are shared with it using its own JWT.

Files

The files service is the central point of interaction for all file operations in the Tapis ecosystem.

Overview

Through the Files service users can perform file listing, uploading, and various operations such as move, copy, mkdir and delete. The service also supports transferring files from one Tapis system to another.

Currently, the files service includes support for systems of type LINUX, S3, IRODS and GLOBUS.

Note that supported functionality varies by system type.

All file operations act upon Tapis *System* resources.¹ For more information on the Systems service please see [Systems](#)

Basic File Operations

Listing

Tapis supports listing files or objects on a Tapis system. The type for items listed will depend on system type. For example, for LINUX they will be posix files and for S3 they will be storage objects. See the next section below for additional considerations for S3 type systems. On S3 systems, for example, the recurse flag is ignored and all objects with keys matching the path as a prefix are always included.

For system types that support directory hierarchies the maximum recursion depth is 20.

To list the files in the effective *rootDir* directory of a Tapis system:

Using the official Tapis Python SDK:

```
t.files.listFiles(systemId="my-tapis-system", path="/")
```


Or using curl:

```
curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/files/ops/my-system/
```

And to list a sub-directory in the system, just add the path to the request:

Using CURL

```
curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/files/ops/aturing-storage/subDir1/subDir2/subDir3/
```

Query Parameters

limit

integer - Max number of results to return, default of 1000

offset

integer - Skip the first N listings

The JSON response of the API will look something like this:

```
{
  "status": "success",
  "message": "ok",
  "result": [
    {
      "mimeType": "text/plain",
      "type": "file",
      "owner": "1003",
      "group": "1003",
      "nativePermissions": "drwxrwxr-x",
      "url": "tapis://dev/aturing-storage/file1.txt",
      "lastModified": "2021-04-29T16:55:57Z",
      "name": "file1.txt",
      "path": "file1.txt",
      "size": 313
    },
    {
      "mimeType": "text/plain",
      "type": "file",
      "owner": "1003",
      "group": "1003",
      "nativePermissions": "-rw-rw-r--",
      "url": "tapis://dev/aturing-storage/file2.txt",
      "lastModified": "2020-12-17T22:46:29Z",
      "name": "file2.txt",
      "path": "file2.txt",
      "size": 21
    }
  ],
  "version": "1.1-84a31617",
  "metadata": {}
}
```

Listings and S3 Support

File listings on S3 type systems have some special considerations. Objects in an S3 bucket do not have a hierarchical structure. There are no directories. Everything is an object associated with a key.

One thing to note is that, as mentioned above, for S3 the recurse flag is ignored and all objects with keys matching the path as a prefix are always included.

Note that for S3 this means that when the path is an empty string all objects in the bucket with a prefix matching *rootDir* will be included. This is especially important to keep in mind when using the delete operation to remove objects matching a path.

The attribute *rootDir* is optional for S3 type systems. When defined it will be prepended to all paths and the resulting path will become the key.

Note

When *rootDir* is defined for an S3 system it typically should not begin with /. For S3 keys are typically created and manipulated using URLs and do not have a leading /.

Handling of symbolic links on Linux systems

If listing contains a symbolic link, it will show type of symbolic link:

```
{
  "group": "1002",
  "lastModified": "2023-05-09T19:53:53Z",
  "mimeType": null,
  "name": "x2",
  "nativePermissions": "rwxrwxrwx",
  "owner": "1002",
  "path": "x2",
  "size": 4,
  "type": "symbolic_link",
  "url": "tapis://mysystem/mySymLinkedFileOrDirectory"
},
```

If a listing for a path that is a symbolic link is requested, the symbolic link is followed, and the information is returned for path that the symbolic link points to. If the path doesn't exist, an error will be returned. If the link points to a file, the file's information will be returned (the type will be "file"). If the symbolic link points to a directory, the contents of that directory will be returned.

Move and Copy

To move or copy a file or directory using the files service, make a PUT request using the path to the current location of the file or folder.

For example, to copy a file located at /file1.txt to /subdir/file1.txt

```
curl -H "X-Tapis-Token: $JWT" -X PUT -d @body.json "https://tacc.tapis.io/v3/files/ops/aturing-storage/file1.txt"
```

with a JSON body of

```
{  
  "operation": "COPY",  
  "newPath": "/subdir/file1.txt"  
}
```

Handling of symbolic links on Linux systems

During a move or copy, if a symbolic link is encountered, it will be handled as shown in the tables below. The first and second columns indicate whether the link is the source or target and if it points to a file or directory.

Copy

Symbolic Link	Points To	Notes
		If the destination path is to a file (this means that the path ends in a component that does exist, and it's a file or a component that does not exist): • a new symbolic link is created that points to the same place as the source. • if the destination path includes directories that do not exist, they will be created.
source	file	If the destination path is to a directory (this means that the path ends in a component that does exist and it is a directory): • If the path given is to an existing directory, a new link with the same name will be created in that directory, and it will point to the same place as the source. Note that if the link is to a relative path, moving it could change where it actually points because the exact relative path will remain the same. If the destination path is to a file (this means that the path ends in a component that does exist, and it's a file or a component that does not exist):
source	directory	• a new symbolic link is created that points to the same place as the source. • if the destination path includes directories that do not exist, they will be created.

Symbolic Link	Points To	Notes
		If the destination path is to a directory (this means that the path ends in a component that does exist and it is a directory): • If the path given is to an existing directory, a new link with the same name will be created in that directory, and it will point to the same place as the source. Note that if the link is to a relative path, moving it could change where it actually points because the exact relative path will remain the same.
destination	file	The destination is replaced by the source. The source could be a file, directory, or link to a file or directory.
destination	directory	The new file, directory, or link is created inside of the existing directory.

Move

Symbolic Link	Points To	Notes
		If the destination path is to a file (this means that the path ends in a component that does exist, and it's a file or a component that does not exist): • the symbolic link is renamed. • if the destination path includes directories that do not exist, they will be created, and the new link will be placed there.
source	file	If the destination path is to a directory (this means that the path ends in a component that does exist and it is a directory): • If the path given is to an existing directory, the link is moved to that directory. Note that if the link is to a relative path, moving it could change where it actually points because the exact relative path will remain the same.
source	directory	If the destination path is to a file (this means that the path ends in a component that does exist, and it's a file or a component that does not exist): • the symbolic link is renamed.

Symbolic Link	Points To	Notes
		• if the destination path includes directories that do not exist, they will be created. If the destination path is to a directory (this means that the path ends in a component that does exist and it is a directory): • if the path given is to an existing directory, the source link will be moved into that directory, and it will point to the same place as the source. Note that if the link is to a relative path, moving it could change where it actually points because the exact relative path will remain the same.
destination	file	The source link is renamed, to the destination path. The destination is replaced. The source could be a file, directory, or link to a file or directory.
destination	directory	The file, directory, or link is moved inside of the existing directory.

Making directories

To create a directory on a Tapis system at the given path, issue a mkdir request. This is not supported for all system types. The mkdir operation is currently supported for LINUX, IRODS and GLOBUS type systems.

Using the Tapis Python SDK:

```
t.files.mkdir(systemId="my-system", path="/folderA/folderB/newDirectory")
```

Using CURL:

```
curl -H "X-Tapis-Token: $JWT" -H "Content-Type:application/json"
-X POST https://tacc.tapis.io/v3/files/ops/<systemId> -d
'{"path":"<directory_path">'
```

Uploading

To upload a file use a POST request. The file will be placed at the location specified in the {path} parameter in the request. Not all system types support this operation. For example, given the system my-system, to upload file someFile.txt to directory ~/folderA/folderB/folderC`:

Using the official Tapis Python SDK:

```
t.upload(source_file_path="experiment-results.hd5", system_id="my-system", dest_file_path="/folderA/folderB/folderC/someFile.txt")
```

```
curl -H "X-Tapis-Token: $JWT" -H "Content-Type:application/json" -X POST -F "file=@someFile.txt" https://tacc.tapis.io/v3/files/ops/my-system/folderA/folderB/folderC/someFile.txt
```

For some system types (such as LINUX) any folders that do not exist in the specified path will automatically be created.

Note that for an S3 system an object will be created with a key of *rootDir*/*{path}*.

Deleting

To delete a file or folder, issue a DELETE request for the path to be removed.

```
curl -H "X-Tapis-Token: $JWT" -X DELETE "https://tacc.tapis.io/v3/files/ops/aturing-storage/file1.txt"
```

The request above would delete *file1.txt*

For an S3 system, the path will represent either a single object or all objects in the bucket with a prefix matching the system *rootDir* if the path is the empty string.

Warning

For an S3 system if the path is the empty string, then all objects in the bucket with a key matching the prefix *rootDir* will be deleted. So if the *rootDir* is also the empty string, then all objects in the bucket will be removed.

Handling of symbolic links and special files on Linux Systems

If Tapis encounters a symbolic link during a delete operation, the link will be deleted. The file or directory that the link points to will be unaffected. If the delete encounters a special file (such as a device file or fifo, etc), it will not be deleted, and an error will be returned. If this is in the middle of a recursive delete operation, some files may have been already deleted.

Creating a directory

To create a directory, use POST and provide the path to the new directory in the request body. Not all system types support this operation.

```
$ curl -H "X-Tapis-Token: $JWT" -H "Content-Type:application/json" -X POST -d @body.json https://tacc.tapis.io/v3/files/ops/my-system
```

with a JSON body of

```
{  
  "path": "path/to/new/directory/"  
}
```

Getting Linux stat information

Get native stat information for a file or directory for a system of type LINUX.

For example, for /subdir/file1.txt

```
curl -H "X-Tapis-Token: $JWT" "https://tacc.tapis.io/v3/files/  
utils/linux/aturing-storage/subdir/file1.txt"
```

Running a Linux native operation

Run a native operation on a path. Operations are *chmod*, *chown* or *chgrp*. For a system of type LINUX.

For example, to change the owner of a file located at /file1.txt to aeinstein

```
curl -H "X-Tapis-Token: $JWT" -H "Content-Type:application/json"  
-X POST -d @body.json "https://tacc.tapis.io/v3/files/utils/linux/  
aturing-storage/file1.txt"
```

with a JSON body of

```
{  
  "operation": "CHOWN",  
  "argument": "aeinstein"  
}
```

Content

Get file or directory contents as a stream of data. Not supported for all system types.

File Contents - Serving files

To return the actual contents (raw bytes) of a file:

```
$ curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/files/  
content/my-system/image.jpg > image.jpg
```

Query Parameters

startByte

integer - Start at byte N of the file

count

integer - Return this number of bytes after startByte

zip

boolean - Zip the contents of a folder

Header Parameters

more

integer - Return 1 KB chunks of UTF-8 encoded text from a file starting after page *more*. This call can be used to page through a text based file. Note that if the contents of the file are not textual (such as an image file or other binary format), the output will be bizarre.

Download using ZIP

The query parameter *zip* may be used to request a stream compressed using the ZIP file format. This is not allowed if system *rootDir* plus *path* would result in all files on the host being included. Please download individual directories, files or objects.

For example, on a linux system a directory may be downloaded as a compressed archive using a command similar to the following:

```
$ curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/files/content/my-linux-system/my_dir > my_dir.zip
```

The program *unzip* may then be used to extract the contents.

If the path being downloaded is a single file and the contents are placed in a file ending in the extension *.gz* then the *gunzip* utility may also be used to extract the contents.

Transfers

File transfers are used to move data between Tapis systems. They should be used for bulk data operations that are too large for the REST api to perform. Transfers occur *asynchronously*, and are executed concurrently where possible to increase performance. As such, the order in which the files are transferred is not deterministic.

When a transfer is initiated, a *bill of materials* is created that creates a record of all the files from the *sourceURI* that are to be transferred to the *destinationURI*. Unless otherwise specified, all files in the *bill of materials* must transfer successfully in order for the overall transfer to be considered successful. A transfer task has an attribute named *status* which is updated as the transfer progresses. The possible states for a transfer are:

ACCEPTED

The initial request has been processed and saved.

IN_PROGRESS

The bill of materials has been created and transfers are either in flight or waiting to begin.

FAILED

The transfer failed.

COMPLETED

The transfer completed successfully, all files have been transferred to the target system.

Unauthenticated HTTP endpoints are also possible to use as a source for transfers. This method can be utilized to include outputs from other APIs into Tapis jobs.

Note

For transfers involving Globus, both the source and destination system must be of type GLOBUS.

The number of files included in the *bill of materials* will depend on the system types and the *sourceURI* values provided in the transfer request. If the source system supports directories and *sourceURI* is a directory then the directory will be processed recursively and all files will be added to the *bill of materials*. If the source system is of type S3 then all objects matching the *sourceURI* path as a prefix will be included.

System types and supported functionality

As discussed above, the files included in a transfer will depend on the source system types and the *sourceURI* values provided in the transfer request. Here is a summary of the behavior:

LINUX/IRODS to LINUX/IRODS

When the *sourceURI* is a directory a recursive listing is made and the files and directory structure are replicated on the *destinationURI* system.

S3 to LINUX/IRODS

All objects matching the *sourceURI* path as a prefix will be created as files on the *destinationURI* system.

LINUX/IRODS to S3

When the *sourceURI* is a directory a recursive listing is made. For each entry in the listing the path relative to the source system rootDir is mapped to a key for the S3 destination system. In other words, a recursive listing is made for the directory on the *sourceURI* system and for each non-directory entry an object is created on the S3 *destinationURI* system.

S3 to S3

All objects matching the *sourceURI* path as a prefix will be re-created as objects on the *destinationURI* system.

HTTP/S to ANY

Transfer of a directory is not supported. Destination system may not be of type GLOBUS. The content of the object from the *sourceURI* URL is used to create a single file or object on the *destinationURI* system.

ANY to HTTP/S

Transfers not supported. Tapis does not support the use of protocol http/s for the *destinationURI*.

Creating Transfers

Lets say our user aturing needs to transfer data between two systems that are registered in tapis. The source system has an id of aturing-storage with the results of an experiment located in directory /experiments/experiment-1/ that should be transferred to a system with id aturing-compute

```
curl -H "X-Tapis-Token: $JWT" -H "Content-Type:application/json"
-X POST -d @body.json https://tacc.tapis.io/v3/files/tranfers
```

```
{
  "tag": "An optional identifier",
  "elements": [
    {
      "sourceURI": "tapis://aturing-storage/experiments/
        experiment-1/",
      "destinationURI": "tapis://aturing-compute/"
    }
  ]
}
```

The request above will initiate a transfer that copies all files and folders in the experiment-1 folder on the source system to the root directory of the destination system aturing-compute

HTTP Source

Unauthenticated HTTP/S endpoints can also be used as a source for a file transfer request. This can be useful, for instance, when the inputs for a job are from a separate web service, or perhaps stored in a public S3 bucket. Note that in this case the *sourceURI* does not refer to a Tapis system.

```
curl -H "X-Tapis-Token: $JWT" -H "Content-Type:application/json"
-X POST -d @body.json https://tacc.tapis.io/v3/files/tranfers
```

```
{
  "tag": "An optional identifier",
  "elements": [
    {
```

```

        "sourceURI": "https://some-web-application.io/calculations/
        12345/results.csv",
        "destinationURI": "tapis://aturing-compute/inputs.csv"
    }
]
}

```

The request above will place the output of the source URI into a file called `inputs.csv` in the `aturing-compute` system.

Getting transfer information

To retrieve information about a transfer including status and bytes transferred, simply make a GET request to the transfers API with the UUID of the transfer.

```
curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/files/
transfers/{UUID}
```

The JSON response should look something like :

```

{
  "status": "success",
  "message": "ok",
  "result": {
    "id": 1,
    "username": "aturing",
    "tenantId": "tacc",
    "tag": "some tag",
    "uuid": "b2dcf71a-bb7b-409a-8c01-1bbs97e749fb",
    "status": "COMPLETED",
    "parentTasks": [
      {
        "id": 17,
        "tenantId": "tacc",
        "username": "aturing",
        "sourceURI": "tapis://sourceSystem/file1.txt",
        "destinationURI": "tapis://destSystem/folderA/",
        "totalBytes": 100000,
        "bytesTransferred": 100000,
        "taskId": 1,
        "children": null,
        "errorMessage": null,
        "uuid": "8fdccda6-a504-4ddf-9464-7b22sa66bcc4",
        "status": "COMPLETED",
        "created": "2021-04-22T14:21:58.933851Z",
        "startTime": "2021-04-22T14:21:59.862356Z",
        "endTime": "2021-04-22T14:22:09.389847Z"
      }
    ],
    "estimatedTotalBytes": 100000,
    "totalBytesTransferred": 100000,
    "totalTransfers": 1,
  }
}

```

```

    "completeTransfers": 1,
    "errorMessage": null,
    "created": "2021-04-22T14:21:58.933851Z",
    "startTime": "2021-04-22T14:21:59.838928Z",
    "endTime": "2021-04-22T14:22:09.376740Z"
  },
  "version": "1.1-094fd38d",
  "metadata": {}
}

```

Handling of symbolic links on Linux Systems

Transfer will always "follow links". If the source of the transfer is a symbolic link to a file or directory, it transfer the file or directory that is pointed to. If it doesn't exist, it will be an error (i.e. the link points to something that doesn't exist). In the case of a directory transfer where one of the entries in the directory encountered is a symbolic link it will be resolved in exactly the same way - the file is added to the archive to be downloaded, or the directory is walked, and it's content added to the archive. Symbolic links can create situations where infinite recursion can occur - for example, suppose you have a directory with a link that points to "../". That means that each time it's expanded the current directory will be added, and the link will be expanded again. Transfers (and really all file operations that involve recursing subdirectories) are limited by a recursion depth. The current maximum depth is 20.

Support for Globus

Please note that your Tapis site installation must have been configured by the site administrator to support Globus. Please see [Globus_Config](#).

The integration of Globus and Tapis allows users to configure and use Globus endpoints and collections just as they would other types of storage systems defined in Tapis. As mentioned previously, not all operations are supported for all system types. For systems of type GLOBUS, the following operations are supported:

- listing
- mkdir
- move
- delete
- transfer between GLOBUS systems

For more information on setting up and registering credentials for a system of type GLOBUS, please see [Systems_Globus](#).

File Permissions

The permissions model allows for fine grained access control of paths on a Tapis system. The system owner may grant READ and MODIFY permission to specific users. MODIFY implies READ.

Please note that Tapis permissions are independent of native permissions enforced by the underlying system host.

Getting permissions

Get the Tapis permissions for a user for the system and path. If no user specified then permissions are retrieved for the user making the request.

```
curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/files/
perms/aturing-storage/experiment1?username=aeinstein
```

Granting permissions

Lets say our user aturing has a system with ID aturing-storage. Alan wishes to allow his collaborator aeinstein to view the results of an experiment located at / experiment1

```
curl -H "X-Tapis-Token: $JWT" -H "Content-Type:application/json"
-X POST -d @body.json https://tacc.tapis.io/v3/files/perms/
aturing-storage/experiment1/
```

with a JSON body with the following shape:

```
{
  "username": "aeinstein",
  "permission": "READ"
}
```

Other users can also be granted permission to write to the system by granting the MODIFY permission. The JSON body would then be:

```
{
  "username": "aeinstein",
  "permission": "MODIFY"
}
```

Revoking permissions

Our user aturing now wishes to revoke his former collaborators access to the folder above. He can issue a DELETE request on the path and specify the username in order to revoke access:

```
curl -H "X-Tapis-Token: $JWT" -X DELETE https://tacc.tapis.io/v3/files/perms/aturing-storage/experiment1?username=aeinstein
```

File Path Sharing

In addition to fine grained permissions support, Tapis also supports a higher level approach to granting access. This approach is known simply as *sharing*. The sharing API allows you to share a file path with a set of users as well as share publicly with all users in a tenant. Sharing provides READ access. When the system has a dynamic *effectiveUserId*, sharing also allows for MODIFY access to all paths for calls made through the Files service.

Note

Note that there is one other case when a system is treated as having a dynamic *effectiveUserId* in the context of sharing, even with a static *effectiveUserId*. This is when the system type is IRODS and the attribute *useProxy* is set to `true`. In this case the connection to the *IRODS* host is made using a special administrative account which then acts as the Tapis user. So please be aware that for this type of system sharing a file path will allow for MODIFY access.

Warning

In the context of using a shared application to run a job, sharing a path will grant users READ and MODIFY access to the path, even for the case of a static *effectiveUserId*.

Note

Tapis permissions and sharing are independent of native permissions enforced by the underlying system host.

For more information on sharing please see [sharing](#)

Getting share information

Retrieve all sharing information for a path on a system. This includes all users with whom the path has been shared and whether or not the path has been made publicly available.

```
curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/files/share/aturing-storage/experiment1
```

Sharing a path with users

Create or update sharing information for a path on a system. The path will be shared with the list of users provided in the request body. Requester must be owner of the system. For LINUX systems path sharing is hierarchical. Sharing a path grants users READ access to the path or, in the context of running a job, it grants users READ and MODIFY access to the path.

```
curl -H "X-Tapis-Token: $JWT" -H "Content-Type:application/json"
-X POST -d @body.json https://tacc.tapis.io/v3/files/share/
aturing-storage/experiment1/
```

with a JSON body with the following shape:

```
{
  "users": [ "aeinstein", "rfeynman" ]
}
```

Sharing a path publicly

Share a path on a system with all users in the tenant. Requester must be owner of the system. Sharing a path grants users READ access to the path or, in the context of running a job, it grants users READ and MODIFY access to the path.

```
curl -H "X-Tapis-Token: $JWT" -X POST https://tacc.tapis.io/v3/
files/share_public/aturing-storage/experiment1/
```

Unsharing a path with users

Update sharing information for a path on a system. The path will be unshared with the list of users provided in the request body. Requester must be owner of the system.

```
curl -H "X-Tapis-Token: $JWT" -H "Content-Type:application/json"
-X POST -d @body.json https://tacc.tapis.io/v3/files/unshare/
aturing-storage/experiment1/
```

with a JSON body with the following shape:

```
{
  "users": [ "rfeynman" ]
}
```

Unsharing a path publicly

Remove public sharing for a path on a system. Requester must be owner of the system.

```
curl -H "X-Tapis-Token: $JWT" -X POST https://tacc.tapis.io/v3/
files/unshare_public/aturing-storage/experiment1/
```

Removing all shares for a path

Remove all shares for a path on a system including public access. If the path is a directory this will also be done for all sub-paths.

```
curl -H "X-Tapis-Token: $JWT" -X POST https://tacc.tapis.io/v3/
files/unshare_all/aturing-storage/experiment1/
```

PostIts

The PostIts service is a URL service that allows you to create pre-authenticated, disposable URLs to files, directories, buckets, etc in the Tapis Platform's Files service. You have control over the lifetime and number of times the URL can be redeemed, and you can expire a PostIt at any time. The most common use of PostIts is to create URLs to files so that you can share with others without having to upload them to a third-party service.

Creating PostIts

To create a PostIt, send a POST request to the Files service's Create PostIt endpoint. The url will contain the systemId and the path that will be shared. The body of the post will contain a json document which describes how long the PostIt is valid, and how many times it can be redeemed. There are default values for each of these parameters if they are not included. If the number of times the PostIt may be redeemed (allowedUses) is set to -1, the PostIt may be redeemed an unlimited number of times. The expiration (validSeconds) must always contain a value. If one is not provided, there is a default that is used. The maximum value of 'validSeconds' is the maximum integer value in Java (Integer.MAX_VALUE = 2147483647).

Note

The maximum value would result in a PostIt that would be valid for nearly 70 years, however it is important to remember that the authentication and authorization are built into the PostIt redeem url. This means anyone who has the url could redeem it. For this reason, it's advisable to keep the uses and expiration times to the minimum required.

Default parameters:

- allowedUses: 1 (One use)
- validSeconds: 2592000 (30 days)

APPLICATION/JSON examples

Creating a postit with the default expiration and uses on a system called "tapisv3-storage" for the path "/myDirectory/myFile.txt"

Using curl

```
curl -X POST "https://tacc.tapis.io/v3/files/postits/tapisv3-storage/myDirectory/myFile.txt" -H "X-Tapis-Token: $JWT" -H "Content-Type: application/json"
```

Using python

```
tapis.files.createPostIt(systemId="tapisv3-storage",  
    path="myDirectory/myFile.txt")
```

Creating a postit supplying expiration (validSeconds 600) and uses (allowedUses 3) on a system called "tapisv3-storage" for the path "/myDirectory/myFile.txt"

Using curl

```
curl -X POST "https://tacc.tapis.io/v3/files/postits/tapisv3-storage/myDirectory/myFile.txt" -H "X-Tapis-Token: $JWT" -H "Content-Type: application/json" -d '{"allowedUses": 3, "validSeconds": 600}'
```

Using python

```
tapis.files.createPostIt(systemId="tapisv3-storage",  
    path="myDirectory/myFile.txt", allowedUses=3,  
    validSeconds=600)
```

Creating a postit supplying allowing unlimited uses (allowedUses -1) and the default value for expiration (default value for validSeconds is 30 days) on a system called "tapisv3-storage" for the path "/myDirectory/myFile.txt"

Using curl

```
curl -X POST "https://tacc.tapis.io/v3/files/postits/tapisv3-storage/myDirectory/myFile.txt" -H "X-Tapis-Token: $JWT" -H "Content-Type: application/json" -d '{"allowedUses": -1}'
```

Using python

```
tapis.files.createPostIt(systemId="tapisv3-storage",  
    path="myDirectory/myFile.txt", allowedUses=-1)
```

Example Postit Creation Response

```
{  
  "status": "success",  
  "message": "FAPI_POSTITS_OP_COMPLETE Operation completed.  
jwtTenant: tacc jwtUser: example_user OboTenant: tacc OboUser:  
example_user Operation: createPostIt System: tapisv3-storage  
Path: myDirectory/myFile.txt Id: e614ce8e-447c-4195-a3f7-55f5dec5d243-010",  
  "result": {  
    "id": "e614ce8e-447c-4195-a3f7-55f5dec5d243-010",
```

```

    "systemId": "tapisv3-storage",
    "path": "myDirectory/myFile.txt",
    "allowedUses": 3,
    "timesUsed": 0,
    "jwtUser": "example_user",
    "jwtTenantId": "tacc",
    "owner": "example_user",
    "tenantId": "tacc",
    "redeemUrl": "https://tacc.tapis.io/v3/files/postits/redeem/
e614ce8e-447c-4195-a3f7-55f5dec5d243-010",
    "expiration": "2023-03-08T15:37:28.533641Z",
    "created": "2023-03-08T15:27:28.534250Z",
    "updated": "2023-03-08T15:27:28.534250Z"
  },
  "version": "1.3.1",
  "commit": "0c13ee3c",
  "build": "2023-03-07T21:56:41Z\n",
  "metadata": {}
}

```

Note

The PostIt returned by the create will contain the redeemUrl. This url may be used to download the content pointed to by the PostIt. No Authentication will be done during this call. The credentials used to access this content will be the credentials of the PostIt owner. If the owner's permissions change between creating the PostIt and redeeming the PostIt so that the owner is no longer allowed to read the content, redeeming the PostIt will fail.

Create PostIt parameters

Name	Type	Location	Default	Description
systemId	String	url	<none>	The systemId of the system containing the path to create the PostIt for.
path	String	url	<none>	The path to create the PostIt for.
allowedUses	integer	body	1	The number of times a postit can be redeemed. Valid values are 1 - 2147483647, or -1 for unlimited uses.
validSeconds	integer	body	2147483647	The number of seconds from creation that the PostIt will be redeemable. An expiration time is computed by adding this value to the current date and time.

Listing PostIts

PostIts can be listed by authenticated users. By default a listing of all PostIts owned by the authenticated user will be returned.

Using curl

```
curl "https://tacc.tapis.io/v3/files/postits" -H "X-Tapis-Token: $JWT"
```

Using python

```
tapis.files.listPostIts()
```

To list all PostIts that are visible to the authenticated user, supply the query parameter `listType` and set it's value to `ALL`. Typically users will only be able to see PostIts that they own, however tenant admins will be allowed to see all PostIts in their tenant.

Using curl

```
curl "https://tacc.tapis.io/v3/files/postits?listType=ALL" -H "X-Tapis-Token: $JWT"
```

Using python

```
tapis.files.listPostIts(listType="ALL")
```

Paging is handled by the query parameters `limit`, `skip`, and `startAfter`. By default 100 PostIts are returned, however this can be changed by setting the query parameter `"limit"`. `Skip` is used to determine how many PostIts to skip. For example to get the second page of a list of PostIts containing 10 items per page, you would need to set the limit to 10 (10 items per page), and set skip to 10 (skip the first page). It's probably a good practice to set `orderBy` also, so that the list is ordered in the same way each time. You could for example set `orderBy` to `id`.

Using curl

```
curl "https://tacc.tapis.io/v3/files/postits?listType=ALL&limit=10&skip=10&orderBy=id" -H "X-Tapis-Token: $JWT"
```

Using python

```
tapis.files.listPostIts(listType="ALL", limit=10, skip=10, orderBy="id")
```

Using `startAfter` is similar to using `skip`. When using `startAfter`, you must provide a PostIt id, and the list will start immediately after that PostIt. You **must** provide the `orderBy` parameter. You may not use `skip` and `startAfter` together.

To control which fields are returned, you can supply the `select` query parameter, and select only certain fields. Setting the `select` query parameter to `id, redeemUrl` and `expiration` would return only those fields (`select=id, redeemUrl, expiration`). Setting `"select"` to `allAttributes` will return all attributes, and setting `"select"` to

summaryAttributes will only return a preset collection of attributes. The default is summaryAttributes. You can also set the value to summaryAttributes with additional attributes (select=summaryAttributes,updated).

Using curl

```
curl "https://tacc.tapis.io/v3/files/postits?listType=ALL&select=id,redeemUrl,expiration" -H "X-Tapis-Token: $JWT"
```

Using python

```
tapis.files.listPostIts(listType="ALL",  
                        select="id,redeemUrl,expiration")
```

Retrieving a Single PostIt

Retrieving a single PostIt can be done by issuing a GET request containing the id of the PostIt. This is **not** the same as redeeming, and does not add to the redeem count. This will allow the owner of the PostIt or a tenant admin to view the PostIt. This could be used to see the number of times it's been retrieved, total number of uses allowed, expiration date, etc.

Using curl

```
curl "https://tacc.tapis.io/v3/files/postits/e614ce8e-447c-4195-a3f7-55f5dec5d243-010" -H "X-Tapis-Token: $JWT"
```

Using python

```
tapis.files.getPostIt(postitId="e614ce8e-447c-4195-a3f7-55f5dec5d243-010")
```

For tenant admins, any PostIts in the tenant can be retrieved in this way. For other users only PostIts that are owned by that user may be retrieved, since access to the redeem url allows redemption of the PostIt.

Updating PostIts

The creator of a PostIt and tenant admins can update a PostIt. When updating a PostIt, the id of the posted is sent as part of the url, and a body containing allowedUses and/or validSeconds can be specified.

Note

The validSeconds parameter will add to the date and time as of the update request to compute the new expiration. It does **not** extend the current expiration by that many seconds.

If you need to update the url, you will need to delete or expire this PostIt and create a new one.

Update a PostIt to allow for 10 uses and to expire in 1 hour

Using curl / PATCH

```
curl -X PATCH "https://tacc.tapis.io/v3/files/postits/e614ce8e-447c-4195-a3f7-55f5dec5d243-010" -H "Content-Type: application/json" -H "X-Tapis-Token: $JWT" -d '{ "allowedUses":10, "validSeconds":3600 }'
```

Using curl / POST

```
curl -X POST "https://tacc.tapis.io/v3/files/postits/e614ce8e-447c-4195-a3f7-55f5dec5d243-010" -H "Content-Type: application/json" -H "X-Tapis-Token: $JWT" -d '{ "allowedUses":10, "validSeconds":3600 }'
```

Using python

```
tapis.files.updatePostIt(postitId="e614ce8e-447c-4195-a3f7-55f5dec5d243-010", allowedUses=10, validSeconds=3600)
```

Redeeming PostIts

To redeem a PostIt, use the redeemUrl from the PostIt to make a non-authenticated HTTP GET request.

Using curl

```
curl -JO "https://tacc.tapis.io/v3/files/postits/redeem/e614ce8e-447c-4195-a3f7-55f5dec5d243-010"
```

Note

The options -J and -O (specified above as -JO) tell curl to download the file content and use the filename in the content-disposition header. It's worth noting that using this filename is not entirely without risk as it could overwrite a file of the same name. If you would prefer to use a name that you specify, you could replace -JO with --output filename

Using python

```
tapis.files.redeemPostIt(postitId="e614ce8e-447c-4195-a3f7-55f5dec5d243-010")
```

By default if the PostIt you are redeeming points to a path that is a directory, you will get a zip file, and if it's a regular file, you will get an uncompressed file. If you want to force a file to be compressed, you can specify the query parameter zip and set it to true.

Using curl

```
curl -JO "https://tacc.tapis.io/v3/files/postits/redeem/e614ce8e-447c-4195-a3f7-55f5dec5d243-010?zip=true"
```

Using python

```
tapis.files.redeemPostIt(postitId="e614ce8e-447c-4195-  
a3f7-55f5dec5d243-010", zip=True)
```

Note

If you specify `zip=false` for a PostIt that points to a directory, you will get an error. Directories can't be returned unless they are compressed.

The redeem URL can also be pasted into the address bar of your favorite browser, and it will download the file pointed to by the PostIt.

Handling of symbolic links on Linux Systems

Redeeming PostIts handles symbolic links exactly the same as [Transfers](#)

Expiring PostIts

There is no special endpoint for expiring a PostIt. To Expire a PostIt just update see [Updating PostIts](#) and set `validSeconds` to 0, or `allowedUses` to 0.

Deleting PostIts

PostIts can be deleted by specifying the PostIt id in the url. This is a hard delete, and cannot be undone. Only an owner or tenant admin can delete a PostIt.

Using curl

```
curl -X DELETE "https://tacc.tapis.io/v3/files/postits/  
e614ce8e-447c-4195-a3f7-55f5dec5d243-010" -H "X-Tapis-Token: $JWT"
```

Using python

```
tapis.files.deletePostIt(postitId="e614ce8e-447c-4195-  
a3f7-55f5dec5d243-010")
```

Footnotes

1. With the exception of the *sourceURI* in a transfer request when the protocol is *http* or *https*.[\[?\]](#)

Applications

In order to run a job on a system you will need to create or have access to a Tapis **application**.

Overview

A Tapis application represents all the information required to run a Tapis job on a Tapis system and produce useful results. Each application is versioned and is associated with a specific tenant and owned by a specific user who has special privileges for the application. In order to support this purpose an application definition includes information which allows the *Jobs* service to:

- Stage input prior to launching the application
- Launch the application
- Monitor the application during execution
- Archive output after application execution

Model

At a high level an application contains some information that is independent of the version and some information that varies by version.

Non-Versioned Attributes

id

A short descriptive name for the application that is unique within the tenant.

owner

A specific user set at application creation. Default is `${apiUserId}`, the user making the request to create the application.

enabled

Indicates if application is currently considered active and available for use. Default is *true*.

deleted

Indicates if application has been soft deleted.

created

When the application was created. Maintained by service.

updated

When the application was last updated. Maintained by service.

Versioned Attributes

version

Applications are expected to evolve over time. Id + version must be unique within a tenant.

description

An optional more verbose description for the application.

versionEnabled

Indicates if specific version is currently considered active and available for use. Default is *true*.

locked

Indicates if version is currently locked. When locked a version may not be modified. Default is *false*.

runtime

Runtime to be used when executing the application. DOCKER, SINGULARITY, ZIP. Default is DOCKER. For more information on how Tapis supports the various runtime types, please see [JobsRuntimeSupport](#).

runtimeVersion

Runtime version to be used when executing the application.

runtimeOptions

Options that apply to the runtime. Note that currently there are no runtimes that require this.

Warning

Please note that support for SINGULARITY_START has been removed. If you have a need for this option please contact Tapis support (cicsupport@tacc.utexas.edu) about an enhancement request.

containerImage

Reference to be used when running the container image.

jobType

FORK or BATCH. Jobs submitted will be of this type by default. May be overridden in the job submit request. This allows an application designer to test an application run as a FORK job, for example, and then move on to running as a BATCH job which typically involves further design work. Default is FORK.

maxJobs

Maximum total number of jobs that can be queued or running for this application on a given execution system at a given time. Note that the execution system may also limit the number of jobs on the system which may further restrict the total number of jobs. Set to -1 for unlimited. Default is unlimited.

maxJobsPerUser

Maximum total number of jobs associated with a specific job owner that can be queued or running for this application on a given execution system at a given time. Note that the execution system may also limit the number of jobs on the system which may further restrict the total number of jobs. Set to -1 for unlimited. Default is unlimited.

strictFileInputs

Indicates if a job request is allowed to have unnamed file inputs. If set to true then a job request may only use the named file inputs defined in the application. See attribute *fileInputs* in the JobAttributes table. Default is *false*.

Job related attributes

Various attributes related to job execution such as *execSystemId*, *execSystemExecDir*, *execSystemInputDir*, *execSystemLogicalQueue*, *archiveSystemId*, *fileInputs*, etc.

Note

Currently dynamic selection of an execution system is not supported. For this reason the job related attribute *dynamicExecSystem* should be set to *false* (the default) and *execSystemConstraints* should not be set.

Required Attributes

When creating a application the required attributes are: *id*, *version* and *containerImage*. Depending on the type of application and specific values for certain attributes there are other requirements.

The restrictions are:

- If *archiveSystemId* is specified then *archiveSystemDir* must be specified.

Attributes and the Job Execution Environment

The runtime environment of a Tapis job is determined by values in the job submit request, the application definition and the execution system definition. Generally speaking, for values that can be assigned in multiple places, the values in the job submit request override those in the application definition, which in turn override those in the system definition. There are special cases, however, where the values from different definitions are merged.

See the jobs/apps/systems parameter [matrix](#) for a detailed description of how each attribute is handled.

Getting Started

Before going into further details about applications, here we give some examples of how to create and view applications. In the examples below we assume you are using the TACC tenant with a base URL of `tacc.tapis.io` and that you have authenticated using PySDK or obtained an authorization token and stored it in the environment variable `JWT`, or perhaps both.

Creating an application

Create a local file named `app_sample.json` with json similar to the following:

```
{
  "id": "tacc-sample-app-<userid>",
  "version": "0.1",
  "description": "My sample application",
  "runtime": "DOCKER",
  "containerImage": "docker.io/hello-world:latest",
  "jobType": "FORK",
  "jobAttributes": {
    "description": "default job description",
    "execSystemId": "execsystem1"
  }
}
```

where `<userid>` is replaced with your user name.

Note

If specified, `execSystemId` must reference a system that exists and has `canExec` set to true. If `execSystemId` is not specified, then it must be provided as part of the job submit request.

Using PySDK:

```
import json
from tapipy.tapis import Tapis
```

```
t = Tapis(base_url='https://tacc.tapis.io', username='<userid>',
          password='*****')
with open('app_sample.json', 'r') as openfile:
    my_app = json.load(openfile)
t.apps.createAppVersion(**my_app)
```

Using CURL:

```
$ curl -X POST -H "content-type: application/json" -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/apps -d @app_sample.json
```

Viewing Applications

Retrieving details for an application

To retrieve details for a specific application, such as the one above:

Note

See the section below on [Selecting](#) to find out how to control the amount of information returned.

Using PySDK:

```
t.apps.getAppLatestVersion(appId='tacc-sample-app-<userid>')
```

Using CURL:

```
$ curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/apps/tacc-sample-app-<userid>
```

The response should look similar to the following:

```
{
  "result": {
    "tenant": "tacc",
    "id": "tacc-sample-app-<userid>",
    "version": "0.1",
    "description": "My sample application",
    "owner": "<userid>",
    "enabled": true,
    "versionEnabled": true,
    "locked": false,
    "runtime": "DOCKER",
    "runtimeVersion": null,
    "runtimeOptions": [],
    "containerImage": "docker.io/hello-world:latest",
    "jobType": "FORK",
    "maxJobs": 0,
    "maxJobsPerUser": 0,
    "strictFileInputs": false,
    "jobAttributes": {
      "description": "default job description",
```

```

"dynamicExecSystem": false,
"execSystemConstraints": [],
"execSystemId": "execsystem1",
"execSystemExecDir": null,
"execSystemInputDir": null,
"execSystemOutputDir": null,
"dtnSystemInputDir": null,
"dtnSystemOutputDir": null,
"execSystemLogicalQueue": null,
"archiveSystemId": null,
"archiveSystemDir": null,
"archiveOnAppError": false,
"archiveMode": "NEVER",
"isMpi": false,
"mpiCmd": null,
"cmdPrefix": null,
"parameterSet": {
  "appArgs": [
    {
      "arg": "arg1",
      "name": "appArg1Name",
      "description": "description for app arg 1",
      "inputMode": "FIXED",
      "notes": {
        "arg1note": "my first arg"
      }
    }
  ],
  "containerArgs": [],
  "schedulerOptions": [],
  "envVariables": [
    {
      "key": "ENV_1",
      "value": "env_1_value",
      "description": "my env var #1",
      "inputMode": "INCLUDE_BY_DEFAULT",
      "notes": {}
    },
    {
      "key": "APP_ONLY_FIXED_SET",
      "value": "app_only_fixed_set",
      "description": "FIXED env var only in app
definition",
      "inputMode": "FIXED",
      "notes": {
        "app_only_fixed_notes": "testing_it"
      }
    },
    {
      "key": "APP_REQUIRED_SET",
      "value": "app_required_set",
      "description": "",
      "inputMode": "REQUIRED",
      "notes": {}
    }
  ]
}

```

```

        }
    ],
    "archiveFilter": {
        "includes": [],
        "excludes": [],
        "includeLaunchFiles": true
    }
},
"fileInputs": [
    {
        "name": "empty",
        "description": "An empty file",
        "inputMode": "OPTIONAL",
        "autoMountLocal": true,
        "notes": {},
        "sourceUrl": "tapis://test-storage-linux/
data_input/empty.txt",
        "targetPath": "empty.txt"
    },
    {
        "name": "file1",
        "description": "A random text file",
        "inputMode": "REQUIRED",
        "autoMountLocal": true,
        "notes": {},
        "sourceUrl": "tapis://test-storage-linux/
data_input/file1.txt",
        "targetPath": "file1.txt"
    },
    {
        "name": "s3_ceph_file",
        "description": "A file from an s3 ceph storage
system.",
        "inputMode": "REQUIRED",
        "autoMountLocal": true,
        "notes": {},
        "sourceUrl": "tapis://test-storage-s3-ceph/
object1",
        "targetPath": "s3_ceph_file.dat"
    },
    {
        "name": "s3_aws_test1",
        "description": "File from an s3 aws storage
system.",
        "inputMode": "REQUIRED",
        "autoMountLocal": true,
        "notes": {},
        "sourceUrl": "tapis://test-s3-storage/object2",
        "targetPath": "s3_aws/test1.dat"
    }
],
"fileInputArrays": [
    {
        "name": "fileInputArray1",

```

```

        "description": "A list of files in a single
directory",
        "sourceUrls": [
            "tapis://test-storage-linux/data_input/
file1a.txt",
            "tapis://test-storage-linux/data_input/
file2a.txt",
            "tapis://test-storage-linux/data_input/
file3a.txt"
        ],
        "targetDir": "myFileInputArrayDir/subdir1"
    }
],
    "nodeCount": 1,
    "coresPerNode": 1,
    "memoryMB": 100,
    "maxMinutes": 10,
    "subscriptions": [
        {
            "description": "Email on job new status",
            "enabled": true,
            "jobEventCategoryFilter": "JOB_NEW_STATUS",
            "deliveryTargets": [
                {
                    "deliveryMethod": "EMAIL",
                    "deliveryAddress": "me@example.com"
                }
            ],
            "ttlMinutes": 10080
        }
    ],
    "tags": []
},
    "tags": [],
    "notes": {},
    "uuid": "40a60a11-41fe-45ea-8674-d2cfe04992f6",
    "deleted": false,
    "created": "2021-04-22T21:30:10.590999Z",
    "updated": "2021-04-22T21:30:10.590999Z"
},
    "status": "success",
    "message": "TAPIS_FOUND App found: tacc-sample-app-<userid>",
    "version": "0.0.1-SNAPSHOT",
    "metadata": null
}

```

Retrieving details for all applications

To see the list of applications that you own:

Using PySDK:

```
t.apps.getApp()
```

Using CURL:

```
$ curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/apps?  
select=allAttributes
```

The response should contain a list of items similar to the single listing shown above.

Note

See the sections below on [Searching](#), [Selecting](#), [Sorting](#) and [Limiting](#) to find out how to control the amount of information returned.

Minimal Definition and Restrictions

When creating an application the required attributes are: *id*, *version* and *containerImage* Depending on the type of application and specific values for certain attributes there are other requirements. The restrictions are:

- If *archiveSystemId* is specified then *archiveSystemDir* is required.

Version

The versioning scheme is at the discretion of the application author. The combination of tenant+id+version uniquely identifies an application in the Tapis environment. It is recommended that a two or three level form of semantic versioning be used. The fully qualified application reference within a tenant is constructed by appending a hyphen to the name followed by the version string. For example, the first two versions of an application might be *myapp-0.0.1* and *myapp-0.0.2*. If a version is not specified when retrieving an application then by default the most recently created version of the application will be returned.

Containerized Application

An application that has been containerized is one that can be executed using a single container image. Tapis will use the appropriate container runtime command and provide support for making the input and output directories available to the container when running the container image.

Note

Currently only containerized applications are supported

Directory Semantics and Macros

At job submission time the Jobs service supports the use of macros based on template variables. These variables may be referenced when specifying directories in an application definition. For a full list of supported variables and more information please see the Jobs Service documentation. Here are some examples of variables that may be used when specifying directories for an application:

- *JobUUID* - The Id of the job determined at job submission.
- *JobOwner* - The owner of the job determined at job submission.
- *JobWorkingDir* - Default parent directory from which a job is run. This will be relative to the effective root directory *rootDir* on the execution system. *rootDir* and *jobWorkingDir* are attributes of the execution system.
- *HOST_EVAL(\$<ENV_VARIABLE>)* - The value of the environment variable *ENV_VARIABLE* when evaluated on the execution system host when logging in under the job's effective user ID. This is a dynamic value determined at job submission time. The function *HOST_EVAL()* extracts specific environment variable values for use during job setup. In particular, the TACC specific values of *\$HOME*, *\$WORK*, *\$SCRATCH* and *\$FLASH* can be referenced. The specified environment variable name is used **as-is**. It is **not** subject to macro substitution. However, the function call can have a path string appended to it, such as in *HOST_EVAL(\$SCRATCH)/tmp/\${JobUUID}*, and macro substitution will be applied to the path string.

Data Transfer Node (DTN) Support

Tapis supports the use of a Data Transfer Node (DTN) when running an application. There are two fields in *jobAttributes* related to DTN support:

dtnSystemInputDir

Directory relative to *rootDir* to which input files will be transferred prior to launching the application.

dtnSystemOutputDir

Directory relative to *rootDir* from which output files will be transferred during the archiving phase.

By default, these are set to the special value *!tapis_not_set*. This value indicates that, by default, a DTN system will not be used for file inputs or archive outputs. In order to trigger the use of a DTN during either file input staging or archive output, these values must be set. For more information on using a DTN, please see [DTN Configuration](#).

Permissions

The permissions model allows for fine grained access control of Tapis applications.

At application creation time the owner is given full access to the application. Permissions for other users may be granted and revoked through the applications API. Please note that grants and revokes through this service only impact the default role for the user. A user may still have access through permissions in another role. So even after revoking permissions through this service when permissions are retrieved the access may still be listed. This indicates access has been granted via another role.

Permissions are specified as either * for all permissions or some combination of the following specific permissions: ("READ", "MODIFY", "EXECUTE"). Specifying permissions in all lower case is also allowed. Having MODIFY implies READ.

Sharing

In addition to fine grained permissions support, Tapis also supports a higher level approach to granting access. This approach is known simply as *sharing*. The sharing API allows you to share an application with a set of users as well as share publicly with all users in a tenant. Sharing grants READ+EXECUTE access and, more importantly, facilitates allowing others to run a job using the application.

Sharing an application gives a user certain implicit access to resources in the context of running a job. When a properly designed application is shared it may be used by many users to run jobs without the need to explicitly grant permissions to associated resources such as systems and file paths.

For more information on sharing please see [sharing](#)

Deletion

An application may be deleted and undeleted. Deletion means the application is marked as deleted and is no longer available for use. Note that although this is a soft delete the operation is intended for use when an application is to be permanently made unavailable for use. To temporarily make an application unavailable for use please use support for enabling and disabling an application.

By default deleted applications will not be included in searches and operations on deleted applications will not be allowed. When listing applications the query parameter *showDeleted* may be used in order to include deleted applications in the results. Note that deletion applies to all version of an application. It is not possible to delete a specific version.

Application Attributes Table

Attribute	Type	Example	Notes
tenant	String	designsafe	• Name of the tenant for which the application is defined. • <i>tenant</i> + \$version* + <i>id</i> must be unique.
id	String	my-ds-app	• Name of the application. URI safe, see RFC 3986. • <i>tenant</i> + \$version* + <i>id</i> must be unique. • Allowed characters: Alphanumeric [0-9a-zA-Z] and special characters [-._~]. • Required at creation time.
version	String	0.0.1	• Version of the application. URI safe, see RFC 3986. • <i>tenant</i> + \$version* + <i>id</i> must be unique. • Allowed characters: Alphanumeric [0-9a-zA-Z] and special characters [-._~]. • Required at creation time.
description	String	A sample application	• Optional description
owner	String	jdoe	• User name of <i>owner</i>. • Variable references: <i>\${apiUserId}</i> • Default is <i>\${apiUserId}</i>
enabled	boolean	FALSE	• Indicates if application currently enabled for use. Default is TRUE.
versionEnabled	boolean	FALSE	• Indicates if specific version is currently

Attribute	Type	Example	Notes
			enabled for use. Default is TRUE.
locked	boolean	FALSE	- Indicates if version is currently locked. Locking disallows updates. - Default is FALSE
runtime	enum	SINGULARITY	- Runtime to be used when executing the application. - Runtimes: DOCKER, SINGULARITY, ZIP - Default is DOCKER
runtimeVersion	String	2.5.2	Optional version or range of versions required.
runtimeOptions	[enum]		- Options that apply to specific runtimes. - Options: NONE, SINGULARITY_RUN - For SINGULARITY leave empty or specify SINGULARITY_RUN. WARNING Please note that use of SINGULARITY_START is no longer supported. - If SINGULARITY_START is needed please contact support about an enhancement request. - Default is NONE.
containerImage	String	docker.io/hello-world	- Reference for the container image. Other examples: - Singularity: shub://GodloveD/lolcow - Docker: tapis/hello-tapis:0.0.1 Required at creation time.

Attribute	Type	Example	Notes
jobType	enum	BATCH	• Default job type. • Types: BATCH, FORK • Jobs will be of this type by default. May be overridden in the job submit request. • Default is FORK.
maxJobs	int	10	• Max number of jobs that can be running for this app on a system. • System may also limit the number of jobs. • Set to -1 for unlimited. Default is unlimited.
maxJobsPerUser	int	2	• Max number of jobs per job owner. • System may also limit the number of jobs. • Set to -1 for unlimited. Default is unlimited.
strictFileInputs	boolean	FALSE	• Indicates if a job request is allowed to have unnamed file inputs. • If TRUE then a job request may only use named file inputs defined in the app. • Default is FALSE.
jobAttributes	JobAttributes		• Various attributes related to job execution. • See table below.
tags	[String]		• List of tags as simple strings.
notes	String	{"project": "myproj"}	• Simple metadata in the form of a Json object. • Not used by Tapis.
uuid	UUID	20281	

Attribute	Type	Example	Notes
			• Auto-generated by service.
created	Timestamp	2020-06-19T15:10:43Z	• When the app was created. Maintained by service.
updated	Timestamp	2020-07-04T23:21:22Z	• When the app was last updated. Maintained by service.

JobAttributes Table

Attribute	Type	Example	Notes
description	String		• Description to be filled in when this • Macros allow this to act as a template
execSystemId	String		• Specific system on which the applica
execSystemExecDir	String		• Directory where application assets a • Current working directory at applica • Macro template variables such as \${ • Default is \${JobWorkingDir}/jobs/\$
execSystemInputDir	String		• Directory where Tapis is to stage the • Macro template variables such as \${ • Default is \${JobWorkingDir}/jobs/\$
execSystemOutputDir	String		• Directory where Tapis expects the ap • results. • Files here are candidates for archivin • Macro template variables such as \${ • Default is \${JobWorkingDir}/jobs/\$
dtNSystemInputDir	String		• Directory relative to DTN rootDir to • Transfer happens prior to launching • Can be overridden by job submission • Optional. If set will trigger use of D

Attribute	Type	Example	Notes
			• Default is <i>!tapis_not_set</i>
dtnSystemOutputDir	String		• Directory relative to DTN rootDir for output to be transferred. • Transfer happens during archiving process. • Can be overridden by job submission. • Optional. If set will trigger use of DTN. • Default is <i>!tapis_not_set</i>
execSystemLogicalQueue	String	normal	• LogicalQueue to use when running the application
archiveSystemId	String		• System to use when archiving outputs
archiveSystemDir	String		• Directory on <i>archiveSystemId</i> where outputs will be archived. • This will be relative to the effective archiveSystemId. • Default is <code>\${JobWorkingDir}/jobs/\${JobId}</code>
archiveOnAppError	boolean		• Indicates if outputs should be archived on application error. • The default is TRUE. • ***** • DEPRECATED. Please use archiveMode
archiveMode	enum	NEVER	• Indicates if outputs should be archived on application error. • Options: ALWAYS, SKIP_ON_FAILURE, NEVER • Default is ALWAYS
isMpi	boolean		• Indicates that application is to be executed using MPI • The default is FALSE.
mpiCmd	String	"mpirun" "ibrun -n 4"	• Command used to launch MPI jobs. • Prepend to the command used to execute the application • Conflicts with cmdPrefix if isMpi is set
cmdPrefix	String		• String prepended to the application command • Conflicts with mpiCmd if isMpi is set
parameterSet	ParameterSet		• Various collections used during job execution • App arguments, container arguments, environment variables, etc.

Attribute	Type	Example	Notes
			• See table below.
fileInputs	[FileInput]		• Collection of file inputs that must be • Each input must have a name. • <i>strictFileInputs</i> =TRUE means only - for job. • See table below.
fileInputArrays	[FileInputArray]		• Collection of arrays of inputs that m • Each input must have a name. All in - directory. • <i>strictFileInputs</i> =TRUE means only - for job. • See table below.
nodeCount	int		• Number of nodes to request during j
coresPerNode	int		• Number of cores per node to request
memoryMB	int		• Memory in megabytes to request dur
maxMinutes	int		• Run time to request during job subm
subscriptions			• Notification subscriptions. • See table below.
tags	[String]		• List of tags as simple strings.

ParameterSet Attributes Table

Attribute	Type	Example	Notes
appArgs	[Arg]		• Command line arguments passed to the application • See table below for more information on <i>Arg</i> type • For more information on <i>appArgs</i> please see the
containerArgs	[Arg]		• Command line arguments passed to the container • See table below for more information on <i>Arg</i> type

Attribute	Type	Example	Notes
			• For more information on <i>containerArgs</i> please see ContainerArgs
schedulerOptions	[Arg]		• Scheduler options passed to the HPC batch scheduler. • See table below for more information on <i>Arg</i> type. • For more information on <i>schedulerOptions</i> please see SchedulerOptions
envVariables	[KeyValuePair]		• Environment variables placed into the runtime environment. • Each entry has <i>key</i> (required) and <i>value</i> (optional) attributes. • See table <i>KeyValuePair Attributes</i> below for more information. • For more information on <i>envVariables</i> please see EnvVariables
archiveFilter	ArchiveFilter		• Sets of files to include or exclude when archiving. • Default is to include all files in <i>execSystemOutput</i>. • ***** • See table below for details of <i>ArchiveFilter</i> structure. • For more information on <i>archiveFilter</i> support please see Jobs

ArchiveFilter Attributes Table

Attribute	Type	Example	Notes
includes	[String]		• Files to include when archiving after execution of the application. • excludes list has precedence.
excludes	[String]		• Files to skip when archiving after execution of the application. • excludes list has precedence.
includeLaunchFiles	boolean		• Indicates if Tapis generated launch scripts are to be included when archiving. • The default is TRUE.

Arg Attributes Table

For more information on how the *inputMode* attribute is used during job processing, please see [JobsArgProcessing](#).

Attribute	Type	Example	Notes
name	String		- Identifying label associated with the argument. Required at creation time.
description	String		Optional description of the argument which may include usage, etc.
inputMode	enum		- Indicates how argument is to be treated when processing individual requests. - Modes: REQUIRED, FIXED, INCLUDE_ON_DEMAND, INCLUDE_BY_DEFAULT - Default is INCLUDE_ON_DEMAND. ***** - REQUIRED: Must be provided in a job request. - FIXED: Completely defined in the application and not overridden in request. - INCLUDE_ON_DEMAND: Included if referenced in a job request. - INCLUDE_BY_DEFAULT: Included unless <i>include=false</i> in request.
arg	String		- Value for the argument Required at creation time.
notes	String	{"fieldType": "int"}	- Metadata in the form of a Json object, such as type, allowed values, etc. - Not used by Tapis.

KeyValuePair Attributes Table

Attribute	Type	Example	Notes
key	String	"INPUT_FILE"	Environment variable name. Required.
value	String	"/tmp/file.input"	Environment variable value
description	String		

Attribute	Type	Example	Notes
			• Description
inputMode	enum	REQUIRED	• Indicates how argument is to be treated when processing requests. • Modes: REQUIRED, FIXED, INCLUDE_ON_DEMAND, INCLUDE_BY_DEFAULT • Default is INCLUDE_BY_DEFAULT. • ***** • REQUIRED: Must be provided in a job request or a job request. • FIXED: Not overridable in application or job request. • INCLUDE_ON_DEMAND: Included if referenced in a job request. • INCLUDE_BY_DEFAULT: Included unless <i>includeByDefault</i> is false in a job request.
notes	String	"{}"	• Simple metadata in the form of a Json object. • Not used by Tapis.

FileInput Attributes Table

Attribute	Type	Example	Notes
name	String		• Identifying label associated with the input. Typically used during a job request. • Required at creation time.
description	String		• Optional description.
inputMode	enum		• Indicates how input is to be treated when processing individual job requests. • REQUIRED, OPTIONAL, FIXED • Default is OPTIONAL.
autoMountLocal	boolean		• Indicates if Jobs service should automatically mount file paths into containers. • Note that not all container runtimes require this.

Attribute	Type	Example	Notes
			- Setting to FALSE allows user complete control using <i>containerArg</i> parameters. - Default is TRUE.
sourceUrl	String		Source used by Jobs service when staging file inputs.
targetPath	String		Target path used by Jobs service when staging file inputs.

FileInputArray Attributes Table

Attribute	Type	Example	Notes
name	String		- Identifying label associated with the input. Typically used during a job request. Required at creation time.
description	String		Optional description.
inputMode	enum		- REQUIRED, OPTIONAL, FIXED - Default is OPTIONAL.
sourceUrls	[String]		Array of sources used by Jobs service when staging file inputs.
targetDir	String		Target directory used by Jobs service when staging file inputs.

Searching

The service provides a way for users to search for applications based on a list of search conditions provided either as query parameters for a GET call or a list of conditions in a request body for a POST call to a dedicated search endpoint.

Search using GET

To search when using a GET request to the apps endpoint a list of search conditions may be specified using a query parameter named `search`. Each search condition must be:

- surrounded with parentheses
- have three parts separated by the character `.`
- be joined using the character `~`.

All conditions are combined using logical AND. The general form for specifying the query parameter is as follows:

```
?  
search=(<attribute_1>.<op_1>.<value_1>)~(<attribute_2>.<op_2>.<value_2>)~...~(<attribute_N>.<op_N>.<value_N>)
```

Attribute names are given in the table above and may be specified using Camel Case or Snake Case.

Supported operators: `eq neq gt gte lt lte in nin like nlike between nbetween`

Example CURL command to search for applications that have `Test` in the `id`, are of type `FORK` and allow for *maxJobs* greater than 5:

```
$ curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/apps?  
search="(id.like.*Test*)~(job_type.eq.FORK)~(max_jobs.gt.5)"
```

Notes:

- For the `like` and `nlike` operators the wildcard character `*` matches zero or more characters and `!` matches exactly one character.
- For the `between` and `nbetween` operators the value must be a two item comma separated list of unquoted values.
- If there is only one condition the surrounding parentheses are optional.
- In a shell environment the character `&` separating query parameters must be escaped with a backslash.
- In a shell environment the query value must be surrounded by double quotes and the following characters must be escaped with a backslash in order to be properly interpreted by the shell:
 - `" \ ``
- Attribute names may be specified using Camel Case or Snake Case.
- Following complex attributes not supported when searching:
 - `jobAttributes notes`

Dedicated Search Endpoint

The service provides the dedicated search endpoint `apps/search/apps` for specifying complex queries. Using a GET request to this endpoint provides functionality similar to above but with a different syntax. For more complex queries a POST request may be used with a request body specifying the search conditions using an SQL-like syntax.

Search using GET on Dedicated Endpoint

Sending a GET request to the search endpoint provides functionality very similar to that provided for the endpoint apps described above. A list of search conditions may be specified using a series of query parameters, one for each attribute. All conditions are combined using logical AND. The general form for specifying the query parameters is as follows:

```
?  
<attribute_1>.<op_1>=<value_1>&<attribute_2>.<op_2>=<value_2>)& ...  
&<attribute_N>.<op_N>=<value_N>
```

Attribute names are given in the table above and may be specified using Camel Case or Snake Case.

Supported operators: eq neq gt gte lt lte in nin like nlike between nbetween

Example CURL command to search for applications that have Test in the id, are of type FORK and allow for *maxJobs* greater than 5:

```
$ curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/apps/  
search/apps?id.like=*Test*&job_type.eq=FORK&max_jobs.gt=5
```

Notes:

- For the like and nlike operators the wildcard character * matches zero or more characters and ! matches exactly one character.
- For the between and nbetween operators the value must be a two item comma separated list of unquoted values.
- In a shell environment the character & separating query parameters must be escaped with a backslash.
- Attribute names may be specified using Camel Case or Snake Case.
- Following complex attributes not supported when searching:
 - jobAttributes tags notes

Search using POST on Dedicated Endpoint

More complex search queries are supported when sending a POST request to the endpoint apps/search/apps. For these requests the request body must contain json with a top level property name of search. The search property must contain an array of strings specifying the search criteria in an SQL-like syntax. The array of strings are concatenated to form the full search query. The full query must be in the form of an SQL-like WHERE clause. Note that not all SQL features are supported.

For example, to search for apps that are owned by jdoe and of type FORK or owned by jsmith and allow for *maxJobs* less than 5 create a local file named app_search.json with following json:

```
{  
  "search":  
    [  
      "(owner = 'jdoe' AND job_type = 'FORK') OR",  
      "(owner = 'jsmith' AND max_jobs < 5)"  
    ]  
}
```

```
}  
]
```

To execute the search use a CURL command similar to the following:

```
$ curl -X POST -H "content-type: application/json" -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/apps/search/apps -d @app_search.json
```

Notes:

- String values must be surrounded by single quotes.
- Values for BETWEEN must be surrounded by single quotes.
- Search query parameters as described above may not be used in conjunction with a POST request.
- SQL features not supported include:
 - IS NULL and IS NOT NULL
 - Arithmetic operations
 - Unary operators
 - Specifying escape character for LIKE operator

Map of SQL operators to Tapis operators

Sql Operator	Tapis Operator
=	eq
<>	neq
<	lt
<=	lte
>	gt
>=	gte
LIKE	like
NOT LIKE	nlike
BETWEEN	between
NOT BETWEEN	nbetween
IN	in
NOT IN	nin

Sort, Limit, Select and ListType

When a list of Applications is retrieved the service provides for sorting, filtering and limiting the results. By default, only resources owned by you will be included. The service provides a way for you to request that all resources accessible to you be included. This is determined by the query parameter *listType*.

When retrieving either a list of resources or a single resource the service also provides a way to *select* which fields (i.e. attributes) are included in the results. Sorting, limiting and attribute selection are supported using query parameters.

Selecting

When retrieving applications the fields (i.e. attributes) to be returned may be specified as a comma separated list using a query parameter named `select`. Attribute names may be given using Camel Case or Snake Case.

Notes:

- Special select keywords are supported: `allAttributes` and `summaryAttributes`
- Summary attributes include:
 - `id`, `version`, `owner`
- By default all attributes are returned when retrieving a single resource via the endpoint `apps/<app_id>`.
- By default summary attributes are returned when retrieving a list of applications.
- Specifying nested attributes is not supported.
- The attribute `id` is always returned.

For example, to return only the attributes `version` and `containerImage` the CURL command would look like this:

```
$ curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/apps?select=version,containerImage
```

The response should look similar to the following:

```
{
  "result": [
    {
      "id": "TestApp1",
      "version": "0.0.1",
      "containerImage": "containterimage1"
    },
    {
      "id": "JobApp1",
      "version": "0.0.1",
      "containerImage": "containterimage1"
    },
    {
      "id": "JobAppWithInput",
      "version": "0.0.1",
      "containerImage": "containterimage1"
    },
    {
      "id": "SleepSeconds",
      "version": "0.0.1",
      "containerImage": "tapis/testapps:main"
    }
  ]
}
```

```

],
"status": "success",
"message": "TAPIS_FOUND Apps found: 11 applications",
"version": "0.0.1-SNAPSHOT",
"metadata": {
  "recordCount": 4,
  "recordLimit": 100,
  "recordsSkipped": 0,
  "orderBy": null,
  "startAfter": null,
  "totalCount": -1
}
}

```

Sorting

The query parameter for sorting is named `orderBy` and the value is the attribute name to sort on with an optional sort direction. The general format is `<attribute_name>(<dir>)`. The direction may be `asc` for ascending or `desc` for descending. The default direction is ascending.

Examples:

- `orderBy=id`
- `orderBy=id(asc)`
- `orderBy=name(desc),created`
- `orderBy=id(asc),created(desc)`

Limiting

Additional query parameters may be used in order to limit the number and starting point for results. This is useful for implementing paging. The query parameters are:

- `limit` - Limit number of items returned. For example `limit=10`.
 - Use 0 or less for unlimited.
 - Default is 100.
- `skip` - Number of items to skip. For example `skip=10`.
 - May not be used with `startAfter`.
 - Default is 0.
- `startAfter` - Where to start when sorting. For example `limit=10&orderBy=id(asc),created(desc)&startAfter=101`
 - May not be used with `skip`.
 - Must also specify `orderBy`.
 - The value of `startAfter` applies to the major `orderBy` field.
 - Condition is context dependent. For ascending the condition is value `> startAfter` and for descending the condition is value `< startAfter`.

When implementing paging it is recommend to always use `orderBy` and when possible use `limit+startAfter` rather than `limit+skip`. Sorting should always be included since returned results are not guaranteed to be in the same order for each call. The combination of `limit+startAfter` is preferred because `limit+skip` is

more likely to result in inconsistent results as records are added and removed. Using `limit+startAfter` works best when the attribute has a natural sequential ordering such as when an attribute represents a timestamp or a sequential ID.

ListType

By default, you will only see the resources that you own. The query parameter *listType* allows you to see additional resources that are available to you.

Options:

OWNED

Include only items owned by you (Default)

SHARED_PUBLIC

Include only items shared publicly

ALL

Include all items you are authorized to view.

Tapis Responses

For requests that return a list of resources the response result object will contain the list of resource records that match the user's query and the response metadata object will contain information related to sorting and limiting.

The metadata object will contain the following information:

- `recordCount` - Actual number of records returned.
- `recordLimit` - The limit query parameter specified in the request. -1 if query parameter was not specified.
- `recordsSkipped` - The skip query parameter specified in the request. -1 if query parameter was not specified.
- `orderBy` - The orderBy query parameter specified in the request. Empty string if query parameter was not specified.
- `startAfter` - The startAfter query parameter specified in the request. Empty string if query parameter was not specified.
- `totalCount` - Total number of records that would have been returned without a limit query parameter being imposed. -1 if total count was not computed.

For performance reasons computation of `totalCount` is only determined on demand. This is controlled by the boolean query parameter `computeTotal`. By default `computeTotal` is *false*.

Example query and response:

Query:

```
$ curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/apps?
limit=2&orderBy=id(desc)
```

Response:

```
{
  "result": [
    {
      "id": "TestApp1",
      "version": "0.0.1",
      "owner": "testuser2"
    },
    {
      "id": "tacc-sample-app",
      "version": "0.1",
      "owner": "testuser2"
    }
  ],
  "status": "success",
  "message": "TAPIS_FOUND Apps found: 2 applications",
  "version": "0.0.1-SNAPSHOT",
  "metadata": {
    "recordCount": 2,
    "recordLimit": 2,
    "recordsSkipped": 0,
    "orderBy": "id(desc)",
    "startAfter": null,
    "totalCount": -1
  }
}
```

PgREST

The PgREST service provides an HTTP-based API to a managed Postgres database. As with the other Tapis v3 service, PgREST utilizes a REST architecture.

Overview

There are two primary collections in the PgREST API. The Management API, provided at the URL `/v3/pgrest/manage`, includes endpoints for managing the collection of tables, views, stored procedures, and other objects defined in the hosted Postgres database server. Each Tapis tenant has their own schema within PgREST's managed Postgres database in which the tables and other objects are housed. When a table is created, endpoints are generated that allow users to interact with the data within a table. These endpoints comprise the Data API, available at the URL `/v3/pgrest/data`. Each collection, `/v3/pgrest/data/{collection}`, within the Data API corresponds to a table defined in the Management API. The Data API is used to create, update, and read the rows within the corresponding tables.

Authentication and Tooling

PgREST currently recognizes Tapis v2 and v3 authentication tokens and uses these for determining access levels. A valid Tapis v2 OAuth token should be passed to all requests to PgREST using the header `Tapis-v2-token`. For example, using curl:

```
$ curl -H "Tapis-v2-token: 419465dg63h8e4782057degk20e3371"
https://tacc.tapis.io/v3/pgrest/manage/tables
```

Tapis v3 OAuth authentication tokens should be passed to all requests to PgREST using the header `X-Tapis-Token`. For example, using curl:

```
$ curl -H "X-Tapis-Token: TOKEN_HERE" https://tacc.tapis.io/v3/
pgrest/manage/tables
```

Additionally, PgREST should be accessible from the Tapis v3 Python SDK (tapipy) now with the addition of v3 authentication.

Permissions and Roles

PgREST currently implements a handful of basic, role-based permissions that leverage the Tapis v3 Security Kernel (SK).

Universal Roles

For now PgREST establishes the following five universal roles:

- `PGREST_ADMIN` -- Grants user read and write access to all objects (e.g. tables, views, roles) in the `/manage` API as well as read and write access to all associated data in the `/data` API.
- `PGREST_ROLE_ADMIN` -- Grants user role creation and management access to roles in the `/manage/roles` API.
- `PGREST_WRITE` -- Grants user read and write access to all associated data in the `/data` API.
- `PGREST_READ` -- Grants user read access to all associated data in the `/data` API.
- `PGREST_USER` -- Grants permission to user `/views` API. Each view has additional permission rules though.

Without any of the above roles, a user will not have access to any PgREST endpoints.

Fine-Tuned Role Access

Along with the general access to endpoints when a user has a role of `PGREST_READ` or above, we have fine-tuned role access to our get views endpoint.

Our get views endpoint requires only the `PGREST_USER` role. However each view itself when created (or modified) has a `permission_rules` field. This field is a list of roles that the user must have in order to have access to that view. Thus it is possible to divvy out what information a user in the `PGREST_USER` role can get from views by restricting views with roles created by the `/manage/roles` endpoint as a `PGREST_ADMIN` or `PGREST_ROLE_ADMIN`.

Tenant Awareness

Note that these roles are granted at the *tenant* level, so a user may be authorized at one level in one tenant and at a different level (or not at all) in another tenant. In PgREST, the base URLs for a given tenant follow the pattern `<tenant_id>.tapis.io`, just as they do for all other Tapis v3 services. Hence, this request:

```
$ curl -H "Tapis-v2-token: $TOKEN" https://tacc.tapis.io/v3/pgrest/manage/tables
```

would list tables in the TACC tenant, while

```
$ curl -H "Tapis-v2-token: $TOKEN" https://cii.tapis.io/v3/pgrest/
manage/tables
```

would list tables in the CII tenant.

Table Manage API

Table management is accomplished with the `/v3/pgrest/manage/tables` endpoint. Creating a table amounts to specifying the table name, the columns on the table, including the type of each column, and any additional validation to be performed when storing data in the column, the root URL where the associated collection will be available within the Data API, and, optionally, which HTTP verbs should not be available on the collection.

Table Creation Example

For example, suppose we wanted to manage a table of "widgets" with four columns. We could create a table by POSTing the following JSON document to the `/v3/pgrest/manage/tables` endpoint:

```
{
  "table_name": "widgets",
  "root_url": "widgets",
  "columns": {
    "name": {
      "data_type": "varchar",
      "char_len": 255,
      "unique": true,
      "null": false
    },
    "widget_type": {
      "data_type": "varchar",
      "char_len": 100,
      "null": true
    },
    "count": {
      "data_type": "integer",
      "null": true
    },
    "is_private": {
      "data_type": "boolean",
      "null": "true",
      "default": "true"
    }
  }
}
```

The JSON describes a table with 4 columns, `name`, `widget_type`, `count`, and `is_private`. The fields within the JSON object describing each column include its type, defined in the `data_type` attribute (and supporting fields such as `char_len`

for varchar columns), as well as optional constraints, such as the NOT NULL and UNIQUE constraint, an optional default value, and an optional primary_key value. Only the data_type attribute is required.

To create this table and the corresponding /data API, we can use curl like so:

```
$ curl -H "tapis-v2-token: $TOKEN" -H "Content-type: application/json" \
  -d "@widgets.json" https://dev.develop.tapis.io/v3/pgrest/manage/tables
```

If all works, the response should look something like this:

```
{
  "status": "success",
  "message": "The request was successful.",
  "version": "dev",
  "result": {
    "table_name": "widgets",
    "table_id": 6,
    "root_url": "widgets",
    "endpoints": [
      "GET_ONE",
      "GET_ALL",
      "CREATE",
      "UPDATE",
      "DELETE"
    ]
  }
}
```

Since the root_url attribute has value widgets, an associated collection at URL / v3/pgrest/data/widgets is automatically made available for managing and retrieving the data (rows) on the table. See the Data API <target data api> section below for more details.

Table Definition Rules

This is a complete list of constraints and properties a table can have in it's table definition. Each table definition has a host of fields, with the column field have a host of options to delegate how to create the postgres column.

- **table_name - required**
 - The name of the table in question.
- **root_url**
 - The root_url for PgRESTs /data endpoint.
 - Ex: root_url "table25" would be accessible via "http://pgrestURL/data/table25".
- **enums**
 - Enum generation is done in table definitions.
 - Provide a dict of enums where the key is enum name and the value is the possible values for the enum.

- Ex: {"accountrole": ["ADMIN", "USER"]}
- Creates an "accountrole" enum that can have values of "ADMIN" or "USER"
- Deletion/Updates are not currently supported. Speak to developer if you're interested in a delete/update endpoint.
- comments
 - Field to allow for better readability of table json. Table comments are saved and outputted on /manage/tables/ endpoints.
- constraints
 - Specification of Postgres table constraints. Currently only allows multi-column unique constraints
 - Constraints available:
 - unique
 - multi-column unique constraint that requires sets of column values to be unique.
 - Note: Unique constraint must have unique naming between tables. Constraints cannot be used more than once.
 - Ex: "constraints": {"unique": {"two_col_pair": ["col_one", "col_two"]}}
 - This means that col_one and col_two cannot have pairs of values that are identical.
 - The constraint name can be specified as well
- columns - **required**
 - Column definitions in the form of a dict. Dict key would be column, value would be column definition.
 - Ex: {"username": {"unique": true, "data_type": "varchar", "char_len": 255}}
 - Columns arguments are as follows.
 - **data_type - required**
 - Specifies the data type for values in this column.
 - Case insensitive.
 - Can be varchar, datetime, {enumName}, text, timestamp, serial, varchar[], boolean, integer, integer[].
 - Note: varchar requires the char_len column definition.
 - Note: Setting a timestamp data_type column default to UPDATETIME or CREATETIME has special properties.
 - CREATETIME sets the field to the UTC time at creation. It is then not changed later.
 - UPDATETIME sets the field to the UTC time at creation. It is updated to the update time when it is updated.
 - Serial Data type
 - Not actually a serial data type. On the postgres side this is INTEGER GENERATED BY DEFAULT AS IDENTITY (START WITH {serial_start} INCREMENT BY {serial_increment})
 - This is to get around the fact that the serial data type isn't that well received by Postgres
 - This also means that users can specify serial_start and serial_increment to adjust how it works.
 - char_len
 - Additional argument for varchar data_types. Required to set max value size.
 - Can be any value from 1 to 255.

- `serial_start`
 - Additional argument for serial data_types. Defaults to 1.
 - Integer that specifies the start of a SQL sequence.
- `serial_increment`
 - Additional argument for serial data_types. Defaults to 1.
 - Integer that specifies the increment to use for serial data_types. Integer.
- `unique`
 - Determines whether or not each value in this column is unique.
 - Can be true or false.
- `null`
 - States whether or not a value can be "null".
 - Can be true or false.
- `comments`
 - Field to allow for better readability of table and column json. Column comments are not saved or used. They are for json readability only.
- `default`
 - Sets default value for column to fallback on if no value is given.
 - Case insensitive.
 - Must follow the data_type for the column.
 - Note: Setting a timestamp data_type column default to UPDATETIME or CREATETIME has special properties.
 - CREATETIME sets the field to the UTC time at creation. It is then not changed later.
 - UPDATETIME sets the field to the UTC time at creation. It is updated to the update time when it is updated.
- `primary_key`
 - Specifies primary_key for the table.
 - This can only be used for one column in the table.
 - This primary_key column will be the value users can use to get a row in the table, /v3/pgrest/data/my_pk.
 - If this is not specified in a table, primary_key defaults to "{table_name}_id".
 - Note that this default cannot be modified and is of data_type=serial.
- `foreign_key`
 - Whether or not this key should reference a key in another table, a "foreign key".
 - Can be true or false.
 - If foreign_key is set to true, columns arguments reference_table, reference_column, and on_delete must also be set.
 - `reference_table`
 - Only needed in the case that foreign_key is set to true.
 - Specifies the foreign table that the foreign_key is in.
 - Can be set to the table_name of any table.
 - `reference_column`
 - Only needed in the case that foreign_key is set to true.
 - Specifies the foreign column that the foreign_key is in.
 - Can be set to the key for any column in the reference_table.
 - `on_event`
 - Only needed in the case that foreign_key is set to true.

- Case insensitive.
- Specifies the event strategy when referencing a foreign key.
- Can be set to an event of `ON DELETE` or `ON UPDATE`, if the key gets deleted or updated, the `event_action` will be completed by postgres
 - `ON DELETE` Sets off `event_action` if reference is deleted.
 - `ON UPDATE` Sets off `event_action` if reference is updated.
- `event_action`
 - Only needed in the case that `foreign_key` is set to true.
 - Case insensitive.
 - Specifies the event action to complete when a reference gets the `on_event` event.
 - `CASCADE` deletes or updates this column when `on_event` occurs to reference.
 - `SET NULL` set this column to null when `on_event` occurs to reference.
 - `SET DEFAULT` set this column to column default when `on_event` occurs to reference.
 - `RESTRICT` prevents deletion/update of a row when `on_event` occurs to reference.
 - `NO ACTION` does nothing, raises error when referenced, when `on_event` occurs to reference.

Retrieving Table Descriptions

You can list all tables you have access to by making a GET request to `/v3/pgrest/manage/tables`. For example

```
$ curl -H "tapis-v2-token: $tok" https://dev.tapis.io/v3/pgrest/manage/tables
```

returns a result like

```
[
  {
    "table_name": "initial_table",
    "table_id": 3,
    "root_url": "init",
    "tenant": "dev",
    "endpoints": [
      "GET_ONE",
      "GET_ALL",
      "CREATE",
      "UPDATE",
      "DELETE"
    ],
    "tenant_id": "dev"
  },
  {
    "table_name": "widgets",
```

```

    "table_id": 6,
    "root_url": "widgets",
    "tenant": "dev",
    "endpoints": [
        "GET_ONE",
        "GET_ALL",
        "CREATE",
        "UPDATE",
        "DELETE"
    ],
    "tenant_id": "dev"
}
]

```

We can also retrieve a single table by id. For example

```
$ curl -H "tapis-v2-token: $tok" https://dev.tapis.io/v3/pgrest/
manage/tables/6
```

```

{
  "table_name": "widgets",
  "table_id": 6,
  "root_url": "widgets",
  "endpoints": [
    "GET_ONE",
    "GET_ALL",
    "CREATE",
    "UPDATE",
    "DELETE"
  ],
  "tenant_id": "dev"
}

```

We can also pass `details=true` query parameter to see the column definitions and validation schema for a particular table. This can be useful to understand exactly what's happening. The call would be as follows:

```
$ curl -H "tapis-v2-token: $tok" https://dev.tapis.io/v3/pgrest/
manage/tables/6?details=true
```

Example of Complex Table

The following is a working complex table definition using all parameters for user reference.

```

{
  "table_name": "UserProfile",
  "root_url": "user-profile",
  "delete": false,
  "enums": {"accountrole": ["ADMIN",
                           "USER",
                           "GUEST"]},

```

```

"comments": "This is the user profile table that keeps track of
            user profiles and data",
"constraints": {"unique": {"unique_first_name_last_name_pair":
                        ["first_name", "last_name"]}},
"columns": {
    "user_profile_id": {
        "data_type": "serial",
        "serial_start": 1,
        "serial_increment": 1,
        "primary_key": true
    },
    "username": {
        "unique": true,
        "data_type": "varchar",
        "char_len": 255
        "comments": "The username used by *** service"
    },
    "role": {
        "data_type": "accountrole"
    },
    "company": {
        "data_type": "varchar",
        "char_len": 255,
        "foreign_key": true,
        "reference_table": "Companys",
        "reference_column": "company_name",
        "on_event": "ON UPDATE",
        "event_action": "cascade"
    },
    "employee_id": {
        "data_type": "integer",
        "foreign_key": true,
        "reference_table": "Employees",
        "reference_column": "employee_id",
        "on_event": "on delete",
        "event_action": "RESTRICT"
    }
    "first_name": {
        "null": true,
        "data_type": "varchar",
        "char_len": 255
    },
    "last_name": {
        "null": true,
        "data_type": "varchar",
        "char_len": 255
    },
    "created_at": {
        "data_type": "timestamp",
        "default": "CREATETIME"
    },
    "last_updated_at": {
        "data_type": "timestamp",

```

```

    "default": "UPDATETIME"
  }
}

```

Editing tables

Tenant admins are able to modify tables with a few select operations by making a PUT request to `/v3/pgrest/manage/tables/{table_id}`. This feature is only available to admins, please contact a service admin in order to see if you qualify for the role.

Important

Support for editing tables for tenant admins was added in version 1.1.0.

This PUT endpoint effectively uses `ALTER TABLE` commands with postgres and also updates the Django database backend to match these changes. Due to the complexity in this workflow, admins are only allowed a fixed set of operations to use. Additionally, only one operation is allowed to take place per PUT request. For example, admins can not edit a table's name, and drop a column in the same request.

The endpoint's error messages are generally verbose, so errors should be pointed out poignantly. A list of all PUT operations available and examples is as follows:

- `root_url`
 - Operation to change the `root_url` currently associated with the table.
 - Ex: Change table with `root_url` `all_people` to `some_people`.
 - Payload is `{"root_url": "some_people"}`
- `table_name`
 - Operation to change the `table_name` currently associated with the table.
 - Ex: Change table with `table_name` `all_people` to `some_people`.
 - Payload is `{"table_name": "some_people"}`
- `comments`
 - Operation to change the comments currently associated with the table (Overwrites existing).
 - Ex: Adding "This column is not case-sensitive" comment to table with `table_id` 84.
 - Payload is `{"comments": "This column is not case-sensitive"}`
- `endpoints`
 - Operation to change the endpoints a table currently has available.
 - Endpoints available are "ALL" (alias to set all endpoints to available), "NONE" (alias to set no endpoints), "GET_ALL", "GET_ONE", "CREATE", "UPDATE", and "DELETE".
 - Ex: Changing table so all endpoints are available.
 - Payload is `{"endpoints": ["ALL"]}`
- `column_type`
 - Operation to change the `column_type` of a particular column in table.
 - `column_types` available are "varchar", "boolean", "integer", "text", "timestamp", "serial", and "datetime".
 - Ex: Changing column names from type integer to varchar.
 - Payload is `{"column_type": "names, varchar"}`

- `add_column`
 - Operation to add a column to a table.
 - Uses the same column definition formatting as when creating a table. All data types and parameters apply.
 - Ex: Adding column names.
 - Payload is `{"add_column": {"names": {"data_type": "varchar", "char_len": 255, "comments": "first name"}}`
- `drop_column`
 - Operation to drop a column in a table (Not reversible).
 - Ex: Dropping column `useless_archive_column`.
 - Payload is `{"drop_column": "useless_archive_column"}`
- `drop_default`
 - Operation to drop a default currently set on a column in a table.
 - Ex: Dropping default for column `i_no_longer_need_a_default`.
 - Payload is `{"drop_default": "i_no_longer_need_a_default"}`
- `set_default`
 - Operation to set a new default on a column in a table.
 - Ex: Setting default on column names to `no_name`.
 - Payload is `{"set_default": "names,no_name"}`

For example, to change the `table_name` of table with `table_id` 3 to `my_new_name`:

```
$ curl -H "tapis-v2-token: $tok" \
      -H "Content-type: application/json" \
      -d '{"table_name': 'my_new_name'}" \
      https://dev.tapis.io/v3/pgrest/manage/tables/3
```

For example, to change the `column_type` of column names from integer to `varchar` in the same table:

```
$ curl -H "tapis-v2-token: $tok" \
      -H "Content-type: application/json" \
      -d '{"column_type': 'names, varchar'}" \
      https://dev.tapis.io/v3/pgrest/manage/tables/3
```

For example, to change the endpoints of table 3 to only `CREATE` and `DELETE`:

```
$ curl -H "tapis-v2-token: $tok" \
      -H "Content-type: application/json" \
      -d '{"endpoints': '['CREATE', 'DELETE']\'}" \
      https://dev.tapis.io/v3/pgrest/manage/tables/3
```

All operations will result in either a verbose error message from the API regarding issues, the postgres error message returned to the API in cases of a postgres error, or a `Table put successfully` message.

Table User API

The Data API provides endpoints for managing and retrieving data (rows) stored on tables defined through the Management API. For each table defined through the Management API, there is a corresponding endpoint within the Data API with URL `/v3/pgrest/data/{root_url}`, where `{root_url}` is the associated attribute on the table.

Continuing with our widgets table from above, the associated endpoint within the Data API would have URL `/v3/pgrest/data/widgets` because the `root_url` property of the widgets table was defined to be `widgets`. Moreover, all 5 default endpoints on the widgets collection are available (none were explicitly restricted when registering the table). The endpoints within the widgets can be described as follows:

GET	POST	PUT	DELETE	Endpoint	Description
X	X	X		<code>/v3/pgrest/data/widgets</code>	List/create widgets; bulk update multiple widgets.
X		X	X	<code>/v3/pgrest/data/widgets/{id}</code>	Get/update/delete a widget by id.

Note that the `id` column is used for referencing a specific row. Currently, PgREST generates this column automatically for each table and calls it `{table_name}_id`. It is a sql serial data type. To override this generic `id` column, you may assign a key of your choice the `primary_key` constraint. We'll then use the values in this field to get a specified rows. `primary_key` columns, must be integers or varchars which are not null and unique.

Additionally, to find the `id` to use for your row, the data endpoints return a `_pkid` field in the results for each row for ease of use. `_pkid` is not currently kept in the database, but is added to the result object between retrieving the database result and returning the result to the user. As such, where queries will NOT work on the `_pkid` field.

Creating a Row

Sending a POST request to the `/v3/pgrest/data/{root_url}` URL will create a new row on the corresponding table. The POST message body should be a JSON document providing values for each of the columns inside a single data object. The values will first be validated with the json schema generated from the columns data sent in on table creation. This will enforce data types, max lengths, and required fields. The row is then added to the table using pure SQL format and is fully ATOMIC.

For example, the following JSON body could be used to create a new row on the widgets example table:

new_row.json:

```
{
  "data": {
    "name": "example-widget",
    "widget_type": "gear",
    "count": 0,
    "is_private": false
  }
}
```

The following curl command would create a row defined by the JSON document above

```
$ curl -H "tapis-v2-token: $TOKEN" -H "Content-type: application/json" -d "@new_row.json" https://<tenant>.tapis.io/v3/pgrest/data/widgets
```

if all goes well, the response should look like

```
{
  "status": "success",
  "message": "The request was successful.",
  "version": "dev",
  "result": [
    {
      "widgets_id": 1,
      "name": "example-widget",
      "widget_type": "gear",
      "count": 0,
      "is_private": false
    }
  ]
}
```

Note that an id of 1 was generated for the new record.

Creating multiple rows at once

The same as creating a single row, POST requests to the/v3/pgrest/data/{root_url} URL containing lists of row definitions will create rows in bulk. POST message body should be a JSON list contain JSON documents providing values for each of the columns inside a single data object. This works exactly the same way as single row creation, but with a list input. The rows are then added to the table using pure SQL format and is fully ATOMIC.

For example, the following JSON body could be used to create three rows on the widgets example table:

new_rows.json:

```
{
  "data": [
    {
      "name": "example-widget1",
      "widget_type": "gear1",
      "count": 0,
      "is_private": false
    },
    {
      "name": "example-widget2",
      "widget_type": "gear2",
      "count": 0,
      "is_private": true
    },
    {
      "name": "example-widget3",
      "widget_type": "gear3",
      "count": 0,
      "is_private": false
    }
  ]
}
```

The following curl command would create a row defined by the JSON document above

```
$ curl -H "tapis-v2-token: $TOKEN" -H "Content-type: application/json" -d "@new_rows.json" https://<tenant>.tapis.io/v3/pgrest/data/widgets
```

if all goes well, the response should return all the rows just created.

Updating a Row

Sending a PUT request to the `/v3/pgrest/data/{root_url}/{id}` URL will update an existing row on the corresponding table. The request message body should be a JSON document providing the columns to be updates and the new values. For example, the following would update the `example-widget` created above:

update_row.json

```
{
  "data": {
    "count": 1
  }
}
```

The following curl command would update the `example-widget` row (with id of `i`) using the JSON document above


```
$ curl -H "tapis-v2-token: $TOKEN" -H "Content-type: application/json" -d "@update_row.json" https://<tenant>.tapis.io/v3/pgrest/data/widgets/1
```

Note that since only the count field is provided in the PUT request body, that is the only column that will be modified.

Updating Multiple Rows

Update multiple rows with a single HTTP request is possible using a where filter (for more details, see the section [Where Stanzas](#) below), provided in the PUT request body. For example, we could update the count column on all rows with a negative count to 0 using the following

update_rows.json

```
{
  "count": 0,
  "where": {
    "count": {
      "operator": "<",
      "value": 0
    }
  }
}
```

This update_rows.json would be used in a PUT request to the root widgets collection, as follows:

```
$ curl -H "tapis-v2-token: $TOKEN" -H "Content-type: application/json" -d "@update_rows.json" https://<tenant>.tapis.io/v3/pgrest/data/widgets
```

Where Stanzas

In PgREST, where stanzas are used in various endpoints throughout the API to filter the collection of results (i.e. rows) that an action (such as retrieving or updating) is applied to. The where stanza should be a JSON object with each key being the name of a column on the table and the value under each key being a JSON object with two properties:

- operator -- a valid operator for the comparison. See the [Valid Operators](#) table below.
- value -- the value to compare the row's column to (using the operator).

Naturally, the type (string, integer, boolean, etc.) of the value property should correspond to the type of the column specified by the key. Note that multiple keys corresponding to the same column or different columns can be included in a single where stanza. For example, the following where stanza would pick out rows whose count was between 0 and 100 and whose is_private property was true:

```

{
  "where": {
    "count": {
      "operator": ">",
      "value": 0
    },
    "count": {
      "operator": "<",
      "value": 100
    },
    "is_private": {
      "operator": "=",
      "value": true
    }
  }
}

```

Valid Operators

PgREST recognizes the following operators for use in where stanzas.

Operator	Postgres Equivalent	Description
<	<	Less than
>	>	Greater than
=	=	Equal
...	...	...

Todo... Full table coming soon

Retrieving Rows

To retrieve data from the /data API, make an HTTP GET request to the associated URL; an HTTP GET to /v3/pgrest/data/{root_url} will retrieve all rows on the associated table, while an HTTP GET to /v3/pgrest/data/{root_url}/{id} will retrieve the individual row.

For example,

```
$ curl -H "tapis-v2-token: $TOKEN" https://dev.tapis.io/v3/pgrest/data/init
```

retrieves all rows of the table "init":

```
[
  {
    "_pkid": 1,
    "initial_table_id": 1,
    "col_one": "col 1 value",
    "col_two": 3,
    "col_three": 8,
    "col_four": false,
    "col_five": null
  },
  {
    "_pkid": 2,
    "initial_table_id": 2,
    "col_one": "val",
    "col_two": 5,
    "col_three": 9,
    "col_four": true,
    "col_five": "hi there"
  },
  {
    "_pkid": 3,
    "initial_table_id": 3,
    "col_one": "value",
    "col_two": 7,
    "col_three": 9,
    "col_four": true,
    "col_five": "hi there again"
  }
]
```

while the following curl:

```
$ curl -H "tapis-v2-token: $TOKEN" https://dev.tapis.io/v3/pgrest/
data/init/3
```

retrieves just the row with id "3":

```
{
  "_pkid": 3,
  "initial_table_id": 3,
  "col_one": "value",
  "col_two": 7,
  "col_three": 9,
  "col_four": true,
  "col_five": "hi there again"
}
```

Retrieving rows with search parameters

PgREST allows users to retrieve table rows from either the /data endpoint or the /view endpoint using search parameters follow the Tapis V3 search specification. All search operations available and examples are detailed below.

Important

Support of retrieving rows with search parameters is available in version 1.1.0. Previous implementation disregarded due to being out of date with Tapis V3 search specifications.

Important

Two additional query parameters are limit and offset. Used like ?limit=20 or/and ?offset=5. This caps the amount of results you get back or gives you results past a set number, respectively.

Operator	Postgres Equivalent	Description
.eq	=	Equal
.neq	!=	Not equal
.lt	<	Less than
.lte	<=	Less than or equal
.gt	>	Greater than
.gte	>=	Greater than or equal
.in	IN	In set of
.nin	NOT IN	Not in set of
.like	LIKE	Like value
.nlike	NOT LIKE	Not like value
.between	BETWEEN	Between set
.nbetween	NOT BETWEEN	Not between set
.null	IS NULL	Set to TRUE/FALSE

These operators are used in an endpoint's query parameters. For example if I had a table with root_url my_table and I wanted to get all rows with column age greater than 15 I could do the following:

```
$ curl -H "tapis-v2-token: $TOKEN" \
  https://tapis.io/v3/pgrest/data/my_table?age.gt=15
```

Another example using .between this time would be finding all rows with column age between 18-30:

```
$ curl -H "tapis-v2-token: $TOKEN" \
  https://tapis.io/v3/pgrest/data/my_table?age.between=18,30
```

If you only wanted all rows with column age that match 20, 30, or 40:

```
$ curl -H "tapis-v2-token: $TOKEN" \
https://tapis.io/v3/pgrest/data/my_table?age.in=20,30,40
```

Views Manage API

Views allow admins to create postgres views to cordone off data from users and give users exactly what they need. These views allow for `permission_rules` which cross reference a users roles, if a user has all roles in the `permission_rules` they have access to view the view.

Admins are able to create views with a post to the `/v3/pgrest/manage/views` endpoint. A get to `/v3/pgrest/manage/views` returns information regarding the views.

View Definition Rules

A post to the `/v3/pgrest/manage/views` endpoint to create a view expects a json formatted view definition. Each view definition can have the following rules.

- **view_name - required**
 - The name of the view in question.
- **select_query - required**
 - Query to select from the table specified with `from_table`
- **from_table - required**
 - Table to read data from
- **root_url**
 - The `root_url` for PgRESTs `/views` endpoint.
 - Ex: `root_url "view25"` would be accessible via `"http://pgrestURL/views/table25"`.
- **where_query**
 - Optional field that allows you to specify a postgres where clause for the view
- **comments**
 - Field to allow for better readability of view json. Table comments are saved and outputted on `/v3/pgrest/manage/views/` endpoints.
- **permission_rules**
 - List of roles required to view this view.
 - If nothing is given, view is open to all.
- **raw_sql - admins only**
 - To allow for better use of postgres's facilities there is a `raw_sql` view creation parameter.
 - To use this parameter you must be an admin (talk to service admins)
 - When using this paramter, `select_query`, `where_query`, and `from_table` are no longer allowed, other parameters are fine.
 - The query follows `CREATE OR REPLACE VIEW {tenant}.{view_name} {raw_sql}` format.

- Example data body:

```
{"view_name": "my_new_test_view",  
  "raw_sql": "AS SELECT * FROM tenant.my_table WHERE  
              col_name >= 600;",  
  "comments": "An example of creating my_new_test_view."}
```

- **materialized_view_raw_sql - admins only**

- To allow for better use of postgres's facilities there is a `materialized_view_raw_sql` view creation parameter.
- To use this parameter you must be an admin (talk to service admins)
- When using this parameter, `select_query`, `raw_sql`, `where_query`, and `from_table` are no longer allowed, other parameters are fine.
- The query follows `CREATE MATERIALIZED VIEW {tenant}.{view_name} {materialized_view_raw_sql}` format.
- Refresh the materialized view data by GET `https://tenant.tapis.io/v3/pgrest/manage/views/{view_id}/refresh`. Detailed later in these docs.
- Data is only updated at creation and during manual refreshes from users.
- Example data body:

```
{"view_name": "my_new_materialized_test_view",  
  "materialized_view_raw_sql": "AS SELECT * FROM  
                                tenant.my_table WHERE col_name >= 600;",  
  "comments": "An example of creating  
              my_new_materialized_test_view."}
```

View Creation Example

```
# new_view.json  
{  
  'view_name': 'test_view',  
  'root_url': 'just_a_cool_url',  
  'select_query': '*',  
  'from_table': 'initial_table_2',  
  'where_query': 'col_one >= 90',  
  'permission_rules': ['lab_6_admin', 'cii_rep'],  
  'comments': 'This is a cool test_view to view all of  
              initial_table_2. Only users with the  
              lab_6_admin and cii_rep role can view this.'  
}
```

The following code block would then be able to create the new view.

```
$ curl -H "tapis-v2-token: $TOKEN" -H "Content-type: application/  
      json" \  
      -d "@new_view.json" https://<tenant>.tapis.io/v3/pgrest/manage/  
      views
```

If you then wanted to get information about the view, but not the result of the view itself, you can make a call to the `/v3/pgrest/manage/views/just_a_cool_url` endpoint.

```
$ curl -H "tapis-v2-token: $TOKEN" \
  https://<tenant>.tapis.io/v3/pgrest/manage/views/just_a_cool_url
```

Materialized View Refresh Example

If you had earlier created a view using the `materialized_view_raw_sql` attribute, you'd have a materialized view that you can refresh the data of manually. Data for materialized views is updated at creation, and at refreshes, there is not other automatic refreshing of the data.

```
$ curl -H "tapis-v2-token: $TOKEN" https://<tenant>.tapis.io/v3/
pgrest/manage/views/{view_id}/refresh
```

Views User API

Users have no way to change views or to modify anything dealing with them, but they are able to get views that they have sufficient permissions to view. The `/v3/pgrest/views/{view_id}` requires the user to have at least a `PGREST_USER` role, this is to force all users to be in some way identified or managed. The `PGREST_USER` role grants no permissions except the permission to call a get on the `/v3/pgrest/views/{view_id}` endpoint. This means that admins can assume `PGREST_USER` users can only view views in which the user satisfies the permissions declared in a views `permission_rules`. This gives admins fine-tuned controls on postgres data by using solely views.

Users can make a get to `/v3/pgrest/views/{view_id}` with the following curl.

```
$ curl -H "tapis-v2-token: $TOKEN" \
  https://<tenant>.tapis.io/v3/pgrest/views/just_a_cool_url
```

Retrieving rows with search parameters

PgREST allows users to retrieve table rows from either the `/data` endpoint or the `/view` endpoint using search parameters follow the Tapis V3 search specification. All search operations available and examples are detailed below.

Important

Support of retrieving rows with search parameters is available in version 1.1.0. Previous implementation disregarded due to being out of date with Tapis V3 search specifications.

Important

Two additional query parameters are limit and offset. Used like ?limit=20 or/and ?offset=5. This caps the amount of results you get back or gives you results past a set number, respectively.

Operator	Postgres Equivalent	Description
.eq	=	Equal
.neq	!=	Not equal
.lt	<	Less than
.lte	<=	Less than or equal
.gt	>	Greater than
.gte	>=	Greater than or equal
.in	IN	In set of
.nin	NOT IN	Not in set of
.like	LIKE	Like value
.nlike	NOT LIKE	Not like value
.between	BETWEEN	Between set
.nbetween	NOT BETWEEN	Not between set
.null	IS NULL	Set to TRUE/FALSE

These operators are used in an endpoint's query parameters. For example if I had a view with root_url my_view and I wanted to get all rows with column age greater than 15 I could do the following:

```
$ curl -H "tapis-v2-token: $TOKEN" \
  https://tapis.io/v3/pgrest/views/my_view?age.gt=15
```

Another example using .between this time would be finding all rows with column age between 18–30:

```
$ curl -H "tapis-v2-token: $TOKEN" \
  https://tapis.io/v3/pgrest/views/my_view?age.between=18,30
```

If you only wanted all rows with column age that match 20, 30, or 40:

```
$ curl -H "tapis-v2-token: $TOKEN" \
  https://tapis.io/v3/pgrest/views/my_view?age.in=20,30,40
```

Role Manage API

Role management is solely allowed for users in the PGREST_ROLE_ADMIN role, or PGREST_ADMIN. These endpoints allow users to create, grant, and revoke SK roles to users to match view permission_rules. Modifiable roles all must start with PGREST_, this ensures users can't change roles that might matter to other services. Along with that, users cannot manage the PGREST_ADMIN, PGREST_ROLE_ADMIN, PGREST_WRITE, or PGREST_READ roles. This might be

changed, but for now, contact an admin to be given these roles. Note, users can grant or revoke the PGREST_USER role, so that role_admins can manage who can see views without unnecessary intervention.

Users have no access to these endpoints or anything regarding roles.

Role Creation

To create a new role users make a post to /v3/pgrest/roles/ with a body such as:

```
# new_role.json
{"role_name": "PGREST_My_New_Role", "description": "A new role!"}
```

The post would then be as follows:

```
$ curl -H "tapis-v2-token: $TOKEN" -H "Content-type: application/
      json" \
      -d "@new_role.json" https://<tenant>.tapis.io/v3/pgrest/roles
```

This would result in the creation of the PGREST_My_New_Role role.

Role Creation Definition

- **role_name - required**
 - Name of the role to create, must start with PGREST_.
- **description - required**
 - A description of the role to be used for future reference in SK.

Role Management

To grant or revoke a role to a specific username, users can make a post to /v3/pgrest/roles/{role_name} with a body such as:

```
# grant_role.json
{"method": "grant", "username": "user3234"}
```

The post would then be as follows:

```
$ curl -H "tapis-v2-token: $TOKEN" -H "Content-type: application/
      json" \
      -d "@grant_role.json" https://<tenant>.tapis.io/v3/pgrest/roles/
      PGREST_My_New_Role
```

This would result in the user, user3234 being granted the role, PGREST_My_New_Role.

Role Management Parameters

- **method - required**
 - String of either "grant" or "revoke", specifying whether to revoke or grant the role to a user.

- username - **required**
 - Username to grant role to, or to revoke role from.

Stored Procedures

Coming soon

API Reference

GET	POST	PUT	DEL	Endpoint	Description
X	X			/v3/pgrest/ manage/tables	Get/Create tables for the tenant.
X		X	X	/v3/pgrest/ manage/tables/ {table_id}	Get/Update/ Delete a specified table.
X	X	X		/v3/pgrest/ data/{table_id}	Get/Create/ Update rows for a specified table.
X		X	X	/v3/pgrest/ data/ {table_id}/ {row_id}	Get/Update/ Delete specific row for a specified table.
X	X			/v3/pgrest/ manage/views	Get/Create view for the tenant.
X			X	/v3/pgrest/ manage/views/ {view_id}	Get/Delete view specified.
X				/v3/pgrest/ views/ {view_id}	Get results from view specified.
X	X			/v3/pgrest/ manage/roles	Get/Create roles in SK for the tenant.
X	X			/v3/pgrest/ manage/roles/ {role_name}	Get/Manage role specified.